

Dokumentacja: Python

Spis treści

- 1. Podstawy języka Python
 - 1.1. Typy zmiennych
 - 1.2. Konwersja typów (rzutowanie)
 - 1.3. Wyświetlanie danych — `print()`
 - 1.4. Formatowanie napisów — f-string
 - 1.5. Pobieranie danych od użytkownika — `input()`
 - 1.6. Komentarze
 - 1.7. Znak nowej linii i znaki specjalne
 - 1.8. Obsługa wyjątków — try / except / finally
 - 1.9. Wczytywanie wielu wartości w jednej linii
- 2. Operatory
 - 2.1. Operatory arytmetyczne
 - 2.2. Operatory porównania
 - 2.3. Operatory logiczne
 - 2.4. Operatory przypisania
 - 2.5. Operatory przynależności i tożsamości
 - 2.6. Priorytet operatorów
- 3. Instrukcje sterujące
 - 3.1. Instrukcja warunkowa `if` / `elif` / `else`
 - 3.2. Instrukcja `match` / `case` — wzorce (switch)
 - 3.3. Pętla `for`
 - 3.4. Funkcja `enumerate()` — iteracja z indeksem
 - 3.5. Pętla `while`
 - 3.6. `break`, `continue` i `else` w pętlach
 - 3.7. Pętle zagnieżdżone
- 4. Struktury danych
 - 4.1. Listy (`list`)
 - 4.2. Krotki (`tuple`)
 - 4.3. Słowniki (`dict`)
 - 4.4. Zbiory (`set`)
 - 4.5. Zagnieżdżone struktury danych
- 5. Funkcje
 - 5.1. Definicja i wywoływanie funkcji
 - 5.2. Parametry domyślne
 - 5.3. Argumenty pozycyjne i nazwane
 - 5.4. Zmienna liczba argumentów — `*args` i `**kwargs`
 - 5.5. Argumenty tylko pozycyjne (/) i tylko nazwane (*)
 - 5.6. Zasięg zmiennych (scope)
 - 5.7. Funkcja `lambda` — funkcje anonimowe
 - 5.8. Funkcje wbudowane — przegląd najważniejszych
- 6. Przetwarzanie łańcuchów znaków
 - 6.1. Podstawowe operacje na napisach
 - 6.2. Metody napisów — kompletny przegląd
 - 6.3. Sprawdzanie rodzaju znaków
 - 6.4. Kody znaków — `ord()` i `chr()`

- 6.5. Iteracja po znakach i budowanie nowego napisu
- 6.6. Obsługa polskich znaków
- 6.7. Przydatne wzorce przetwarzania tekstu
- 6.8. Usuwanie powtarzających się znaków
- 7. Obsługa plików
 - 7.1. Otwieranie i zamykanie plików
 - 7.2. Odczyt pliku tekstowego
 - 7.3. Zapis do pliku tekstowego
 - 7.4. Praca z plikami CSV (dane tabelaryczne)
 - 7.5. Sprawdzanie istnienia pliku
 - 7.6. Obsługa błędów przy pracy z plikami
 - 7.7. Operacje na ścieżkach — moduł `os.path`
 - 7.8. Tworzenie i usuwanie plików/folderów
 - 7.9. Wczytywanie danych z pliku do tablicy
 - 7.10. Praca z plikami JSON
 - 7.11. Podsumowanie metod pracy z plikami
- 8. Operacje matematyczne
 - 8.1. Wbudowane funkcje matematyczne
 - 8.2. Moduł `math`
 - 8.3. Dzielenie — porównanie operatorów
 - 8.4. Konwersja systemów liczbowych
- 9. Moduł random — generowanie losowych danych
 - 9.1. Importowanie
 - 9.2. Generowanie liczb losowych
 - 9.3. Losowanie z kolekcji
 - 9.4. Wypełnianie tablicy losowymi danymi
 - 9.5. Losowanie bez powtórzeń — trzy sposoby
 - 9.6. Ustawienie ziarna (seed) — powtarzalność wyników
- 10. Programowanie obiektowe — podstawy
 - 10.1. Definicja klasy
 - 10.2. Konstruktor `__init__` i pola instancji
 - 10.3. Metody instancji
 - 10.4. `self` — czym jest i dlaczego jest konieczne
 - 10.5. Metoda `__str__` — reprezentacja tekstowa obiektu
 - 10.6. `__repr__` vs `__str__` — dwie reprezentacje tekstowe
- 11. Programowanie obiektowe — hermetyzacja
 - 11.1. Poziomy dostęp w Pythonie
 - 11.2. Gettery i settery — metody dostępu
 - 11.3. Dekorator `@property` — pythoniczny getter/setter
- 12. Programowanie obiektowe — zmienne klasowe i metody statyczne
 - 12.1. Zmienne klasowe (static fields)
 - 12.2. Zmienne klasowe — tablice i kolekcje
 - 12.3. Metody statyczne — `@staticmethod`
 - 12.4. Metody klasowe — `@classmethod`
- 13. Programowanie obiektowe — konstruktory zaawansowane
 - 13.1. Konstruktor z wartościami domyślnymi
 - 13.2. Wzorzec z `None` — pełny zamiennik dwóch konstruktorów
 - 13.3. Walidacja w konstruktorze
 - 13.4. Kopiowanie obiektów
- 14. Programowanie obiektowe — dziedziczenie, polimorfizm i klasy abstrakcyjne

- 14.1. Podstawy dziedziczenia
- 14.2. `super()` — wywoływanie metod rodzica
- 14.3. Hermetyzacja a dziedziczenie
- 14.4. Wzorzec: logika stanów z dziedziczeniem
- 14.5. Sprawdzanie typu i dziedziczenia
- 14.6. Podsumowanie — kiedy co stosować w OOP
- 14.7. Polimorfizm
- 14.8. Klasy abstrakcyjne — moduł `abc`
- 14.9. Kompleksowa implementacja: System Quiz
- 15. Algorytmy w Pythonie
 - 15.1. Sortowanie i porządkowanie danych
 - 15.1.1. Sortowanie przez wybieranie (Selection Sort)
 - 15.1.2. Sortowanie bąbelkowe (Bubble Sort)
 - 15.1.3. Sortowanie przez wstawianie (Insertion Sort)
 - 15.1.4. Sortowanie przez scalanie (Merge Sort)
 - 15.1.5. Sortowanie szybkie (Quick Sort)
 - 15.1.6. Sortowanie malejące własnym algorytmem
 - 15.1.7. Liczenie liczby zamian w Bubble Sort
 - 15.1.8. Sprawdzanie, czy tablica jest posortowana
 - 15.1.9. Scalanie dwóch posortowanych tablic
 - 15.1.10. Podział tablicy względem pivota
 - 15.2. Wyszukiwanie i analiza list
 - 15.2.1. Wyszukiwanie liniowe
 - 15.2.2. Wyszukiwanie z wartownikiem (Sentinel Search)
 - 15.2.3. Wyszukiwanie binarne (Binary Search)
 - 15.2.4. Znajdowanie minimum, maksimum i ich indeksów
 - 15.2.5. Drugie największe i drugie najmniejsze
 - 15.2.6. Wyszukiwanie pierwszego i ostatniego wystąpienia wartości
 - 15.2.7. Wyszukiwanie binarne z liczeniem kroków
 - 15.2.8. Znajdowanie brakującej liczby z zakresu 1..n
 - 15.2.9. Znajdowanie elementów wspólnych dwóch list
 - 15.2.10. Znajdowanie elementów występujących tylko raz
 - 15.2.11. Usuwanie duplikatów z listy z zachowaniem kolejności
 - 15.2.12. Zliczanie wystąpień wartości w danych
 - 15.2.13. Obliczanie średniej arytmetycznej
 - 15.2.14. Wyodrębnianie elementów spełniających warunek
 - 15.3. Działania matematyczne i dzielniki
 - 15.3.1. Algorytm Euklidesa (NWD)
 - 15.3.2. Najmniejsza Wspólna Wielokrotność (NWW)
 - 15.3.3. NWD dla wielu liczb
 - 15.3.4. NWW dla wielu liczb
 - 15.3.5. Liczby względnie pierwsze
 - 15.3.6. Suma dzielników liczby
 - 15.3.7. Liczba doskonała
 - 15.3.8. Liczby zaprzyjaźnione
 - 15.3.9. Skracanie ułamka
 - 15.3.10. Dodawanie dwóch ułamków
 - 15.4. Liczby pierwsze, cyfry i cechy liczb
 - 15.4.1. Sprawdzanie czy liczba jest pierwsza
 - 15.4.2. Sito Eratostenesa

- 15.4.3. Rozkład liczby na czynniki pierwsze
- 15.4.4. Rozkład liczby na czynniki pierwsze z potęgami
- 15.4.5. Najbliższa liczba pierwsza większa od n
- 15.4.6. Liczenie liczb pierwszych w przedziale
- 15.4.7. Liczba Armstronga
- 15.4.8. Statystyka cyfr liczby
- 15.4.9. Odwracanie cyfr liczby bez zamiany na napis
- 15.5. Systemy liczbowe i operacje binarne
 - 15.5.1. Konwersja systemów liczbowych (dziesiętny $\leftrightarrow$ binarny)
 - 15.5.2. Konwersja dziesiętna na dowolny system od 2 do 16
 - 15.5.3. Konwersja z dowolnego systemu od 2 do 16 na dziesiętny
 - 15.5.4. Dodawanie dwóch liczb binarnych zapisanych jako napisy
- 15.6. Teksty, słowniki i zliczanie
 - 15.6.1. Palindromy i anagramy
 - 15.6.2. Zliczanie liter w tekście
 - 15.6.3. Najczęściej występujący znak
 - 15.6.4. Najczęściej występujące słowo w zdaniu
 - 15.6.5. Usuwanie wszystkich znaków niebędących literami
 - 15.6.6. Sprawdzanie pangramu
 - 15.6.7. Sprawdzanie poprawności nawiasów
 - 15.6.8. Usuwanie powtarzających się znaków sąsiednich
 - 15.6.9. Usuwanie sąsiednich duplikatów aż do skutku
 - 15.6.10. Najdłuższy wspólny prefiks listy słów
 - 15.6.11. Sprawdzanie, czy dwa teksty są izomorficzne
- 15.7. Szyfry i kodowanie tekstu
 - 15.7.1. Szyfr Cezara
 - 15.7.2. Odszyfrowanie szyfru Cezara
 - 15.7.3. Łamanie szyfru Cezara metodą brute force
 - 15.7.4. Szyfr Vigenere'a
- 15.8. Rekurencja i dziel i zwyciężaj
 - 15.8.1. Silnia i Fibonacciego rekurencyjnie
 - 15.8.2. Generowanie ciągu Fibonacciego iteracyjnie
 - 15.8.3. Szybkie potęgowanie
 - 15.8.4. Wieże Hanoi
 - 15.8.5. Suma elementów tablicy rekurencyjnie
- 15.9. Tablice dwuwymiarowe
 - 15.9.1. Przeszukiwanie tablicy 2D
 - 15.9.2. Suma przekątnych macierzy kwadratowej
 - 15.9.3. Obrót macierzy o 90 stopni w prawo
- 15.10. Walidacja danych i zadania plikowe
 - 15.10.1. Walidacja danych z wagami (np. PESEL)
 - 15.10.2. Prosty system punktacji z pliku CSV
 - 15.10.3. Zapis rankingu do pliku CSV
- 16. Tablice dwuwymiarowe
 - 16.1. Tworzenie tablic 2D
 - 16.2. Dostęp i iteracja
 - 16.3. Wypełnianie i wyświetlanie
 - 16.4. Operacje na tablicach 2D
- 17. Testowanie jednostkowe — unittest i pytest
 - 17.1. Czym jest testowanie jednostkowe

- 17.2. Moduł `unittest` — podstawy
- 17.3. Asercje — kompletna lista
- 17.4. Struktura projektu z testami
- 17.5. Nazewnictwo testów
- 17.6. `setUp` i `tearDown` — przygotowanie i sprzątanie
- 17.7. Testowanie klas i obiektów
- 17.8. Testowanie wartości losowych
- 17.9. `pytest` — prostsza alternatywa
- 17.10. Podsumowanie — uruchamianie testów
- 18. Przydatne wzorce i idiomy Pythona
 - 18.1. Zamiana wartości dwóch zmiennych
 - 18.2. `if __name__ == "__main__":`
 - 18.3. Rozpakowywanie z `*` (operatorem gwiazdki)
 - 18.4. Dictionary comprehension
 - 18.5. Formatowanie wypisywania tabel w konsoli
 - 18.6. Podkreślnik `_` — konwencje
 - 18.7. Najczęstsze pułapki i błędy w Pythonie
 - 18.8. Wzorzec menu programu (pętla główna)
 - 18.9. Sortowanie obiektów — parametr `key`
 - 18.10. Generatory i `yield`
- 19. Najczęstsze błędy (Wyjątki i błędy konsolowe)
 - 19.1. `SyntaxError`
 - 19.2. `IndentationError`
 - 19.3. `NameError`
 - 19.4. `TypeError`
 - 19.5. `ValueError`
 - 19.6. `IndexError`
 - 19.7. `KeyError`
 - 19.8. `AttributeError`
 - 19.9. `ImportError`
 - 19.10. `ModuleNotFoundError`
 - 19.11. `ZeroDivisionError`
 - 19.12. `FileNotFoundError`
 - 19.13. `RecursionError`
 - 19.14. `StopIteration`
 - 19.15. `UnboundLocalError`
 - 19.16. `NotImplementedError`
 - 19.17. `TabError`
 - 19.18. `PermissionError`
 - 19.19. `OverflowError`
 - 19.20. `UnicodeDecodeError`

Jak korzystać z tego poradnika: Każdy rozdział jest samodzielny — możesz czytać je w dowolnej kolejności. Następujące po sobie rozdziały OOP (10–14) warto jednak czytać po kolei, ponieważ każdy z nich buduje na wiedzy z poprzedniego.

1. Podstawy języka Python

Python jest językiem programowania wysokiego poziomu, stworzonym przez Guido van Rossuma w 1991 roku. Wyróżnia się prostą i czytelną składnią, której fundamentem jest **wcięciowanie** (indentacja) — nie używa się klamerek `{ }` do oznaczania bloków kodu, lecz spacji lub tabulatorów. Standardowo stosuje się **4 spacje** na każdy poziom wcięcia.

Python jest językiem **interpretowanym** — kod nie jest kompilowany do postaci binarnej, lecz wykonywany linia po linii przez interpreter. Jest też **dynamicznie typowany**, co oznacza, że typ zmiennej jest ustalany automatycznie w momencie przypisania wartości, a nie deklarowany z góry przez programistę.

1.1. Typy zmiennych

W Pythonie nie trzeba deklarować typu zmiennej — interpreter sam rozpoznaje typ na podstawie przypisanej wartości. To fundamentalna różnica w porównaniu z językami takimi jak C++ czy Java, gdzie typ musi być podany jawnie (np. `int x = 5;`).

Poniższa tabela przedstawia podstawowe typy danych w Pythonie:

TYP	OPIS	PRZYKŁADOWE WARTOŚCI
<code>int</code>	Liczba całkowita (dowolnej wielkości)	<code>42</code> , <code>-7</code> , <code>0</code> , <code>1000000</code>
<code>float</code>	Liczba zmiennoprzecinkowa	<code>3.14</code> , <code>-0.5</code> , <code>2.0</code>
<code>str</code>	Łańcuch znaków (napis, tekst)	<code>"Witaj"</code> , <code>'Python'</code> , <code>""</code>
<code>bool</code>	Wartość logiczna (prawda/fałsz)	<code>True</code> , <code>False</code>
<code>None</code>	Wartość pusta (brak wartości)	<code>None</code>
<code>list</code>	Lista (modyfikowalna kolekcja)	<code>[1, 2, 3]</code> , <code>["a", "b"]</code>
<code>tuple</code>	Krotka (niemodyfikowalna kolekcja)	<code>(1, 2, 3)</code>
<code>dict</code>	Słownik (pary klucz-wartość)	<code>{"imie": "Jan"}</code>
<code>set</code>	Zbiór (unikalne elementy)	<code>{1, 2, 3}</code>

Przykłady deklaracji zmiennych:

```
# Liczby całkowite (int)
wiek = 25
liczba_elementow = 100
temperatura = -5

# Liczby zmiennoprzecinkowe (float)
temperatura = 36.6
srednia = 78.5
pi = 3.14159

# Łańcuchy znaków (str)
# Można używać zarówno cudzysłowów podwójnych, jak i pojedynczych
imie = "Jan"
komunikat = 'Witaj w programie'
pusty_napis = ""

# Wartości logiczne (bool)
# Uwaga: True i False są pisane z wielkiej litery!
jest_pelnoletni = True
czy_aktywny = False
```

```
# Wartość pusta (None)

# None oznacza "brak wartości" – używany np. jako domyślna wartość parametru

wynik = None
```

Sprawdzanie typu zmiennej:

Funkcja wbudowana `type()` zwraca typ obiektu. Jest przydatna podczas debugowania, gdy chcemy upewnić się, jakiego typu jest dana zmienna. Z kolei `isinstance()` pozwala sprawdzić, czy obiekt jest instancją określonego typu — ta funkcja uwzględnia również dziedziczenie.

```
x = 42
print(type(x))      # <class 'int'>

y = "tekst"
print(type(y))      # <class 'str'>

z = 3.14
print(type(z))      # <class 'float'>

w = True
print(type(w))      # <class 'bool'>
```

Funkcja `isinstance()` jest szczególnie przydatna, gdy chcemy sprawdzić, czy zmienna należy do jednego z kilku typów jednocześnie:

```
x = 42

if isinstance(x, int):
    print("x jest liczbą całkowitą")

# Sprawdzenie przynależności do wielu typów naraz
if isinstance(x, (int, float)):
    print("x jest liczbą (całkowitą lub zmiennoprzecinkową)")
```

1.2. Konwersja typów (rzutowanie)

Konwersja typów (ang. *type casting*) polega na zamianie jednego typu danych na inny. Jest to operacja niezwykle częsta — na przykład funkcja `input()` zawsze zwraca tekst `str`, więc aby wykonywać operacje arytmetyczne na danych pobranych od użytkownika, trzeba je zamienić na `int` lub `float`.

FUNKCJA	KONWERTUJE NA	PRZYKŁAD	WYNIK
<code>int()</code>	liczbę całkowitą	<code>int("42")</code>	42
<code>float()</code>	liczbę zmiennoprzecinkową	<code>float("3.14")</code>	3.14
<code>str()</code>	łańcuch znaków	<code>str(42)</code>	"42"
<code>bool()</code>	wartość logiczną	<code>bool(0)</code>	False
<code>list()</code>	listę	<code>list("abc")</code>	["a", "b", "c"]

```
# String → int – najczęstsza konwersja (dane z input())

tekst = "42"
```

```

liczba = int(tekst)          # 42

# String → float
tekst = "3.14"
wartosc = float(tekst)      # 3.14

# Int → string – potrzebne przy łączeniu z innymi stringami
liczba = 42
tekst = str(liczba)         # "42"

# Float → int – UWAGA: obcina część dziesiętną, nie zaokrągla!
wartosc = 3.7
calkowita = int(wartosc)    # 3 (nie 4!)
wartosc2 = 3.2
calkowita2 = int(wartosc2)  # 3

```

Konwersja na `bool` — zasada „truthiness”:

W Pythonie każda wartość ma swoją „prawdziwość”. Zrozumienie, co jest traktowane jako `True`, a co jako `False`, jest kluczowe przy pisaniu warunków:

WARTOŚĆ	<code>bool()</code>	WYJAŚNIENIE
<code>0</code>	<code>False</code>	Zero liczbowe
<code>0.0</code>	<code>False</code>	Zero zmiennoprzecinkowe
<code>""</code>	<code>False</code>	Pusty napis
<code>[]</code>	<code>False</code>	Pusta lista
<code>{}</code>	<code>False</code>	Pusty słownik
<code>None</code>	<code>False</code>	Brak wartości
<code>1</code> , <code>-5</code>	<code>True</code>	Każda niezerowa liczba
<code>"abc"</code>	<code>True</code>	Niepusty napis
<code>[1, 2]</code>	<code>True</code>	Niepusta lista

```

print(bool(0))      # False
print(bool(1))      # True
print(bool(-5))     # True – każda liczba ≠ 0 to True
print(bool(""))     # False – pusty napis
print(bool("abc"))  # True – niepusty napis
print(bool(None))   # False
print(bool([]))     # False – pusta lista
print(bool([0]))    # True – lista z jednym elementem (nawet zerem!)

```

1.3. Wyświetlanie danych — `print()`

Funkcja `print()` jest podstawowym narzędziem do wyświetlania danych w konsoli. Domyślnie dodaje znak nowej linii (`\n`) na końcu każdego wywołania, ale to zachowanie można zmienić.

Parametry funkcji `print()`:

PARAMETR	DOMYŚLNIE	OPIS
<code>*args</code>	—	Dowolna liczba wartości do wyświetlenia
<code>sep</code>	<code>" "</code> (spacja)	Separator między wartościami
<code>end</code>	<code>"\n"</code> (nowa linia)	Co dodać na końcu

```
# Podstawowe wyświetlanie
print("Witaj!")

# Wyświetlanie wielu wartości – automatycznie rozdzielone spacją
imie = "Anna"
wiek = 20
print("Imię:", imie, "Wiek:", wiek)
# Wynik: Imię: Anna Wiek: 20

# Zmiana separatora – parametr sep
print("A", "B", "C")           # A B C
print("A", "B", "C", sep=", ") # A, B, C
print("A", "B", "C", sep=" | ") # A | B | C
print("A", "B", "C", sep="")   # ABC (bez separatora)

# Zmiana zakończenia linii – parametr end
print("Bez nowej linii", end=" ")
print("kontynuacja")
# Wynik: Bez nowej linii kontynuacja

# Wyświetlanie pustej linii
print()
```

1.4. Formatowanie napisów — f-string

F-stringi (formatted string literals) to najwygodniejszy i najczęściej stosowany sposób formatowania tekstu w Pythonie 3.6+. Tworzymy je, dodając literę `f` przed cudzysłowem, a zmienne lub wyrażenia umieszczamy w klamrach `{ }`.

```
imie = "Jan"
wiek = 25
srednia = 87.4567

# Podstawowe wstawianie zmiennych
print(f"Cześć, {imie}!")           # Cześć, Jan!
print(f"Mam {wiek} lat")           # Mam 25 lat

# Wyrażenia wewnątrz klamr – mogą to być obliczenia, wywołania funkcji itd.
print(f"Za 5 lat będę mieć {wiek + 5} lat") # Za 5 lat będę mieć 30 lat
print(f"Imię wielkimi: {imie.upper()}")     # Imię wielkimi: JAN
```

Formatowanie precyzji liczb:

Wewnątrz klamr po dwukropku `:` można podać specyfikator formatu. Jest to szczególnie ważne przy wyświetlaniu liczb zmiennoprzecinkowych i wyrównywaniu kolumn:

SPECYFIKATOR	OPIS	PRZYKŁAD	WYNIK
<code>:.2f</code>	2 miejsca po przecinku	<code>f"{3.14159:.2f}"</code>	<code>3.14</code>
<code>:.4f</code>	4 miejsca po przecinku	<code>f"{3.14159:.4f}"</code>	<code>3.1416</code>
<code>:05d</code>	5 cyfr, uzupełnienie zerami	<code>f"{42:05d}"</code>	<code>00042</code>
<code>>10</code>	Wyrównanie do prawej, szer. 10	<code>f"{'abc':>10}"</code>	<code>abc</code>
<code><10</code>	Wyrównanie do lewej, szer. 10	<code>f"{'abc':<10}"</code>	<code>abc</code>
<code>^10</code>	Wyśrodkowanie, szer. 10	<code>f"{'abc':^10}"</code>	<code>abc</code>

```
srednia = 87.4567
```

```
print(f"Średnia: {srednia:.2f}")      # Średnia: 87.46
print(f"Średnia: {srednia:.0f}")      # Średnia: 87
print(f"Liczba: {42:05d}")           # Liczba: 00042
print(f"{'Imię':<15}{'Wiek':>5}")    # Imię      Wiek
```

Starsze sposoby formatowania (na wypadek, gdyby były potrzebne):

```
# Metoda .format()
print("Imię: {}, Wiek: {}".format(imie, wiek))

# Operator % (styl C)
print("Imię: %s, Wiek: %d" % (imie, wiek))
```

1.5. Pobieranie danych od użytkownika — `input()`

Funkcja `input()` wyświetla komunikat (prompt) i czeka na dane od użytkownika. Po naciśnięciu Enter zwraca wpisany tekst. **Zawsze zwraca wartość typu `str`** — nawet jeśli użytkownik wpisze liczbę, dla Pythona to nadal będzie tekst.

```
# Pobranie tekstu — nie wymaga konwersji
imie = input("Podaj imię: ")

# Pobranie liczby całkowitej — wymaga konwersji int()
wiek = int(input("Podaj wiek: "))

# Pobranie liczby zmiennoprzecinkowej — wymaga konwersji float()
wzrost = float(input("Podaj wzrost (m): "))
```

Obsługa błędnych danych wejściowych:

Gdy użytkownik wpisze tekst zamiast liczby, wywołanie `int()` spowoduje błąd `ValueError`. Blok `try/except` pozwala przechwycić ten błąd i zareagować na niego w kontrolowany sposób:

```
try:
    liczba = int(input("Podaj liczbę: "))
    print(f"Podano: {liczba}")
```

```
except ValueError:
    print("To nie jest prawidłowa liczba!")
```

Można też zbudować pętlę, która wymusi podanie poprawnej wartości:

```
while True:
    try:
        liczba = int(input("Podaj liczbę całkowitą: "))
        break # Wyjście z pętli gdy konwersja się powiodła
    except ValueError:
        print("Błąd! Podaj prawidłową liczbę.")
```

1.6. Komentarze

Komentarze to fragmenty kodu ignorowane przez interpreter. Służą do wyjaśniania logiki programu, dokumentowania funkcji oraz tymczasowego wyłączania fragmentów kodu. Dobry komentarz wyjaśnia **dlaczego** coś zostało zrobione, a nie **co** robi dany fragment — to drugie powinno być czytelne z samego kodu.

```
# To jest komentarz jednoliniowy – zaczyna się od znaku #

x = 42 # Komentarz może być też na końcu linii kodu

"""
To jest komentarz wieloliniowy (technicznie to napis – string literal).
Jeśli nie jest przypisany do zmiennej, Python go po prostu ignoruje.
Często używany do opisywania dłuższych bloków logiki.
"""

'''
Alternatywna forma komentarza wieloliniowego
z użyciem apostrofów zamiast cudzysłowów.
'''
```

1.7. Znak nowej linii i znaki specjalne

W napisach można używać znaków specjalnych (sekwencji ucieczki), które zaczynają się od odwrotnego ukośnika `\`:

SEKWENCJA	ZNACZENIE	PRZYKŁAD
<code>\n</code>	Nowa linia	<code>"linia1\nlinia2"</code>
<code>\t</code>	Tabulacja	<code>"kolumna1\tkolumna2"</code>
<code>\\</code>	Dostówny ukośnik <code>\</code>	<code>"C:\\folder"</code>
<code>\"</code>	Cudzysłów w napisie	<code>"Mówi \"cześć\""</code>
<code>\'</code>	Apostrof w napisie	<code>'It\'s okay'</code>

```
print("Pierwsza linia\nDruga linia")
# Pierwsza linia
# Druga linia

print("Kol1\tKol2\tKol3")
```

```
# Kol1    Kol2    Kol3

# Raw string (r"...") – ignoruje znaki specjalne
print(r"C:\nowy\folder") # C:\nowy\folder (bez nowej linii)
```

1.8. Obsługa wyjątków — try / except / finally

Błędy są naturalną częścią programowania. Python oferuje mechanizm wyjątków, który pozwala na eleganckie reagowanie na błędy bez przerywania działania programu. Blok `try` zawiera kod, który może spowodować błąd. Blok `except` obsługuje konkretny typ błędu. Opcjonalny blok `finally` wykonuje się zawsze — niezależnie od tego, czy błąd wystąpił.

Najczęstsze typy wyjątków:

WYJĄTEK	KIEDY WYSTĘPUJE	PRZYKŁAD
<code>ValueError</code>	Niepoprawna wartość	<code>int("abc")</code>
<code>TypeError</code>	Niepoprawny typ	<code>"tekst" + 5</code>
<code>IndexError</code>	Indeks poza zakresem	<code>lista[100]</code> gdy <code>len(lista) < 101</code>
<code>KeyError</code>	Brak klucza w słowniku	<code>słownik["brak"]</code>
<code>ZeroDivisionError</code>	Dzielenie przez zero	<code>10 / 0</code>
<code>FileNotFoundError</code>	Plik nie istnieje	<code>open("brak.txt")</code>
<code>AttributeError</code>	Brak atrybutu/metody	<code>"tekst".nieistniejąca()</code>
<code>NameError</code>	Niezdefiniowana zmienna	<code>print(zmienna)</code> bez deklaracji

Podstawowa obsługa wyjątków:

```
try:
    liczba = int(input("Podaj liczbę: "))
    wynik = 100 / liczba
    print(f"Wynik: {wynik}")
except ValueError:
    print("To nie jest prawidłowa liczba!")
except ZeroDivisionError:
    print("Nie można dzielić przez zero!")
```

Łapanie wielu wyjątków naraz:

```
try:
    # Kod, który może rzucić różne wyjątki
    dane = [1, 2, 3]
    indeks = int(input("Podaj indeks: "))
    print(dane[indeks])
except (ValueError, IndexError) as e:
    print(f"Błąd: {e}")
```

Blok `else` — wykonuje się tylko, gdy NIE było wyjątku:

```
try:
    liczba = int(input("Podaj liczbę: "))
except ValueError:
    print("Niepoprawna wartość!")
else:
    print(f"Podano poprawną liczbę: {liczba}")
```

Blok **finally** — wykonuje się ZAWSZE:

Blok **finally** jest gwarancją, że pewne operacje zostaną wykonane niezależnie od tego, co się wydarzy — np. zamknięcie pliku, zwolnienie zasobów:

```
try:
    plik = open("dane.txt", "r")
    zawartosc = plik.read()
except FileNotFoundError:
    print("Plik nie istnieje!")
finally:
    # Ten blok wykona się ZAWSZE — nawet przy błędzie
    print("Kończymy pracę z plikiem")
```

Kompletna struktura try/except/else/finally:

```
try:
    # Kod, który może spowodować błąd
    wynik = ryzykowna_operacja()
except TypBledu:
    # Obsługa konkretnego błędu
    print("Obsługa błędu")
except Exception as e:
    # Obsługa DOWOLNEGO innego błędu
    print(f"Nieoczekiwany błąd: {e}")
else:
    # Wykonuje się tylko gdy NIE było błędu
    print(f"Operacja udana: {wynik}")
finally:
    # Wykonuje się ZAWSZE
    print("Sprzątanie zasobów")
```

Pętla z ponawianiem po błędzie:

```
while True:
    try:
        wiek = int(input("Podaj wiek (0-150): "))
        if not 0 <= wiek <= 150:
            raise ValueError("Wiek poza zakresem")
        break # Poprawna wartość — wyjście z pętli
    except ValueError as e:
        print(f"Błąd: {e}. Spróbuj ponownie.")
```

```
print(f"Twój wiek: {wiek}")
```

W powyższym przykładzie `raise ValueError(...)` to ręczne rzucenie wyjątku — przydatne do walidacji danych.

1.9. Wczytywanie wielu wartości w jednej linii

Czasem programy wymagają pobrania kilku wartości w jednej linii, oddzielonych spacją lub innym separatorem. Python oferuje kilka eleganckich sposobów, aby to zrobić.

Metoda `split()` + `map()`:

Funkcja `map()` stosuje podaną funkcję (np. `int`) do każdego elementu uzyskanego z `split()`. To najczęstszy wzorec wczytywania wielu liczb:

```
# Wczytywanie dwóch liczb z jednej linii
# Użytkownik wpisuje np.: 10 20
a, b = map(int, input("Podaj dwie liczby: ").split())
print(f"a = {a}, b = {b}")

# Wczytywanie trzech wartości
x, y, z = map(float, input("Podaj x, y, z: ").split())

# Wczytywanie wielu liczb do listy (gdy nie wiemy ile)
liczby = list(map(int, input("Podaj liczby: ").split()))
print(f"Podano {len(liczby)} liczb: {liczby}")
```

Co się tu dzieje krok po kroku:

KROK	WYRAŻENIE	WYNIK
1.	<code>input("...")</code>	<code>"10 20 30"</code> (string)
2.	<code>.split()</code>	<code>["10", "20", "30"]</code> (lista stringów)
3.	<code>map(int, ...)</code>	Iterator: 10, 20, 30 (inty)
4.	<code>list(...)</code>	<code>[10, 20, 30]</code> (lista intów)

Separator inny niż spacja:

```
# Wczytywanie wartości oddzielonych przecinkiem
dane = input("Podaj wartości (oddzielone przecinkiem): ") # np. "Jan,25,Warszawa"
czesci = dane.split(",")
imie = czesci[0]
wiek = int(czesci[1])
miasto = czesci[2]

# Lub jednolinijkowo z rozpakowaniem
imie, wiek, miasto = input("Podaj dane: ").split(",")
wiek = int(wiek) # Konwersja na int osobno
```

Wczytywanie danych w pętli (n linii):

```
n = int(input("Ile osób? "))
osoby = []
for _ in range(n):
    imie, wiek = input("Podaj imię i wiek: ").split()
    osoby.append({"imie": imie, "wiek": int(wiek)})

for osoba in osoby:
    print(f"{osoba['imie']} ma {osoba['wiek']} lat")
```

2. Operatory

Operatory to symbole lub słowa kluczowe, które wykonują operacje na wartościach (operandach). Python oferuje bogaty zestaw operatorów, podzielonych na kilka kategorii.

2.1. Operatory arytmetyczne

Operatory arytmetyczne służą do wykonywania działań matematycznych. Warto zwrócić szczególną uwagę na różnicę między zwykłym dzieleniem `/` (które zawsze daje `float`) a dzieleniem całkowitym `//`.

OPERATOR	NAZWA	PRZYKŁAD	WYNIK	UWAGI
<code>+</code>	Dodawanie	<code>17 + 5</code>	<code>22</code>	
<code>-</code>	Odejmowanie	<code>17 - 5</code>	<code>12</code>	
<code>*</code>	Mnożenie	<code>17 * 5</code>	<code>85</code>	
<code>/</code>	Dzielenie	<code>17 / 5</code>	<code>3.4</code>	Zawsze zwraca <code>float</code> !
<code>//</code>	Dzielenie całkowite	<code>17 // 5</code>	<code>3</code>	Zaokrągla w dół
<code>%</code>	Modulo (reszta)	<code>17 % 5</code>	<code>2</code>	Kluczowy operator
<code>**</code>	Potęgowanie	<code>2 ** 10</code>	<code>1024</code>	

```
a = 17
b = 5

print(a + b)    # 22    - dodawanie
print(a - b)    # 12    - odejmowanie
print(a * b)    # 85    - mnożenie
print(a / b)    # 3.4    - dzielenie (ZAWSZE float, nawet 10/2 = 5.0)
print(a // b)   # 3      - dzielenie całkowite
print(a % b)    # 2      - reszta z dzielenia
print(a ** b)   # 1419857 - potęgowanie (175)
```

Operator modulo `%` — dlaczego jest tak ważny:

Modulo zwraca resztę z dzielenia. To jeden z najczęściej używanych operatorów w programowaniu. Służy do:

1. **Sprawdzania parzystości** — jeśli `n % 2 == 0`, liczba jest parzysta
2. **Cyklicznego przechodzenia** — np. po alfabecie (po `z` wracamy do `a`)
3. **Walidacji danych** — np. suma kontrolna PESEL
4. **Okresowych powtórzeń** — np. co n-ty element

```

# Sprawdzanie parzystości
liczba = 42
if liczba % 2 == 0:
    print("Parzysta")
else:
    print("Nieparzysta")

# Cykliczne przechodzenie po tablicy (po ostatnim elemencie → pierwszy)
tablica = ["A", "B", "C"]
for i in range(10):
    print(tablica[i % len(tablica)], end=" ")

# Wynik: A B C A B C A B C A

# Wyodrębnianie ostatniej cyfry liczby
liczba = 12345
ostatnia_cyfra = liczba % 10 # 5

# Sprawdzanie podzielności
if 100 % 7 == 0:
    print("100 jest podzielne przez 7")
else:
    print("100 NIE jest podzielne przez 7")

```

2.2. Operatory porównania

Operatory porównania porównują dwie wartości i zwracają wartość logiczną `True` lub `False`. Są fundamentalne dla instrukcji warunkowych `if` i pętli `while`.

OPERATOR	ZNACZENIE	PRZYKŁAD	WYNIK
<code>==</code>	Równe	<code>5 == 5</code>	<code>True</code>
<code>!=</code>	Różne	<code>5 != 3</code>	<code>True</code>
<code><</code>	Mniejsze	<code>3 < 5</code>	<code>True</code>
<code>></code>	Większe	<code>5 > 3</code>	<code>True</code>
<code><=</code>	Mniejsze lub równe	<code>5 <= 5</code>	<code>True</code>
<code>>=</code>	Większe lub równe	<code>3 >= 5</code>	<code>False</code>

Ważna uwaga: `==` to porównanie, a `=` to przypisanie. To częsty błąd początkujących programistów.

```

a = 10
b = 20

print(a == b) # False
print(a != b) # True
print(a < b)  # True
print(a > b)  # False
print(a <= b) # True
print(a >= b) # False

```


Python pozwala na łańcuchowe porównania, co jest unikalną cechą tego języka:

```
x = 15

# Zamiast: if x >= 1 and x <= 100:
if 1 <= x <= 100:
    print("x jest w zakresie 1-100")

# Zamiast: if a < b and b < c:
if a < b < c:
    print("a < b < c")
```

2.3. Operatory logiczne

Operatory logiczne służą do łączenia warunków. Python używa angielskich słów kluczowych `and`, `or`, `not` zamiast symboli `&&`, `||`, `!` znanych z C++ czy Javy.

OPERATOR	ZNACZENIE	PRAWDA GDY...
and	Koniunkcja (I)	Oba warunki są True
or	Alternatywa (LUB)	Przynajmniej jeden warunek jest True
not	Negacja (NIE)	Warunek jest False

Tabela prawdy:

A	B	A AND B	A OR B	NOT A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

```
wiek = 20
ma_bilet = True

if wiek >= 18 and ma_bilet:
    print("Możesz wejść")

x = 5
if x < 0 or x > 100:
    print("Poza zakresem")
else:
    print("W zakresie")

jest_zamkniety = False
if not jest_zamkniety:
    print("Sklep jest otwarty")
```

2.4. Operatory przypisania

Operatory przypisania to skrócona forma zapisu typowych operacji arytmetycznych na zmiennej:

OPERATOR	RÓWNOWAŻNE	PRZYKŁAD
<code>=</code>	—	<code>x = 10</code>
<code>+=</code>	<code>x = x + 5</code>	<code>x += 5</code>
<code>-=</code>	<code>x = x - 3</code>	<code>x -= 3</code>
<code>*=</code>	<code>x = x * 2</code>	<code>x *= 2</code>
<code>/=</code>	<code>x = x / 4</code>	<code>x /= 4</code>
<code>//=</code>	<code>x = x // 2</code>	<code>x //= 2</code>
<code>%=</code>	<code>x = x % 2</code>	<code>x %= 2</code>
<code>**=</code>	<code>x = x ** 3</code>	<code>x **= 3</code>

```

x = 10
x += 5    # x = 15
x -= 3    # x = 12
x *= 2    # x = 24
x /= 5    # x = 4
x %= 3    # x = 1

```

2.5. Operatory przynależności i tożsamości

Operator `in` sprawdza, czy element znajduje się w kolekcji (liście, napisie, zbiorze, słowniku). Jest niesamowicie przydatny i czytelny:

```

lista = [1, 2, 3, 4, 5]
print(3 in lista)      # True
print(6 in lista)      # False
print(6 not in lista)  # True

tekst = "Python"
print("th" in tekst)   # True
print("xyz" in tekst)  # False

# W słowniku sprawdza KLUCZE (nie wartości!)
słownik = {"imie": "Jan", "wiek": 25}
print("imie" in słownik) # True
print("Jan" in słownik)  # False (to wartość, nie klucz)

```

Operator `is` sprawdza **tożsamość obiektów** — czy dwa obiekty to ten sam obiekt w pamięci (nie czy mają tę samą wartość). W praktyce używa się go głównie do porównywania z `None`:

```

x = None

# ✅ Prawidłowo — porównanie z None zawsze przez is
if x is None:
    print("x jest None")

if x is not None:
    print("x nie jest None")

```

```
# ❌ Nieprawidłowo (choć zwykle zadziała)
# if x == None:
```

2.6. Priorytet operatorów

Kiedy w jednym wyrażeniu pojawia się kilka operatorów, Python wykonuje je w określonej kolejności — dokładnie tak jak w matematyce. Nawias `()` zawsze ma najwyższy priorytet i pozwala wymusić żądaną kolejność.

PRIORYTET	OPERATOR	OPIS
1 (najwyższy)	<code>()</code>	Nawiasy
2	<code>**</code>	Potęgowanie
3	<code>+x</code> , <code>-x</code> , <code>~x</code>	Jednoargumentowe: plus, minus, negacja bitowa
4	<code>*</code> , <code>/</code> , <code>//</code> , <code>%</code>	Mnożenie, dzielenie, dzielenie całkowite, reszta
5	<code>+</code> , <code>-</code>	Dodawanie, odejmowanie
6	<code><<</code> , <code>>></code>	Przesunięcia bitowe
7	<code>&</code>	Bitowe AND
8	<code>^</code>	Bitowe XOR
9	<code> </code>	Bitowe OR
10	<code>=</code> , <code>!=</code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code> , <code>in</code> , <code>not in</code> , <code>is</code> , <code>is not</code>	Porównania
11	<code>not</code>	Logiczne NOT
12	<code>and</code>	Logiczne AND
13 (najniższy)	<code>or</code>	Logiczne OR

Przykłady — kolejność działań:

```
# Mnożenie przed dodawaniem (jak w matematyce)
wynik = 2 + 3 * 4      # 14, a nie 20!

# Nawiasy zmieniają kolejność
wynik = (2 + 3) * 4     # 20

# Potęgowanie prawostronnie łączne
wynik = 2 ** 3 ** 2     # 2 ** (3 ** 2) = 2 ** 9 = 512
wynik = (2 ** 3) ** 2   # 8 ** 2 = 64

# Dzielenie całkowite i reszta – ten sam priorytet co * i /
wynik = 10 + 4 // 3     # 10 + 1 = 11
wynik = 10 % 3 + 1      # 1 + 1 = 2

# not ma wyższy priorytet niż and i or
wynik = not True or False # (not True) or False = False or False = False
wynik = not (True or False) # not True = False
```

Praktyczna zasada: Jeśli nie jesteś pewien kolejności — użyj nawiasów. Kod z nawiasami jest bardziej czytelny i mniej podatny na błędy.

```
# ✅ Czytelnie — intencja jest natychmiast widoczna
wynik = (a + b) * (c - d)

# ❌ Niebezpiecznie — czytelnik musi znać priorytety na pamięć
wynik = a + b * c - d
```

3. Instrukcje sterujące

Instrukcje sterujące kontrolują przepływ wykonania programu — pozwalają na podejmowanie decyzji (`if`) oraz wielokrotne wykonywanie bloków kodu (pętle `for`, `while`).

3.1. Instrukcja warunkowa `if` / `elif` / `else`

Instrukcja `if` to podstawowy mechanizm podejmowania decyzji w programie. Warunek jest wyrażeniem, które Python ewaluuje do `True` lub `False`. Jeśli jest prawdziwy, wykonywany jest blok kodu wcięty pod `if`.

Słowo kluczowe `elif` (skrót od „else if”) pozwala na sprawdzanie kolejnych warunków, a `else` obsługuje przypadek, gdy żaden wcześniejszy warunek nie był spełniony.

```
liczba = int(input("Podaj liczbę: "))

if liczba > 0:
    print("Liczba jest dodatnia")
elif liczba < 0:
    print("Liczba jest ujemna")
else:
    print("Liczba jest zerem")
```

Zagnieżdżone warunki — instrukcje `if` mogą być umieszczone wewnątrz innych instrukcji `if`:

```
wiek = 25
posiada_prawo_jazdy = True

if wiek ≥ 18:
    if posiada_prawo_jazdy:
        print("Może prowadzić")
    else:
        print("Brak prawa jazdy")
else:
    print("Za młody")
```

Operator trójargumentowy (ternary operator):

Python oferuje skróconą formę instrukcji warunkowej, zapisywaną w jednej linii. Jest bardzo przydatna przy prostych przypisaniach:

```
wiek = 20
status = "dorosły" if wiek ≥ 18 else "niepełnoletni"
print(status) # dorosły
```

```
# Przydatne do szybkiego wyboru wartości  
a, b = 5, 10  
maksimum = a if a > b else b # 10
```

3.2. Instrukcja `match` / `case` — wzorce (switch)

Python **nie ma** słowa kluczowego `switch` (jak w C, Java czy C#). Do wersji 3.9 jedynym sposobem obsługi wielu wariantów było łańcuchowanie `if` / `elif` / `else`. Od Pythona 3.10 dostępna jest instrukcja `match` / `case`, która jest odpowiednikiem `switch` — i przy okazji znacznie potężniejsza.

Uwaga dla egzaminu: Jeśli zadanie nie precyzuje wersji Pythona, bezpieczniej używać `if/elif/else`. Instrukcja `match` działa tylko od Python 3.10+.

Podstawowa składnia:

```
match wyrażenie:  
    case wartość1:  
        # kod dla wartości1  
    case wartość2:  
        # kod dla wartości2  
    case _:  
        # domyślny przypadek (odpowiednik "default" / "else")
```

Przykład 1 — prosty wybór dnia tygodnia:

```
dzien = input("Podaj numer dnia (1-7): ")  
numer = int(dzien)  
  
match numer:  
    case 1:  
        print("Poniedziałek")  
    case 2:  
        print("Wtorek")  
    case 3:  
        print("Środa")  
    case 4:  
        print("Czwartek")  
    case 5:  
        print("Piątek")  
    case 6:  
        print("Sobota")  
    case 7:  
        print("Niedziela")  
    case _:  
        print("Nieprawidłowy numer dnia")
```

Przykład 2 — kilka wartości w jednym case (operator `|`):

```
miesiąc = int(input("Podaj numer miesiąca (1-12): "))

match miesiąc:
    case 12 | 1 | 2:
        print("Zima")
    case 3 | 4 | 5:
        print("Wiosna")
    case 6 | 7 | 8:
        print("Lato")
    case 9 | 10 | 11:
        print("Jesień")
    case _:
        print("Nieprawidłowy miesiąc")
```

Przykład 3 — dopasowanie do napisu:

```
komenda = input("Podaj komendę (start/stop/pause): ").lower()

match komenda:
    case "start":
        print("Uruchamiam...")
    case "stop":
        print("Zatrzymuję...")
    case "pause":
        print("Wstrzymuję...")
    case _:
        print(f"Nieznana komenda: {komenda}")
```

Odpowiednik `match/case` w wersji `if/elif/else` (kompatybilny z każdą wersją Pythona):

Poniższy kod działa identycznie jak przykład z miesiącami powyżej, ale jest kompatybilny z Pythonem 3.9 i starszym:

```
miesiąc = int(input("Podaj numer miesiąca (1-12): "))

if miesiąc in (12, 1, 2):
    print("Zima")
elif miesiąc in (3, 4, 5):
    print("Wiosna")
elif miesiąc in (6, 7, 8):
    print("Lato")
elif miesiąc in (9, 10, 11):
    print("Jesień")
else:
    print("Nieprawidłowy miesiąc")
```

Przykład 4 — symulator kalkulatora (typowe zadanie egzaminacyjne):

```
a = float(input("Podaj pierwszą liczbę: "))
operator = input("Podaj operator (+, -, *, /): ")
b = float(input("Podaj drugą liczbę: "))

match operator:
    case "+":
        wynik = a + b
    case "-":
        wynik = a - b
    case "*":
        wynik = a * b
    case "/":
        if b == 0:
            print("Błąd: dzielenie przez zero!")
        else:
            wynik = a / b
            print(f"{a} / {b} = {wynik}")
    case _:
        print(f"Nieznany operator: {operator}")
```

Ten sam przykład z `if/elif/else` (Python 3.9 i starszy):

```
a = float(input("Podaj pierwszą liczbę: "))
operator = input("Podaj operator (+, -, *, /): ")
b = float(input("Podaj drugą liczbę: "))

if operator == "+":
    wynik = a + b
    print(f"{a} + {b} = {wynik}")
elif operator == "-":
    wynik = a - b
    print(f"{a} - {b} = {wynik}")
elif operator == "*":
    wynik = a * b
    print(f"{a} * {b} = {wynik}")
elif operator == "/":
    if b == 0:
        print("Błąd: dzielenie przez zero!")
    else:
        wynik = a / b
        print(f"{a} / {b} = {wynik}")
else:
    print(f"Nieznany operator: {operator}")
```

Porównanie: `match/case` vs `if/elif` vs słownik funkcji:

METODA	WERSJA PYTHON	ZALETY
<code>if / elif / else</code>	Każda	Zawsze działa, czytelna dla prostych przypadków
<code>match / case</code>	3.10+	Bardziej czytelna przy wielu wariantach, obsługa wzorców
Słownik funkcji	Każda	Elegancka przy wielu wariantach bez zagnieżdżeń

Technika z słownikiem — zamiennik `switch` dostępny w każdej wersji Pythona:

```
def dodaj(a, b):    return a + b
def odejmij(a, b):  return a - b
def mnoz(a, b):     return a * b
def dziel(a, b):    return a / b if b != 0 else "Dzielenie przez zero!"

operacje = {
    "+": dodaj,
    "-": odejmij,
    "*": mnoz,
    "/": dziel,
}

a = float(input("a: "))
op = input("Operator: ")
b = float(input("b: "))

if op in operacje:
    print(f"Wynik: {operacje[op](a, b)}")
else:
    print("Nieznany operator")
```

3.3. Pętla `for`

Pętla `for` w Pythonie działa inaczej niż w większości języków programowania. Zamiast klasycznej pętli liczącej (`for (int i = 0; i < n; i++)` z C++ lub Javy), Python iteruje bezpośrednio po elementach kolekcji — listy, napisu, zakresu liczb itp. To sprawia, że kod jest krótszy i bardziej czytelny, ale wymaga przyzwyczajenia, jeśli znasz inne języki.

Do tworzenia sekwencji liczbowych służy funkcja `range()`, która generuje ciąg wartości bez tworzenia listy w pamięci (to tzw. generator — leniwa ewaluacja).

Funkcja `range()` — generuje sekwencję liczb:

WYWOŁANIE	GENERUJE	OPIS
<code>range(5)</code>	0, 1, 2, 3, 4	Od 0 do n-1
<code>range(2, 8)</code>	2, 3, 4, 5, 6, 7	Od start do stop-1
<code>range(0, 20, 3)</code>	0, 3, 6, 9, 12, 15, 18	Z krokiem 3
<code>range(10, 0, -1)</code>	10, 9, 8, ..., 2, 1	Odliczanie w dół
<code>range(5, 5)</code>	(pusto)	Start == stop → nic

```
# Iteracja po zakresie liczb
for i in range(5):
```



```

print(i)    # 0, 1, 2, 3, 4

# Zakres od-do (UWAGA: koniec jest WYŁĄCZNY – do 7, nie do 8)
for i in range(2, 8):
    print(i)    # 2, 3, 4, 5, 6, 7

# Zakres z krokiem
for i in range(0, 20, 3):
    print(i)    # 0, 3, 6, 9, 12, 15, 18

# Odliczanie w dół
for i in range(10, 0, -1):
    print(i)    # 10, 9, 8, 7, 6, 5, 4, 3, 2, 1

```

Iteracja po kolekcjach:

```

# Iteracja po liście – zmienna przybiera wartość kolejnego elementu
kolory = ["czerwony", "zielony", "niebieski"]
for kolor in kolory:
    print(kolor)

# Iteracja po napisie – każdy znak osobno
for znak in "Python":
    print(znak) # P, y, t, h, o, n

```

3.4. Funkcja `enumerate()` — iteracja z indeksem

Funkcja `enumerate()` to niezwykle przydatne narzędzie, które podczas iteracji po kolekcji podaje jednocześnie **indeks** i **wartość** każdego elementu. Bez niej trzeba by ręcznie zarządzać licznikiem.

Składnia: `enumerate(kolekcja, start=0)` — domyślnie indeksy zaczynają się od 0, ale można zmienić wartość startową.

```

owoce = ["jabłko", "banan", "wiśnia", "gruszka"]

# Bez enumerate – trzeba osobno zarządzać indeksem
i = 0
for owoc in owoce:
    print(f"{i}: {owoc}")
    i += 1

# Z enumerate – elegancko i bezpiecznie
for i, owoc in enumerate(owoce):
    print(f"{i}: {owoc}")

# 0: jabłko
# 1: banan
# 2: wiśnia
# 3: gruszka

# Indeksowanie od 1 zamiast od 0
for numer, owoc in enumerate(owoce, start=1):
    print(f"{numer}. {owoc}")

# 1. jabłko

```

```
# 2. banan
# 3. wiśnia
# 4. gruszka
```

`enumerate()` zwraca pary `(indeks, element)`, które rozpakowujemy do dwóch zmiennych. Jest szczególnie przydatna, gdy potrzebujemy jednocześnie pozycji elementu i jego wartości — na przykład przy wyszukiwaniu, modyfikowaniu listy, czy wyświetlaniu numerowanych wyników.

3.5. Pętla `while`

Pętla `while` powtarza blok kodu **dopóki warunek jest prawdziwy**. Jest odpowiednia, gdy nie wiemy z góry, ile razy pętla powinna się wykonać.

```
# Podstawowa pętla while - odliczanie
licznik = 5
while licznik > 0:
    print(licznik)
    licznik -= 1
print("Start!")

# Pętla nieskończona z warunkiem wyjścia
while True:
    odpowiedz = input("Czy kontynuować? (t/n): ")
    if odpowiedz == "n":
        break
    print("Kontynuujemy...")
```

Ważna uwaga: Pętla `while True:` tworzy pętlę nieskończoną. Jedynym sposobem na wyjście z niej jest instrukcja `break`. Jest to częsty wzorec w programowaniu — używany np. do menu programu, powtarzania operacji do momentu poprawnego wpisania danych itp.

3.6. `break`, `continue` i `else` w pętlach

W trakcie wykonywania pętli może zająć potrzeba przerwania jej działania, pominięcia pewnych iteracji, lub wykonania kodu po jej naturalnym zakończeniu. Do tego służą trzy słowa kluczowe, które dają precyzyjną kontrolę nad przebiegiem pętli:

SŁOWO KLUCZOWE	DZIAŁANIE
<code>break</code>	Natychmiast kończy pętlę (wyjście)
<code>continue</code>	Pomija resztę bieżącej iteracji, przechodzi do następnej
<code>else</code> (po pętli)	Wykonuje się, gdy pętla zakończyła się bez <code>break</code>

```
# break - przerywa pętlę po znalezieniu elementu
for i in range(10):
    if i == 5:
        print(f"Znaleziono {i}, przerywam!")
        break
    print(i)
# Wypisze: 0, 1, 2, 3, 4, Znaleziono 5, przerywam!

# continue - pomija parzyste, wypisuje nieparzyste
for i in range(10):
    if i % 2 == 0:
        continue
    print(i)
# Wypisze: 1, 3, 5, 7, 9
```

```

        continue    # Pomiń tę iterację

    print(i)

# Wypisze: 1, 3, 5, 7, 9

# else w pętli – rzadko używane, ale przydatne
for i in range(10):
    if i == 100: # Nigdy nie będzie True
        break

    else:
        print("Pętla zakończyła się normalnie (bez break)")

# To się WYKONA, bo break nie został użyty

```

3.7. Pętle zagnieżdżone

Pętle zagnieżdżone to pętle umieszczone wewnątrz innych pętli. Zewnętrzna pętla wykonuje się raz, a dla każdej jej iteracji wewnętrzna pętla wykonuje się od początku do końca. Są niezbędne do pracy z tablicami dwuwymiarowymi i algorytmami sortowania.

```

# Tabliczka mnożenia 5x5
for i in range(1, 6):
    for j in range(1, 6):
        print(f"{i*j:4d}", end=" ")
    print() # Nowa linia po każdym wierszu

# Wynik:
#  1  2  3  4  5
#  2  4  6  8 10
#  3  6  9 12 15
#  4  8 12 16 20
#  5 10 15 20 25

```

4. Struktury danych

Python oferuje kilka wbudowanych typów kolekcji, z których każdy ma swoje zastosowanie. Wybór odpowiedniej struktury danych jest jedną z najważniejszych decyzji przy pisaniu programu — wpływa na wydajność, czytelność i łatwość modyfikacji kodu. Znajomość właściwości każdego typu pozwala wybrać ten najlepiej pasujący do danego problemu.

Poniższa tabela podsumowuje kluczowe różnice:

STRUKTURA	MODYFIKOWALNA	UPORZĄDKOWANA	DUPLIKATY	SKŁADNIA
<code>list</code>	✅ Tak	✅ Tak	✅ Tak	<code>[1, 2, 3]</code>
<code>tuple</code>	❌ Nie	✅ Tak	✅ Tak	<code>(1, 2, 3)</code>
<code>dict</code>	✅ Tak	✅ Tak (od 3.7)	❌ Klucze unikalne	<code>{"a": 1}</code>
<code>set</code>	✅ Tak	❌ Nie	❌ Nie	<code>{1, 2, 3}</code>

4.1. Listy (`list`)

Lista jest **najważniejszą** i najczęściej używaną strukturą danych w Pythonie. Pełni rolę tablicy (array) z innych języków, ale jest znacznie bardziej elastyczna — może przechowywać elementy różnych typów, automatycznie zmienia swój rozmiar, i oferuje bogaty zestaw wbudowanych metod.

Tworzenie list:

```
pusta_lista = []
liczby = [1, 2, 3, 4, 5]
mieszana = [1, "dwa", 3.0, True]          # Różne typy w jednej liście
zera = [0] * 10                            # [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
zakres = list(range(1, 11))                # [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
znaki = list("Python")                    # ['P', 'y', 't', 'h', 'o', 'n']
```

Indeksowanie — dostęp do elementów:

Indeksy w Pythonie zaczynają się od `0`. Można też używać indeksów ujemnych, które liczą od końca listy:

Indeks dodatni:	0	1	2	3	4
	[10,	20,	30,	40,	50]
Indeks ujemny:	-5	-4	-3	-2	-1

```
lista = [10, 20, 30, 40, 50]

print(lista[0])    # 10    - pierwszy element
print(lista[2])    # 30    - trzeci element
print(lista[-1])   # 50    - ostatni element
print(lista[-2])   # 40    - przedostatni element
```

Wycinanie (slicing) — pozwala wybrać fragment listy. Składnia: `lista[start:stop:krok]`, gdzie `stop` jest wyłączny:

```
lista = [10, 20, 30, 40, 50]

print(lista[1:4])  # [20, 30, 40] - od indeksu 1 do 3 (4 jest wyłączny)
print(lista[:3])   # [10, 20, 30] - pierwsze 3 elementy
print(lista[2:])   # [30, 40, 50] - od indeksu 2 do końca
print(lista[::-1]) # [50, 40, 30, 20, 10] - odwrócona lista
print(lista[::2])  # [10, 30, 50] - co drugi element
```

Modyfikacja listy:

```
lista = [1, 2, 3, 4, 5]

# Zmiana elementu pod konkretnym indeksem
lista[0] = 99          # [99, 2, 3, 4, 5]

# Dodawanie elementów
lista.append(6)         # Dodaje na koniec → [99, 2, 3, 4, 5, 6]
lista.insert(0, 0)      # Wstawia na indeks 0 → [0, 99, 2, 3, 4, 5, 6]
lista.extend([7, 8])    # Dodaje wiele elementów → [0, 99, 2, 3, 4, 5, 6, 7, 8]

# Usuwanie elementów
lista.remove(99)        # Usuwa PIERWSZE wystąpienie wartości 99
element = lista.pop()   # Usuwa i ZWRACA ostatni element
element = lista.pop(0)  # Usuwa i ZWRACA element o indeksie 0
```

```
del lista[2]                # Usuwa element o indeksie 2 (bez zwracania)
lista.clear()               # Usuwa WSZYSTKIE elementy → []
```

Przydatne metody i operacje:

```
lista = [3, 1, 4, 1, 5, 9, 2, 6]

# Informacje o liście
print(len(lista))          # 8    – liczba elementów
print(min(lista))          # 1    – wartość minimalna
print(max(lista))          # 9    – wartość maksymalna
print(sum(lista))          # 31   – suma elementów

# Wyszukiwanie
print(lista.index(4))       # 2    – indeks PIERWSZEGO wystąpienia
print(lista.count(1))       # 2    – ile razy wartość 1 występuje
print(5 in lista)          # True  – czy element jest w liście

# Sortowanie
lista.sort()                # Sortuje rosnąco NA MIEJSCU (modyfikuje listę)
lista.sort(reverse=True)    # Sortuje malejąco NA MIEJSCU

# Sortowanie BEZ modyfikacji oryginału – funkcja sorted()
nowa = sorted(lista)        # Zwraca NOWĄ posortowaną listę
nowa = sorted(lista, reverse=True) # Malejąco

# Odwracanie
lista.reverse()             # Odwraca NA MIEJSCU
nowa = lista[::-1]          # Zwraca NOWĄ odwróconą listę (oryginał bez zmian)

# Kopiowanie listy
kopia = lista.copy()        # Tworzy płytką kopię
kopia = lista[:]            # Alternatywna forma kopiowania
# UWAGA: kopia = lista NIE kopiuje – obie zmienne wskazują na TĘ SAMĄ listę!
```

List comprehension (wyrażenia listowe):

List comprehension to elegancki, jednoliniowy sposób tworzenia list na podstawie wyrażenia. To jeden z najbardziej „pythonowych” wzorców — pozwala zastąpić kilka linii pętli `for` jednym czytelnym wyrażeniem:

```
# Kwadraty liczb 0-9
kwadraty = [x**2 for x in range(10)]
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# Tylko parzyste z zakresu 0-19
parzyste = [x for x in range(20) if x % 2 == 0]
# [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

# Transformacja – zamiana na wielkie litery
imiona = ["jan", "anna", "piotr"]
wielkie = [imie.upper() for imie in imiona]
# ["JAN", "ANNA", "PIOTR"]
```

```
# Równoważna forma z pętlą (dłuższa):
wielkie = []
for imie in imiona:
    wielkie.append(imie.upper())
```

4.2. Krotki (tuple)

Krotka jest **niemodyfikowalną** (immutable) wersją listy. Po utworzeniu nie można zmienić jej elementów — nie można dodawać, usuwać ani modyfikować. Krotki są szybsze od list i mogą służyć jako klucze słownika (czego lista nie może).

```
# Tworzenie krotek
punkt = (3, 5)
kolory = ("red", "green", "blue")
jeden = (42,) # UWAGA: krotka jednoelementowa MUSI mieć przecinek!
nie_krotka = (42) # To jest zwykła liczba 42, nie krotka!

# Dostęp do elementów — identycznie jak w liście
print(kolory[0]) # "red"
print(kolory[-1]) # "blue"
print(len(kolory)) # 3
```

Rozpakowanie krotki (tuple unpacking):

To niezwykle przydatna technika — pozwala przypisać elementy krotki do osobnych zmiennych w jednej instrukcji:

```
punkt = (3, 5)
x, y = punkt
print(x, y) # 3 5

# Funkcja zwracająca krotkę
def podziel(a, b):
    return a // b, a % b # Python automatycznie tworzy krotkę

iloraz, reszta = podziel(17, 5)
print(iloraz, reszta) # 3 2

# Zamiana wartości dwóch zmiennych — idiom Pythona
a, b = 1, 2
a, b = b, a # Teraz a=2, b=1 (bez zmiennej tymczasowej!)
```

4.3. Słowniki (dict)

Słownik przechowuje pary **klucz → wartość** (key-value pairs). Klucze muszą być unikalne i niemodyfikowalne (np. `str`, `int`, `tuple`), a wartości mogą być dowolnego typu. Słowniki są zoptymalizowane pod kątem szybkiego wyszukiwania po kluczu.

```
# Tworzenie słownika
pusty = {}
osoba = {"imie": "Jan", "wiek": 25, "miasto": "Warszawa"}

# Dostęp do wartości
```

```

print(osoba["imie"])          # "Jan"
print(osoba.get("wiek"))      # 25
print(osoba.get("telefon", "brak")) # "brak" – wartość domyślna gdy brak klucza
# osoba["telefon"]           # KeyError! – klucz nie istnieje

# Dodawanie i modyfikacja
osoba["email"] = "jan@mail.com" # Dodaje nowy klucz
osoba["wiek"] = 26               # Modyfikuje istniejący

# Usuwanie
del osoba["miasto"]

wartosc = osoba.pop("email")     # Usuwa i zwraca wartość

# Scalanie słowników – metoda update()
osoba2 = {"telefon": "123-456", "wiek": 30} # wiek nadpisze stary!
osoba.update(osoba2)             # dodaje/nadpisuje pary z osoba2
print(osoba)                     # {..., "telefon": "123-456", "wiek": 30}

# Alternatywnie od Python 3.9 – operator |
skrotka = {"a": 1} | {"b": 2}    # {"a": 1, "b": 2}

# Sprawdzanie obecności klucza
if "imie" in osoba:
    print("Klucz 'imie' istnieje")

```

Iteracja po słowniku:

```

osoba = {"imie": "Jan", "wiek": 25, "miasto": "Kraków"}

# Po kluczach (domyślnie)
for klucz in osoba:
    print(klucz)

# Po wartościach
for wartosc in osoba.values():
    print(wartosc)

# Po parach (klucz, wartość)
for klucz, wartosc in osoba.items():
    print(f"{klucz}: {wartosc}")

```

Zastosowanie: liczenie wystąpień — bardzo częsty wzorec:

```

tekst = "programowanie"
wystapienia = {}

for znak in tekst:
    if znak in wystapienia:
        wystapienia[znak] += 1
    else:
        wystapienia[znak] = 1

```

```
print(wystapienia)

# {'p': 1, 'r': 2, 'o': 2, 'g': 1, 'a': 2, 'm': 1, ...}
```

Zastosowanie: mapowanie wartości — konwersja liczba → tekst:

```
slovnice = {
    1: "jeden", 2: "dwa", 3: "trzy",
    4: "cztery", 5: "pięć", 6: "sześć"
}

liczba = 3
print(slovnice[liczba])          # "trzy"
print(slovnice.get(7, "nieznana")) # "nieznana" — bezpieczne pobranie
```

4.4. Zbiory (`set`)

Zbiór to kolekcja **unikalnych** elementów — automatycznie eliminuje duplikaty. Nie zachowuje kolejności elementów. Jest zoptymalizowany do sprawdzania, czy element istnieje w zbiorze (operacja `in` jest bardzo szybka).

```
# Tworzenie zbiorów
pusty = set()          # UWAGA: {} to pusty słownik, nie zbiór!
liczby = {1, 2, 3, 4, 5}
z_listy = set([1, 2, 2, 3, 3, 3]) # {1, 2, 3} — duplikaty usunięte

# Operacje
liczby.add(6)          # Dodaje element
liczby.remove(3)       # Usuwa (błąd jeśli nie istnieje)
liczby.discard(99)     # Usuwa (BEZ błędu jeśli nie istnieje)

# Operacje na zbiorach
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a | b)          # {1, 2, 3, 4, 5, 6} — suma (union)
print(a & b)          # {3, 4} — przecięcie (intersection)
print(a - b)          # {1, 2} — różnica (difference)
```

Praktyczne zastosowanie — usuwanie duplikatów z listy:

```
lista = [1, 2, 2, 3, 3, 3, 4, 4, 4, 4]
unikalne = list(set(lista))
print(unikalne) # [1, 2, 3, 4] (kolejność może się różnić)
```

4.5. Zagnieżdżone struktury danych

W praktyce rzadko pracujemy z prostymi, płaskimi listami. Znacznie częściej spotykamy się z danymi złożonymi — listami słowników, słownikami list, a nawet listami list słowników. Umiejętność swobodnego poruszania się po takich strukturach jest kluczowa.

Lista słowników — najczęstszy wzorec:

To podstawowy sposób przechowywania kolekcji obiektów (np. lista uczniów, produktów, zamówień):


```

uczniowie = [
    {"imie": "Jan", "wiek": 17, "oceny": [4, 5, 3]},
    {"imie": "Anna", "wiek": 16, "oceny": [5, 5, 5]},
    {"imie": "Piotr", "wiek": 18, "oceny": [3, 4, 2]}
]

# Dostęp do danych
print(uczniowie[0]["imie"])          # "Jan"
print(uczniowie[1]["oceny"][2])      # 5 (trzecia ocena Anny)

# Iteracja po liście słowników
for uczen in uczniowie:
    srednia = sum(uczen["oceny"]) / len(uczen["oceny"])
    print(f"{uczen['imie']}: średnia {srednia:.2f}")

# Jan: średnia 4.00
# Anna: średnia 5.00
# Piotr: średnia 3.00

```

Sortowanie listy słowników:

```

# Sortowanie po imieniu (alfabetycznie)
posortowani = sorted(uczniowie, key=lambda u: u["imie"])

# Sortowanie po wieku (malejąco)
posortowani = sorted(uczniowie, key=lambda u: u["wiek"], reverse=True)

# Sortowanie po średniej ocen
posortowani = sorted(uczniowie, key=lambda u: sum(u["oceny"]) / len(u["oceny"]), reverse=True)

```

Słownik list — grupowanie danych:

```

katalog = {
    "owoce": ["jabłko", "banan", "gruszka"],
    "warzywa": ["marchew", "ziemniak", "cebula"],
    "napoje": ["woda", "sok", "herbata"]
}

# Dostęp
print(katalog["owoce"][0])          # "jabłko"

# Dodanie elementu do konkretnej listy
katalog["owoce"].append("pomarańcza")

# Iteracja po grupach
for kategoria, produkty in katalog.items():
    print(f"\n{kategoria.upper().capitalize()}:")
    for produkt in produkty:
        print(f"    - {produkt}")

```

Wyszukiwanie w zagnieżdżonych strukturach:

```
# Znajdź ucznia po imieniu
def znajdz_ucznia(uczniowie, imie):
    for uczen in uczniowie:
        if uczen["imie"] == imie:
            return uczen
    return None

uczen = znajdz_ucznia(uczniowie, "Anna")
if uczen:
    print(f"Znaleziono: {uczen['imie']}, wiek: {uczen['wiek']}")
else:
    print("Nie znaleziono")
```

5. Funkcje

Funkcje to wielokrotnie używalne bloki kodu, które wykonują określone zadanie. Pozwalają na organizację programu w logiczne, niezależne części. Poprawiają czytelność kodu i eliminują powtórzenia.

W Pythonie funkcje definiujemy słowem kluczowym `def`, po którym następuje nazwa funkcji, nawiasy z ewentualnymi parametrami i dwukropek.

5.1. Definicja i wywołanie funkcji

Funkcja może nie przyjmować parametrów, przyjmować jeden lub wiele, a także może (ale nie musi) zwracać wartość.

```
# Funkcja bez parametrów i bez wartości zwracanej
def powitanie():
    print("Witaj w programie!")

powitanie() # Wywołanie – samo wpisanie nazwy z nawiasami

# Funkcja z parametrami
def powitaj(imie):
    print(f"Cześć, {imie}!")

powitaj("Anna") # Cześć, Anna!

# Funkcja z wartością zwracaną (return)
def dodaj(a, b):
    return a + b

wynik = dodaj(3, 5)
print(wynik) # 8

# Funkcja z wieloma wartościami zwracanymi (krotka)
def podziel_z_reszta(a, b):
    iloraz = a // b
    reszta = a % b
    return iloraz, reszta
```

```
i, r = podziel_z_reszta(17, 5)
print(f"Iloraz: {i}, Reszta: {r}") # Iloraz: 3, Reszta: 2
```

Jeśli funkcja nie ma instrukcji `return` lub ma `return` bez wartości, domyślnie zwraca `None`.

5.2. Parametry domyślne

Parametry domyślne pozwalają na wywoływanie funkcji bez podawania wszystkich argumentów — te z wartością domyślną zostaną automatycznie uzupełnione. Parametry z wartościami domyślnymi muszą być **na końcu** listy parametrów.

```
def powitaj(imie, powitanie="Cześć"):
    print(f"{powitanie}, {imie}!")

powitaj("Jan")           # Cześć, Jan!
powitaj("Jan", "Witaj")  # Witaj, Jan!
powitaj("Jan", powitanie="Hej") # Hej, Jan!
```

Wzorzec z `None` — zamiennik przeciążania funkcji:

W Pythonie nie można mieć dwóch funkcji o tej samej nazwie, ale z różnymi parametrami (tak jak w Javie czy C++). Zamiast tego stosuje się parametr domyślny `None` i sprawdzanie warunku wewnątrz funkcji:

```
def utworz_element(wartosc=None):
    if wartosc is None:
        print("Tworzenie pustego elementu")
        return 0
    else:
        print(f"Tworzenie elementu z wartością: {wartosc}")
        return wartosc

utworz_element()      # Tworzenie pustego elementu
utworz_element(42)    # Tworzenie elementu z wartością: 42
```

Ten wzorzec jest szczególnie ważny w kontekście konstruktorów klas (omówiony szczegółowo w rozdziale 12).

5.3. Argumenty pozycyjne i nazwane

Python pozwala na przekazywanie argumentów zarówno na podstawie pozycji, jak i nazwy:

```
def info(imie, wiek, miasto):
    print(f"{imie}, {wiek} lat, {miasto}")

# Argumenty pozycyjne – kolejność ma znaczenie
info("Jan", 25, "Warszawa")

# Argumenty nazwane – kolejność NIE ma znaczenia
info(miasto="Kraków", imie="Anna", wiek=30)

# Mieszane – pozycyjne MUSZĄ być przed nazwanymi
info("Piotr", miasto="Gdańsk", wiek=22)
```

5.4. Zmienna liczba argumentów — `*args` i `**kwargs`

W sytuacjach, gdy nie wiemy z góry, ile argumentów zostanie przekazanych do funkcji, z pomocą przychodzą specjalne operatory `*args` oraz `**kwargs`. Są one szczególnie przydatne, gdy chcemy napisać elastyczne funkcje, dekoratory, lub przekazywać argumenty w dziedziczeniu klas (do `super().__init__()`).

Argumenty zmiennej długości — `*args` (Arbitrary Arguments):

Użycie pojedynczej gwiazdki `*` przed nazwą parametru sprawia, że funkcja staje się zdolna do przyjęcia dowolnej liczby argumentów **pozycyjnych**. Wewnątrz funkcji stają się one dostępne jako krotka (tuple).

```
def suma_wielu(*args):
    # args to tuple, np. (1, 2, 3, 4)
    suma = 0
    for liczba in args:
        suma += liczba
    return suma

print(suma_wielu(5, 10))          # 15
print(suma_wielu(1, 2, 3, 4, 5)) # 15
print(suma_wielu())               # 0

def powitaj_grupe(*imiona):
    for imie in imiona:
        print(f"Cześć, {imie}!")

powitaj_grupe("Jan", "Anna", "Piotr")
```

👉 **Wskazówka:** Nazwa `args` to tylko konwencja (od ang. *arguments*). Ważny jest znak `*`, możesz nazwać parametr `*imiona` czy `*liczby`.

Pary klucz-wartość — `**kwargs` (Arbitrary Keyword Arguments):

Dwie gwiazdki `**` pozwalają na przekazanie dowolnej liczby argumentów **nazwanych** (klucz=wartość). Wewnątrz funkcji są one zapisywane jako słownik (dictionary).

```
def wyswietl_dane_osobowe(**kwargs):
    # kwargs to słownik, np. {'imie': 'Jan', 'wiek': 25}
    for klucz, wartosc in kwargs.items():
        print(f"{klucz.capitalize()}: {wartosc}")

wyswietl_dane_osobowe(imie="Jan", wiek=25, miasto="Warszawa")
# Imie: Jan
# Wiek: 25
# Miasto: Warszawa

wyswietl_dane_osobowe(stanowisko="Programista", pensja=5000)
```

👉 **Wskazówka:** Podobnie jak z `args`, nazwa `kwargs` (od *keyword arguments*) to konwencja. Kluczowe są znaki `**`.

Łączenie zwykłych argumentów, `*args` i `**kwargs`:

Możesz łączyć te mechanizmy w jednej funkcji, jednak musisz bezwzględnie zachować następującą kolejność w definicji:

1. Argumenty podstawowe (pozycyjne)
2. `*args`

3. Parametry ze zdefiniowaną wartością domyślną

4. `**kwargs`

```
def zloz_zamowienie(klient, *produkty, dostawa="Standard", **opcje_dodatkowe):
    print(f"Zamówienie dla: {klient}")

    print("Mój koszyk:")
    for produkt in produkty:
        print(f"- {produkt}")

    print(f"Typ dostawy: {dostawa}")

    if opcje_dodatkowe:
        print("Opcje dodatkowe:")
        for opcja, wartosc in opcje_dodatkowe.items():
            print(f"- {opcja}: {wartosc}")

zloz_zamowienie(
    "Anna Nowak",          # argument podstawowy (klient)
    "Myszka", "Klawiatura", # *args (produkty)
    dostawa="Express",      # nadpisanie parametru domyślnego
    faktura=True,           # **kwargs (opcje_dodatkowe)
    pakowanie_prezent=False # **kwargs (opcje_dodatkowe)
)
```

Rozpakowywanie argumentów (Unpacking):

Gwiazdki działają również w drugą stronę! Jeśli masz już listę lub słownik układający się w odpowiednie parametry, możesz użyć `*` by "rozpakować" listę na argumenty pozycyjne lub `**` by rozpakować słownik na nazwane.

```
dane = [3, 4, 5]
print(suma_wielu(*dane)) # równe wywołaniu suma_wielu(3, 4, 5)

osoba = {"imie": "Marek", "wiek": 40}
wyswietl_dane_osobowe(**osoba) # równe wywołaniu wyswietl_dane_osobowe(imie="Marek", wiek=40)
```

To używany wzorec m.in. przy rozszerzaniu konstruktorów z użyciem funkcji `super()`, gdy podrzędna klasa przyjmuje szerszą liczbę argumentów, które po prostu podaje dalej do konstruktora rodzica.

5.5. Argumenty tylko pozycyjne (/) i tylko nazwane (*)

W nowszych wersjach Pythona (3.8+) wprowadzono symbole `/` oraz `*` w definicji parametrów funkcji, aby jawnie określić, jak argumenty mogą być przekazywane. Pozwala to na większą kontrolę nad API funkcji i zapobiega pomyłkom.

Argumenty tylko pozycyjne (/):

Wszystkie parametry znajdujące się **przed** symbolem `/` muszą zostać przekazane wyłącznie na podstawie pozycji. Nie można użyć ich nazw.

```
def pole_prostokata(a, b, /):
    return a * b
```

```
print(pole_prostokata(5, 10)) # ✅ OK (pozycyjne)
# print(pole_prostokata(a=5, b=10)) # ❌ TypeError: positional-only arguments
```

Argumenty tylko nazwane (*):

Wszystkie parametry znajdujące się po symbolu * muszą zostać przekazane wyłącznie jako argumenty nazwane. Często stosowane, aby uniknąć pomyłek przy wielu parametrach logicznych.

```
def wyslij_wiadomosc(tresc, *, pilne=False, do_logow=True):
    print(f"Wysyłanie: {tresc} (Pilne: {pilne})")

wyslij_wiadomosc("Dzień dobry", pilne=True) # ✅ OK
# wyslij_wiadomosc("Hej", True) # ❌ TypeError: takes 1 positional argument but 2 were given
```

Kompletna składnia mieszana:

```
def func(pos_only, /, standard, *, kw_only):
    print(pos_only, standard, kw_only)

func(10, standard=20, kw_only=30) # ✅ OK
func(10, 20, kw_only=30) # ✅ OK
```

5.6. Zasięg zmiennych (scope)

Zmienne utworzone wewnątrz funkcji są **lokalne** — nie istnieją poza funkcją. Zmienne utworzone poza funkcjami są **globalne** — są widoczne w całym programie. Zrozumienie tego mechanizmu jest kluczowe do unikania błędów.

```
x = 10 # Zmienna globalna

def funkcja():
    x = 20 # Zmienna LOKALNA — to inna zmienna niż globalne x!
    print(f"Wewnątrz: {x}") # 20

funkcja()
print(f"Na zewnątrz: {x}") # 10 — globalne x nie zostało zmienione

# Aby zmodyfikować zmienną globalną wewnątrz funkcji, trzeba użyć 'global'
def funkcja_global():
    global x
    x = 99

funkcja_global()
print(f"Po zmianie: {x}") # 99
```

W praktyce należy unikać **global** — lepiej przekazywać dane jako parametry i zwracać wyniki przez **return**.

5.7. Funkcja **lambda** — funkcje anonimowe

Lambda to krótka, jednolinijkowa funkcja anonimowa (bez nazwy). Składnia: **lambda parametry: wyrażenie**. Lambda automatycznie zwraca wynik wyrażenia — nie używamy słowa **return**.

```
# Zwykła funkcja
def kwadrat(x):
    return x ** 2

# Odpowiednik lambda
kwadrat = lambda x: x * 2

print(kwadrat(5)) # 10

# Lambda jest szczególnie przydatna jako argument sortowania
osoby = [("Jan", 30), ("Anna", 25), ("Piotr", 35)]
osoby.sort(key=lambda osoba: osoba[1])
print(osoby) # [('Anna', 25), ('Jan', 30), ('Piotr', 35)]

# Sortowanie listy słowników
pracownicy = [
    {"imie": "Jan", "pensja": 5000},
    {"imie": "Anna", "pensja": 7000},
    {"imie": "Piotr", "pensja": 4500}
]
pracownicy.sort(key=lambda p: p["pensja"], reverse=True)
# Posortowani malejąco wg pensji
```

5.8. Funkcje wbudowane — przegląd najważniejszych

Python oferuje wiele wbudowanych funkcji, które nie wymagają importowania żadnego modułu:

FUNKCJA	OPIS	PRZYKŁAD
<code>len()</code>	Długość kolekcji/napisu	<code>len([1,2,3])</code> → 3
<code>print()</code>	Wyświetlanie	<code>print("Hej")</code>
<code>input()</code>	Pobieranie danych	<code>input("Podaj: ")</code>
<code>type()</code>	Typ obiektu	<code>type(42)</code> → <code><class 'int'></code>
<code>int()</code> , <code>float()</code> , <code>str()</code>	Konwersja typów	<code>int("5")</code> → 5
<code>abs()</code>	Wartość bezwzględna	<code>abs(-7)</code> → 7
<code>round()</code>	Zaokrąglenie	<code>round(3.7)</code> → 4
<code>min()</code> , <code>max()</code>	Minimum / maksimum	<code>min(3,1,7)</code> → 1
<code>sum()</code>	Suma elementów	<code>sum([1,2,3])</code> → 6
<code>sorted()</code>	Sortowanie (nowa lista)	<code>sorted([3,1,2])</code> → [1,2,3]
<code>range()</code>	Zakres liczb	<code>range(5)</code> → 0,1,2,3,4
<code>enumerate()</code>	Iteracja z indeksem	Patrz 3.3
<code>zip()</code>	Łączenie kolekcji	Patrz niżej
<code>map()</code>	Zastosowanie funkcji	<code>map(str, [1,2])</code> → "1","2"
<code>isinstance()</code>	Sprawdzenie typu	<code>isinstance(5, int)</code> → True

Funkcja `zip()` — łączenie kolekcji:

Funkcja `zip()` łączy elementy z dwóch (lub więcej) kolekcji w pary, krotkę po krotce:

```
imiona = ["Jan", "Anna", "Piotr"]
wiek = [25, 30, 22]

# zip() tworzy pary z elementów o tym samym indeksie
for imie, w in zip(imiona, wiek):
    print(f"{imie}: {w} lat")

# Jan: 25 lat
# Anna: 30 lat
# Piotr: 22 lat

# Przydatne do tworzenia słowników
słownik = dict(zip(imiona, wiek))
print(słownik) # {'Jan': 25, 'Anna': 30, 'Piotr': 22}

# zip() z wagami – częste w algorytmach walidacji
cyfry = [1, 2, 3, 4]
wagi = [7, 3, 1, 9]
suma = sum(c * w for c, w in zip(cyfry, wagi))
print(suma) # 1*7 + 2*3 + 3*1 + 4*9 = 52
```

Funkcja `map()` — zastosowanie funkcji do każdego elementu:

```
# Konwersja listy stringów na inty
numery_str = ["1", "2", "3", "4", "5"]
numery_int = list(map(int, numery_str))
print(numery_int) # [1, 2, 3, 4, 5]

# Podniesienie do kwadratu
kwadraty = list(map(lambda x: x**2, [1, 2, 3, 4]))
print(kwadraty) # [1, 4, 9, 16]
```

6. Przetwarzanie łańcuchów znaków

Łańcuchy znaków (`str`) w Pythonie są **niemodyfikowalne** (immutable) — nie można zmienić pojedynczego znaku w istniejącym napisie. Każda operacja „modyfikująca” napis tak naprawdę tworzy nowy napis. Python natywnie obsługuje Unicode, więc polskie i inne znaki diakrytyczne działają bezproblemowo.

6.1. Podstawowe operacje na napisach

```
tekst = "Programowanie w Pythonie"

# Długość napisu
print(len(tekst)) # 24

# Indeksowanie – identycznie jak w listach
print(tekst[0]) # "P"
print(tekst[-1]) # "e"
print(tekst[5]) # "a"
```



```
# Slicing (wycinanie)
print(tekst[0:5])      # "Progr"
print(tekst[:5])       # "Progr"
print(tekst[16:])      # "Pythonie"
print(tekst[::-1])     # "einohtyP w einawomargorP" – odwrócenie

# Konkatenacja (łączenie napisów)
imie = "Jan"
nazwisko = "Kowalski"
pelne = imie + " " + nazwisko # "Jan Kowalski"

# Powtarzanie napisu
kreska = "-" * 40        # "-----"
print(kreska)
```

6.2. Metody napisów — kompletny przegląd

Metody napisów nie modyfikują oryginału — zwracają nowy napis. Oto najważniejsze:

Zmiana wielkości liter:

METODA	OPIS	PRZYKŁAD	WYNIK
<code>.upper()</code>	Wszystkie wielkie	<code>"abc".upper()</code>	<code>"ABC"</code>
<code>.lower()</code>	Wszystkie małe	<code>"ABC".lower()</code>	<code>"abc"</code>
<code>.capitalize()</code>	Pierwsza wielka	<code>"abc def".capitalize()</code>	<code>"Abc def"</code>
<code>.title()</code>	Każde słowo wielka	<code>"abc def".title()</code>	<code>"Abc Def"</code>
<code>.swapcase()</code>	Zamiana wielkości	<code>"AbC".swapcase()</code>	<code>"aBc"</code>

Wyszukiwanie i analiza:

METODA	OPIS	WYNIK GDY ZNALEZIONO	WYNIK GDY NIE
<code>.find(x)</code>	Indeks pierwszego wystąpienia	Indeks (int)	<code>-1</code>
<code>.index(x)</code>	Indeks pierwszego wystąpienia	Indeks (int)	<code>ValueError</code>
<code>.count(x)</code>	Liczba wystąpień	Liczba (int)	<code>0</code>
<code>.startswith(x)</code>	Czy zaczyna się od x	<code>True</code>	<code>False</code>
<code>.endswith(x)</code>	Czy kończy się na x	<code>True</code>	<code>False</code>

```
tekst = "Programowanie w Pythonie"

print(tekst.find("Python"))    # 16
print(tekst.find("Java"))      # -1 (nie znaleziono)
print(tekst.count("o"))        # 2
print(tekst.startswith("Progr")) # True
print("@" in "email@domena.pl") # True – operator in działa też na napisach
```

Modyfikacja (tworzenie nowego napisu):

```

tekst = "  Witaj, Świecie!  "

# Usuwanie białych znaków
print(tekst.strip())      # "Witaj, Świecie!" – z obu stron
print(tekst.lstrip())     # "Witaj, Świecie!  " – tylko z lewej
print(tekst.rstrip())     # "  Witaj, Świecie!" – tylko z prawej

# Zamiana fragmentu
tekst2 = "Programowanie w Pythonie"
print(tekst2.replace("Python", "Java")) # "Programowanie w Javie"

# Dzielenie na listę
zdanie = "jeden,dwa,trzy,cztery"
czesci = zdanie.split(",")           # ["jeden", "dwa", "trzy", "cztery"]

słowa = "Witaj swiecie jak tam".split() # Domyślnie dzieli po spacjach
# ["Witaj", "swiecie", "jak", "tam"]

# Łączenie listy w napis – metoda wywoływana na SEPARATORZE
lista = ["A", "B", "C"]
print(", ".join(lista))             # "A, B, C"
print(" | ".join(lista))            # "A | B | C"
print("".join(lista))               # "ABC"

```

6.3. Sprawdzanie rodzaju znaków

Te metody są bardzo przydatne przy walidacji danych wejściowych:

METODA	ZWRACA <code>TRUE</code> GDY...	PRZYKŁAD
<code>.isdigit()</code>	Same cyfry	<code>"123".isdigit()</code> → <code>True</code>
<code>.isalpha()</code>	Same litery	<code>"abc".isalpha()</code> → <code>True</code>
<code>.isalnum()</code>	Litery i/lub cyfry	<code>"abc123".isalnum()</code> → <code>True</code>
<code>.isspace()</code>	Same białe znaki	<code>" ".isspace()</code> → <code>True</code>
<code>.isupper()</code>	Same wielkie litery	<code>"ABC".isupper()</code> → <code>True</code>
<code>.islower()</code>	Same małe litery	<code>"abc".islower()</code> → <code>True</code>

6.4. Kody znaków — `ord()` i `chr()`

Te dwie funkcje pozwalają na konwersję między znakiem a jego kodem numerycznym (ASCII/Unicode). Są fundamentalne przy algorytmach szyfrujących, przetwarzaniu na poziomie pojedynczych znaków i operacjach na alfabecie.

FUNKCJA	KIERUNEK	PRZYKŁAD	WYNIK
<code>ord(znak)</code>	Znak → liczba	<code>ord('A')</code>	65
<code>chr(liczba)</code>	Liczba → znak	<code>chr(65)</code>	'A'

Kluczowe zakresy kodów ASCII:

ZNAKI	ZAKRES KODÓW	PIERWSZY	OSTATNI
Cyfry 0 – 9	48–57	<code>ord('0')</code> = 48	<code>ord('9')</code> = 57
Wielkie A – Z	65–90	<code>ord('A')</code> = 65	<code>ord('Z')</code> = 90
Małe a – z	97–122	<code>ord('a')</code> = 97	<code>ord('z')</code> = 122
Spacja	32	<code>ord(' ')</code> = 32	—

```
print(ord('A'))    # 65
print(ord('a'))    # 97
print(ord('0'))    # 48
print(chr(65))     # 'A'
print(chr(97))     # 'a'
```

Przesunięcie znaku w alfabecie z zawijaniem:

To kluczowy wzorec stosowany w szyfrach i algorytmach operujących na literach. Zawijanie oznacza, że po **z** wracamy do **a**:

```
def przesun_znak(znak, przesuniecie):
    if 'a' <= znak <= 'z':
        numer = ord(znak) - ord('a')      # a=0, b=1, ..., z=25
        nowy_numer = (numer + przesuniecie) % 26 # Modulo 26 = zawijanie
        return chr(nowy_numer + ord('a'))
    return znak # Znak spoza zakresu a-z zwracamy bez zmian

print(przesun_znak('a', 3))    # 'd'
print(przesun_znak('x', 5))    # 'c' (zawinięcie: x→y→z→a→b→c)
print(przesun_znak('d', -3))   # 'a'
print(przesun_znak(' ', 5))    # ' ' (spacja – bez zmian)
```

6.5. Iteracja po znakach i budowanie nowego napisu

Ponieważ napisy w Pythonie są niemodyfikowalne, tworzenie nowego napisu wymaga jednej z dwóch technik:

```
tekst = "Python"

# Technika 1: Konkatencja (łączenie) – prosta, ale wolna dla długich napisów
nowy = ""
for znak in tekst:
    nowy += znak.upper()
print(nowy) # "PYTHON"

# Technika 2: Lista + join – efektywniejsza
nowy = "".join([znak.upper() for znak in tekst])
print(nowy) # "PYTHON"
```

6.6. Obsługa polskich znaków

Python 3 natywnie obsługuje Unicode, więc polskie znaki nie wymagają żadnej specjalnej konfiguracji. Warto jednak pamiętać o nich przy operacjach na literach (np. zliczanie samogłosek):

```

samogloski = "aąęiouóyAĄĘĘIOUÓY"

def policz_samogloski(tekst):
    licznik = 0
    for znak in tekst:
        if znak in samogloski:
            licznik += 1
    return licznik

print(policz_samogloski("Zażółć gęślą jaźń")) # 6

```

6.7. Przydatne wzorce przetwarzania tekstu

Kilka często spotykanych wzorców przetwarzania napisów, które warto znać:

Odwracanie kolejności słów w zdaniu:

```

zdanie = "Python jest super"
słowa = zdanie.split()          # ["Python", "jest", "super"]
słowa.reverse()                 # ["super", "jest", "Python"]
wynik = " ".join(słowa)         # "super jest Python"

# Lub jednolinijkowo:
wynik = " ".join(zdanie.split()[::-1])

```

Sprawdzanie czy napis jest palindromem:

Palindrom to napis, który czyta się tak samo od przodu i od tyłu (np. "kajak", "racecar"):

```

def jest_palindromem(tekst):
    tekst = tekst.lower().replace(" ", "") # Ignorujemy wielkość liter i spacje
    return tekst == tekst[::-1]

print(jest_palindromem("kajak"))      # True
print(jest_palindromem("Python"))     # False
print(jest_palindromem("Kobyła ma mały bok".replace(" ", ""))) # True

```

Zamiana pierwszej litery każdego słowa na wielką:

```

tekst = "jan kowalski z warszawy"
wynik = tekst.title()
print(wynik) # "Jan Kowalski Z Warszawy"

# Ręcznie – gdy potrzebujemy większej kontroli:
słowa = tekst.split()
wynik = " ".join(s[0].upper() + s[1:] for s in słowa)

```

Zliczanie słów w tekście:

```

tekst = "Python jest super i Python jest prosty"
slova = tekst.lower().split()

wystapienia = {}
for slovo in slova:
    wystapienia[slovo] = wystapienia.get(slovo, 0) + 1

for slovo, ile in sorted(wystapienia.items(), key=lambda x: x[1], reverse=True):
    print(f"{slovo}: {ile}")

# python: 2
# jest: 2
# super: 1
# i: 1
# prosty: 1

```

Metoda `dict.get(klucz, domyslna)` jest tu kluczowa — zwraca wartość domyślną `0` zamiast rzucania `KeyError`, gdy klucz nie istnieje. Dzięki temu nie potrzebujemy bloku `if/else`.

6.8. Usuwanie powtarzających się znaków

Częstym zadaniem przy pracy z tekstem jest usuwanie powtarzających się bezpośrednio po sobie znaków (sąsiednich duplikatów). Może to być przydatne np. do korekty błędów typograficznych (zamiana `"banannowy"` na `"bananowy"`).

Algorytm ten przegląda tekst znak po znaku i buduje nowy napis, dołączając tylko te znaki, które różnią się od swojego bezpośredniego poprzednika.

```

def usun_powtarzajace_sie_znaki(tekst):
    if not tekst:
        return ""

    czysty_tekst = tekst[0]
    for i in range(1, len(tekst)):
        if tekst[i] != tekst[i-1]:
            czysty_tekst += tekst[i]
    return czysty_tekst

# Przykład użycia:
print(usun_powtarzajace_sie_znaki("banannowy")) # "bananowy"
print(usun_powtarzajace_sie_znaki("heeeelloo")) # "heło"

```

7. Obsługa plików

Praca z plikami to podstawowa umiejętność programisty. Python oferuje prosty i wygodny interfejs do odczytu i zapisu plików tekstowych oraz binarnych. Najważniejszą zasadą jest **zawsze zamykanie pliku po zakończeniu pracy** — najwygodniej osiągnąć to za pomocą instrukcji `with`.

7.1. Otwieranie i zamykanie plików

Funkcja `open()` otwiera plik i zwraca obiekt pliku. Przyjmuje dwa główne argumenty: ścieżkę do pliku i tryb otwarcia.

Tryby otwarcia pliku:

TRYB	OPIS	TWORZENIE PLIKU	NADPISUJE
"r"	Odczyt (read) — domyślny	✗ Plik musi istnieć	—
"w"	Zapis (write) — nadpisuje	✓ Tak	✓ Tak
"a"	Dopisywanie (append)	✓ Tak	✗ Dopisuje na koniec
"x"	Tworzenie (exclusive)	✓ Tak	✗ Błąd jeśli istnieje
"r+"	Odczyt i zapis	✗ Plik musi istnieć	Zależy od pozycji
"rb"	Odczyt binarny	✗ Plik musi istnieć	—
"wb"	Zapis binarny	✓ Tak	✓ Tak

Instrukcja `with` — zalecany sposób pracy z plikami:

Blok `with` automatycznie zamyka plik po zakończeniu bloku kodu, nawet jeśli wystąpi błąd. To najlepszy i najbezpieczniejszy sposób pracy z plikami:

```
# ✓ ZALECANY sposób — with
with open("dane.txt", "r") as plik:
    zawartosc = plik.read()
    print(zawartosc)

# Plik jest automatycznie zamknięty po wyjściu z bloku with

# ✗ STARSZY sposób — wymaga ręcznego zamykania
plik = open("dane.txt", "r")
zawartosc = plik.read()
plik.close() # łatwo zapomnieć!
```

7.2. Odczyt pliku tekstowego

Python oferuje kilka sposobów odczytu zawartości pliku. Wybór zależy od rozmiaru pliku i tego, w jaki sposób chcemy przetworzyć dane.

`read()` — odczyt całego pliku jako jednego stringa:

```
with open("dane.txt", "r", encoding="utf-8") as plik:
    calosc = plik.read()
    print(calosc)
    print(f"Plik ma {len(calosc)} znaków")
```

Parametr `encoding="utf-8"` zapewnia poprawne odczytywanie polskich znaków i powinien być stosowany zawsze.

`readline()` — odczyt jednej linii:

```
with open("dane.txt", "r", encoding="utf-8") as plik:
    linia1 = plik.readline() # Pierwsza linia (ze znakiem \n na końcu)
    linia2 = plik.readline() # Druga linia
    print(linia1.strip())    # strip() usuwa \n z końca
    print(linia2.strip())
```

`readlines()` — odczyt wszystkich linii jako listy:

```
with open("dane.txt", "r", encoding="utf-8") as plik:
    linie = plik.readlines()    # Lista stringów — każdy to jedna linia

for i, linia in enumerate(linie, start=1):
    print(f"Linia {i}: {linia.strip()}")
```

Iteracja po liniach — najbardziej efektywny sposób:

Najbardziej „pythonowym” i efektywnym sposobem czytania pliku linia po linii jest bezpośrednia iteracja po obiekcie pliku. Nie ładuje całego pliku do pamięci:

```
with open("dane.txt", "r", encoding="utf-8") as plik:
    for linia in plik:
        linia = linia.strip()    # Usuwamy \n z końca
        if linia:               # Pomijamy puste linie
            print(linia)
```

7.3. Zapis do pliku tekstowego

`write()` — zapis tekstu:

Metoda `write()` nie dodaje automatycznie znaku nowej linii — trzeba go dodać samodzielnie jako `\n`:

```
with open("wynik.txt", "w", encoding="utf-8") as plik:
    plik.write("Pierwsza linia\n")
    plik.write("Druga linia\n")
    plik.write(f"Wynik: {42}\n")
```

Tryb `"w"` (write) **nadpisuje** cały plik — jeśli plik istniał wcześniej, jego zawartość zostanie usunięta!

`writelines()` — zapis listy linii:

```
linie = ["Linia 1\n", "Linia 2\n", "Linia 3\n"]

with open("wynik.txt", "w", encoding="utf-8") as plik:
    plik.writelines(linie)
```

Dopisywanie na koniec pliku — tryb `"a"` (append):

Tryb `"a"` nie nadpisuje pliku — dodaje nową treść na końcu. Jeśli plik nie istnieje, zostanie utworzony:

```
with open("log.txt", "a", encoding="utf-8") as plik:
    plik.write("Nowy wpis w logu\n")
    plik.write("Kolejny wpis\n")
```

7.4. Praca z plikami CSV (dane tabelaryczne)

Pliki CSV (Comma-Separated Values) to prosty format przechowywania danych tabelarycznych, gdzie wartości w wierszu są oddzielone separatorem (zwykle przecinkiem lub średnikiem).

Ręczne parsowanie CSV (bez modułu):

```
# Odczyt CSV
dane = []
with open("uczniowie.csv", "r", encoding="utf-8") as plik:
    for linia in plik:
        linia = linia.strip()
        if linia:
            pola = linia.split(";") # Dzielenie po separatorze
            dane.append(pola)

# Wyświetlanie
for wiersz in dane:
    print(wiersz)

# ['Imię', 'Nazwisko', 'Ocena']
# ['Jan', 'Kowalski', '4']
# ['Anna', 'Nowak', '5']
```

Zapis danych w formacie CSV:

```
uczniowie = [
    ["Imię", "Nazwisko", "Ocena"],
    ["Jan", "Kowalski", "4"],
    ["Anna", "Nowak", "5"],
    ["Piotr", "Wiśniewski", "3"]
]

with open("uczniowie.csv", "w", encoding="utf-8") as plik:
    for wiersz in uczniowie:
        linia = ",".join(wiersz) # Łączenie listy w string z separatorem
        plik.write(linia + "\n")
```

Moduł `csv` — profesjonalne podejście:

Python posiada wbudowany moduł `csv`, który obsługuje trudniejsze przypadki (np. wartości zawierające separator):

```
import csv

# Odczyt
with open("dane.csv", "r", encoding="utf-8") as plik:
    czytnik = csv.reader(plik, delimiter=";")
    for wiersz in czytnik:
        print(wiersz) # Lista wartości

# Zapis
with open("dane.csv", "w", encoding="utf-8", newline="") as plik:
    pisarz = csv.writer(plik, delimiter=";")
    pisarz.writerow(["Imię", "Wiek", "Miasto"])
    pisarz.writerow(["Jan", "25", "Warszawa"])
    pisarz.writerow(["Anna", "30", "Kraków"])
```


7.5. Sprawdzanie istnienia pliku

Przed otwarciem pliku warto sprawdzić, czy istnieje. Do tego służy moduł `os`:

```
import os

sciezka = "dane.txt"

# Sprawdzenie czy plik istnieje
if os.path.exists(sciezka):
    print("Plik istnieje")
else:
    print("Plik nie istnieje")

# Sprawdzenie czy to plik (nie folder)
if os.path.isfile(sciezka):
    print("To jest plik")

# Sprawdzenie czy to folder
if os.path.isdir("moj_folder"):
    print("To jest folder")

# Rozmiar pliku w bajtach
if os.path.exists(sciezka):
    rozmiar = os.path.getsize(sciezka)
    print(f"Rozmiar: {rozmiar} bajtów")
```

7.6. Obsługa błędów przy pracy z plikami

Pliki mogą nie istnieć, możemy nie mieć uprawnień, dysk może być pełny. Dlatego warto zabezpieczyć się blokiem `try/except`:

```
try:
    with open("nieistniejacy.txt", "r") as plik:
        zawartosc = plik.read()
except FileNotFoundError:
    print("Błąd: plik nie został znaleziony!")
except PermissionError:
    print("Błąd: brak uprawnień do odczytu pliku!")
except Exception as e:
    print(f"Wystąpił nieoczekiwany błąd: {e}")
```

7.7. Operacje na ścieżkach — moduł `os.path`

```
import os

sciezka = "/Users/jan/dokumenty/raport.txt"

# Wyodrębnianie nazwy pliku
print(os.path.basename(sciezka))    # "raport.txt"

# Wyodrębnianie katalogu
print(os.path.dirname(sciezka))      # "/Users/jan/dokumenty"
```

```
# Rozdzielenie nazwy i rozszerzenia
nazwa, rozszerzenie = os.path.splitext("raport.txt")
print(nazwa)          # "raport"
print(rozszerzenie)   # ".txt"

# Łączenie ścieżek (bezpieczne – działa na każdym systemie)
nowa = os.path.join("dokumenty", "raport.txt")
print(nowa)           # "dokumenty/raport.txt" (lub "dokumenty\raport.txt" na Windows)
```

7.8. Tworzenie i usuwanie plików/folderów

```
import os

# Tworzenie folderu
os.makedirs("wyniki/raporty", exist_ok=True)
# exist_ok=True – nie rzuca błędu jeśli folder już istnieje

# Wylistowanie zawartości folderu
pliki = os.listdir(".") # "." = bieżący folder
for nazwa in pliki:
    print(nazwa)

# Usuwanie pliku
if os.path.exists("tymczasowy.txt"):
    os.remove("tymczasowy.txt")

# Zmiana nazwy pliku
os.rename("stara_nazwa.txt", "nowa_nazwa.txt")
```

7.9. Wczytywanie danych z pliku do tablicy

To bardzo częsty wzorzec — wczytanie liczb z pliku do listy:

```
# Plik dane.txt zawiera po jednej liczbie na linię:
# 42
# 17
# 85
# 3

def wczytaj_liczby(nazwa_pliku):
    """Wczytuje liczby z pliku – jedna liczba na linię."""
    liczby = []
    with open(nazwa_pliku, "r", encoding="utf-8") as plik:
        for linia in plik:
            linia = linia.strip()
            if linia:
                liczby.append(int(linia))
    return liczby

# Użycie
dane = wczytaj_liczby("dane.txt")
```

```
print(dane)          # [42, 17, 85, 3]
print(sum(dane))     # 147
print(max(dane))     # 85
```

Zapisywanie tablicy do pliku:

```
def zapisz_liczby(nazwa_pliku, liczby):
    """Zapisuje listę liczb do pliku – jedna na linię."""
    with open(nazwa_pliku, "w", encoding="utf-8") as plik:
        for liczba in liczby:
            plik.write(f"{liczba}\n")

wyniki = [10, 20, 30, 40, 50]
zapisz_liczby("wyniki.txt", wyniki)
```

7.10. Praca z plikami JSON

JSON (JavaScript Object Notation) to popularny format wymiany danych, który w Pythonie naturalnie mapuje się na słowniki i listy. Moduł `json` jest wbudowany.

Zapis danych do pliku JSON:

```
import json

dane = {
    "imie": "Jan",
    "wiek": 25,
    "oceny": [4, 5, 3, 5, 4],
    "adres": {
        "miasto": "Warszawa",
        "ulica": "Nowa 15"
    }
}

with open("dane.json", "w", encoding="utf-8") as plik:
    json.dump(dane, plik, indent=4, ensure_ascii=False)
    # indent=4 – czytelne formatowanie z wcięciami
    # ensure_ascii=False – zachowuje polskie znaki
```

Plik `dane.json` będzie wyglądał tak:

```
{
  "imie": "Jan",
  "wiek": 25,
  "oceny": [4, 5, 3, 5, 4],
  "adres": {
    "miasto": "Warszawa",
    "ulica": "Nowa 15"
  }
}
```

Odczyt danych z pliku JSON:

```
import json

with open("dane.json", "r", encoding="utf-8") as plik:
    dane = json.load(plik)

print(dane["imie"])           # Jan
print(dane["oceny"])          # [4, 5, 3, 5, 4]
print(dane["adres"]["miasto"]) # Warszawa
```

Konwersja między JSON a stringiem (bez pliku):

```
import json

# Słownik → string JSON
słownik = {"klucz": "wartość", "liczba": 42}
json_str = json.dumps(słownik, ensure_ascii=False)
print(json_str)           # {"klucz": "wartość", "liczba": 42}
print(type(json_str))     # <class 'str'>

# String JSON → słownik
odczytany = json.loads(json_str)
print(odczytany["klucz"]) # wartość
print(type(odczytany))   # <class 'dict'>
```

Mapowanie typów Python ↔ JSON:

PYTHON	JSON
dict	{ } (obiekt)
list, tuple	[] (tablica)
str	"string"
int, float	number
True / False	true / false
None	null

7.11. Podsumowanie metod pracy z plikami

OPERACJA	TRYB	METODA	UWAGI
Odczyt całego pliku	"r"	plik.read()	Cały plik jako jeden string
Odczyt linia po linii	"r"	for linia in plik:	Najefektywniejszy sposób
Odczyt wszystkich linii	"r"	plik.readlines()	Lista stringów
Zapis (nadpisanie)	"w"	plik.write(tekst)	Kasuje poprzednią zawartość!
Dopisanie na koniec	"a"	plik.write(tekst)	Bez kasowania
Zapis wielu linii	"w"	plik.writelines(lista)	Nie dodaje \n automatycznie

8. Operacje matematyczne

8.1. Wbudowane funkcje matematyczne

Te funkcje nie wymagają importowania żadnego modułu:

FUNKCJA	OPIS	PRZYKŁAD	WYNIK
<code>abs(x)</code>	Wartość bezwzględna	<code>abs(-5)</code>	5
<code>round(x)</code>	Zaokrąglenie	<code>round(3.7)</code>	4
<code>round(x, n)</code>	Zaokr. do n miejsc	<code>round(3.14159, 2)</code>	3.14
<code>pow(x, y)</code>	Potęga x ^y	<code>pow(2, 10)</code>	1024
<code>min(a, b, ...)</code>	Minimum	<code>min(3, 1, 7)</code>	1
<code>max(a, b, ...)</code>	Maksimum	<code>max(3, 1, 7)</code>	7
<code>divmod(a, b)</code>	Iloraz i reszta	<code>divmod(17, 5)</code>	(3, 2)

```
print(abs(-5))      # 5
print(round(3.7))   # 4
print(round(3.14159, 2)) # 3.14
print(pow(2, 10))   # 1024
print(min(3, 1, 7, 2)) # 1
print(max(3, 1, 7, 2)) # 7

iloraz, reszta = divmod(17, 5)
print(iloraz, reszta) # 3 2
```

8.2. Moduł `math`

Moduł `math` zawiera bardziej zaawansowane funkcje matematyczne. Wymaga importowania:

```
import math

print(math.sqrt(16))      # 4.0    - pierwiastek kwadratowy
print(math.floor(3.7))    # 3      - zaokrąglenie w DÓŁ (podłoga)
print(math.ceil(3.2))     # 4      - zaokrąglenie w GÓRĘ (sufit)
print(math.pi)            # 3.141592653589793
print(math.e)             # 2.718281828459045

# Alternatywa dla math.sqrt() - potęgowanie ułamkowe
print(16 ** 0.5)          # 4.0
print(27 ** (1/3))        # 3.0 (pierwiastek sześcienny)
```

Typowe zastosowanie `math.sqrt()` — ograniczenie pętli do $\sqrt{n}$:

W wielu algorytmach (np. sito Eratostenesa, sprawdzanie czy liczba jest pierwsza) wystarczy sprawdzić dzielniki do pierwiastka z n:

```
import math
```

```
n = 100
limit = int(math.sqrt(n)) # 10
for i in range(2, limit + 1):
    print(i, end=" ")
# 2 3 4 5 6 7 8 9 10
```

8.3. Dzielenie — porównanie operatorów

Zrozumienie różnic między operatorami dzielenia jest kluczowe:

WYRAŻENIE	WYNIK	TYP	WYJAŚNIENIE
<code>10 / 3</code>	<code>3.3333...</code>	<code>float</code>	Dzielenie dokładne
<code>10 / 2</code>	<code>5.0</code>	<code>float</code>	Mimo wyniku całkowitego — zawsze float!
<code>10 // 3</code>	<code>3</code>	<code>int</code>	Dzielenie całkowite (zaokr. w dół)
<code>-10 // 3</code>	<code>-4</code>	<code>int</code>	W dół = w kierunku $-\infty$, nie ku zeru!
<code>10 % 3</code>	<code>1</code>	<code>int</code>	Reszta: $10 = 3 \cdot 3 + 1$

8.4. Konwersja systemów liczbowych

W programowaniu często pracujemy z liczbami w systemach innych niż dziesiętny — binarnym (2), ósemkowym (8) i szesnastkowym (16). Python oferuje wbudowane funkcje do konwersji między nimi.

Funkcje konwertujące liczbę na string w danym systemie:

FUNKCJA	SYSTEM	PREFIKS	PRZYKŁAD	WYNIK
<code>bin(x)</code>	Dwójkowy (binarny)	<code>0b</code>	<code>bin(42)</code>	<code>'0b101010'</code>
<code>oct(x)</code>	Ósemkowy (oktalny)	<code>0o</code>	<code>oct(42)</code>	<code>'0o52'</code>
<code>hex(x)</code>	Szesnastkowy (heksadecymalny)	<code>0x</code>	<code>hex(42)</code>	<code>'0x2a'</code>

```
liczba = 42

print(bin(liczba))    # 0b101010
print(oct(liczba))    # 0o52
print(hex(liczba))    # 0x2a

# Bez prefiksu — usuwamy pierwsze 2 znaki
print(bin(liczba)[2:]) # 101010
print(hex(liczba)[2:]) # 2a

# Formatowanie z f-stringiem
print(f"{liczba:b}")   # 101010 (binarnie, bez prefiksu)
print(f"{liczba:o}")   # 52 (ósemkowo)
print(f"{liczba:x}")   # 2a (szesnastkowo, małe litery)
print(f"{liczba:X}")   # 2A (szesnastkowo, WIELKIE litery)
print(f"{liczba:08b}") # 00101010 (8-bitowy zapis binarny)
```

Konwersja z dowolnego systemu na dziesiętny — `int(string, baza)` :

```
# String w systemie binarnym → int
print(int("101010", 2))    # 42
print(int("0b101010", 2))  # 42 (z prefiksem też działa)

# Ósemkowy → int
print(int("52", 8))        # 42

# Szesnastkowy → int
print(int("2a", 16))       # 42
print(int("FF", 16))      # 255

# Dowolna baza (2-36)
print(int("z", 36))        # 35
```

Przydatne operacje na cyfrach liczby:

```
# Ile bitów potrzeba do zapisania liczby
liczba = 42
print(liczba.bit_length())  # 6 (bo 101010 ma 6 cyfr)

# Zamiana cyfr liczby na listę
cyfry = [int(c) for c in str(12345)]
print(cyfry) # [1, 2, 3, 4, 5]

# Suma cyfr liczby
print(sum(int(c) for c in str(12345))) # 15
```

9. Moduł random — generowanie losowych danych

Moduł `random` dostarcza narzędzia do generowania danych pseudolosowych. Są to wartości wyglądające na losowe, ale generowane algorytmicznie — przy tym samym „ziarnie” (seed) zawsze wygenerują te same wyniki. W praktyce dla potrzeb programowania to wystarczające.

Warto wiedzieć, że moduł `random` **nie jest** bezpieczny kryptograficznie — nie powinien być używany do generowania haseł czy kluczy szyfrowania. Do tych celów Python oferuje moduł `secrets`. Jednak do symulacji, gier, testowania i algorytmów `random` jest w zupełności wystarczający.

9.1. Importowanie

```
import random

# Lub importowanie wybranych funkcji
from random import randint, choice, sample, shuffle
```

9.2. Generowanie liczb losowych

FUNKCJA	OPIS	ZAKRES	TYP WYNIKU
<code>random.randint(a, b)</code>	Losowa liczba całkowita	[a, b] (oba włącznie)	<code>int</code>
<code>random.random()</code>	Losowa liczba zmiennoprzecinkowa	[0.0, 1.0)	<code>float</code>
<code>random.uniform(a, b)</code>	Losowa liczba zmiennoprzecinkowa	[a, b]	<code>float</code>

```
import random

# Losowa liczba całkowita z zakresu [a, b] – OBA KOŃCE WŁĄCZNIE
x = random.randint(1, 100)    # Losowa 1-100
x = random.randint(1, 6)      # Symulacja rzutu kością (1-6)

# Losowa liczba zmiennoprzecinkowa z zakresu [0.0, 1.0)
x = random.random()           # np. 0.7342...

# Losowa zmiennoprzecinkowa z zakresu [a, b]
x = random.uniform(1.0, 10.0) # np. 5.247...
```

9.3. Losowanie z kolekcji

```
import random

kolory = ["czerwony", "zielony", "niebieski", "żółty"]

# Losowy element z listy
kolor = random.choice(kolory)    # np. "zielony"

# Losowanie n unikalnych elementów (BEZ powtórzeń)
wybrane = random.sample(kolory, 2) # np. ["żółty", "czerwony"]

# Losowanie unikalnych liczb z zakresu
unikalne = random.sample(range(1, 50), 6) # 6 unikalnych z 1-49

# Mieszanie listy na miejscu (modyfikuje oryginał)
random.shuffle(kolory)
print(kolory) # np. ["niebieski", "żółty", "czerwony", "zielony"]
```

9.4. Wypełnianie tablicy losowymi danymi

```
import random

# Tablica 50 losowych liczb z zakresu 1-100
tablica = [random.randint(1, 100) for _ in range(50)]

# Funkcja generująca tablicę
def wypelnij_losowo(rozmiar, minimum, maksimum):
    return [random.randint(minimum, maksimum) for _ in range(rozmiar)]

dane = wypelnij_losowo(30, 1, 100)
```



```
print(dane)
print(f"Długość: {len(dane)}")
```

Zmienna `_` (podkreślnik) to konwencja Pythona oznaczająca „ta zmienna nie będzie używana”. Tu jest potrzebna tylko jako licznik iteracji.

9.5. Losowanie bez powtórzeń — trzy sposoby

Losowanie unikalnych wartości (np. 6 różnych numerów z zakresu 1–49) można zrealizować na kilka sposobów:

```
import random

# Sposób 1: random.sample() — NAJPROSTSZY I ZALECANY
zestaw = random.sample(range(1, 50), 6)
# Zwraca listę 6 unikalnych liczb z zakresu 1-49

# Sposób 2: Pętla z kontrolą duplikatów
zestaw = []
while len(zestaw) < 6:
    liczba = random.randint(1, 49)
    if liczba not in zestaw: # Sprawdzamy czy już nie wylosowaliśmy
        zestaw.append(liczba)

# Sposób 3: Przez zbiór (set) — automatycznie odrzuca duplikaty
zestaw = set()
while len(zestaw) < 6:
    zestaw.add(random.randint(1, 49))
zestaw = list(zestaw) # Konwersja zbioru na listę
```

9.6. Ustawienie ziarna (seed) — powtarzalność wyników

Dla celów testowania i debugowania można ustawić „ziarno” generatora, co gwarantuje te same „losowe” wyniki:

```
import random

random.seed(42) # Ustawienie ziarna
print(random.randint(1, 100)) # Zawsze 82
print(random.randint(1, 100)) # Zawsze 15

random.seed(42) # Reset ziarna
print(random.randint(1, 100)) # Znowu 82
```

10. Programowanie obiektowe — podstawy

Programowanie obiektowe (OOP — Object-Oriented Programming) to paradygmat, w którym program jest organizowany wokół **obiektów** — autonomicznych bytów łączących dane (atrybuty) i zachowania (metody).

10.1. Definicja klasy

Klasa to „szablon” lub „projekt”, na podstawie którego tworzymy obiekty. Nazwy klas w Pythonie zapisujemy zgodnie z konwencją **PascalCase** (np. `MojaKlasa`).

```
class Robot:
    """Prosta klasa reprezentująca robota."""
    pass

# Tworzenie instancji (obiektu)
moj_robot = Robot()
print(type(moj_robot)) # <class '__main__.Robot'>
```

10.2. Konstruktor `__init__` i pola instancji

Metoda `__init__` jest wywoływana automatycznie podczas tworzenia nowego obiektu. Służy do nadawania obiektowi początkowego stanu (inicjalizacji pól).

```
class Samochod:
    def __init__(self, marka, model, rok):
        # Pola instancji (unikalne dla każdego obiektu)
        self.marka = marka
        self.model = model
        self.rok = rok
        self.czy_odpalony = False

# Przekazujemy argumenty z pominięciem 'self'
auto1 = Samochod("Toyota", "Corolla", 2022)
auto2 = Samochod("Tesla", "Model 3", 2023)

print(auto1.marka) # Toyota
print(auto2.marka) # Tesla
```

10.3. Metody instancji

Metody to funkcje zdefiniowane wewnątrz klasy, które mają dostęp do danych obiektu poprzez parametr `self`.

```
class Pies:
    def __init__(self, imie):
        self.imie = imie

    def szczekaj(self):
        print(f"{self.imie} mówi: Hau! Hau!")

    def przedstaw_sie(self, tytul="Pies"):
        print(f"Jestem {tytul} i nazywam się {self.imie}.")

reksio = Pies("Reksio")
reksio.szczekaj() # Reksio mówi: Hau! Hau!
reksio.przedstaw_sie("Azor") # Jestem Azor i nazywam się Reksio.
```

10.4. `self` — czym jest i dlaczego jest konieczne

`self` to pierwszy parametr każdej metody instancji. Reprezentuje on **konkretny obiekt**, na którym metoda została wywołana.

- **Dlaczego `self`?** Python nie używa słowa kluczowego `this` jak Java czy C#, a przekazywanie referencji do obiektu musi być jawne w definicji (choć niejawnie przy wywołaniu).
- **Jak to działa pod spodem?** Wywołanie `auto.jedz(10)` jest przez Pythona zamieniane na `Samochod.jedz(auto, 10)`.

10.5. Metoda `__str__` — reprezentacja tekstowa obiektu

Pozwala zdefiniować, co ma się wyświetlić, gdy prześlemy obiekt do funkcji `print()` lub `str()`.

```
class Ksiazka:
    def __init__(self, tytul, autor):
        self.tytul = tytul
        self.autor = autor

    def __str__(self):
        return f'"{self.tytul}" autorstwa {self.autor}'

ksiazka = Ksiazka("Wiedźmin", "Andrzej Sapkowski")
print(ksiazka) # 'Wiedźmin' autorstwa Andrzej Sapkowski
```

10.6. `__repr__` vs `__str__` — dwie reprezentacje tekstowe

METODA	CEL	GRUPA DOCELOWA	WYWOŁANIE
<code>__str__</code>	Nieformalna, ładna reprezentacja.	Użytkownik końcowy.	<code>print(obj)</code> , <code>str(obj)</code>
<code>__repr__</code>	Formalna, techniczna informacja.	Programista (debugowanie).	Konsola, f-string <code>!r</code> , listy.

Zasada: Jeśli nie zdefiniujesz `__str__`, Python użyje `__repr__`. Jeśli nie zdefiniujesz żadnej, zobaczysz adres w pamięci.

```
class Produkt:
    def __init__(self, nazwa, cena):
        self.nazwa = nazwa
        self.cena = cena

    def __repr__(self):
        return f"Produkt(nazwa='{self.nazwa}', cena={self.cena})"

p = Produkt("Chleb", 4.50)
print(p) # Produkt(nazwa='Chleb', cena=4.5)
lista = [p]
print(lista) # [Produkt(nazwa='Chleb', cena=4.5)]
```

11. Programowanie obiektowe — hermetyzacja

Hermetyzacja (enkapsulacja) to ukrywanie danych wewnętrznych obiektu i udostępnianie ich tylko poprzez kontrolowany interfejs. Zapobiega to przypadkowej modyfikacji stanu obiektu.

11.1. Poziomy dostęp w Pythonie

W Pythonie nie ma ścisłych ograniczeń dostępu (jak `private` w Java). Stosujemy konwencje nazewnictwa.

TYP	ZAPIS	DOSTĘPNOŚĆ	ZNACZENIE
Publiczne	<code>self.nazwa</code>	Wszędzie.	Standardowe pole.
Chronione	<code>self._nazwa</code>	Klasa i podklasy.	"Proszę, nie ruszaj mnie poza klasą" (konwencja).
Prywatne	<code>self.__nazwa</code>	Tylko klasa.	Silnie ukryte (uruchamia mechanizm Name Mangling).

Name Mangling: Atrybut `self.__tajne` w klasie `Konto` jest zamieniany na `_Konto__tajne`. Można się do niego dostać z zewnątrz, ale jest to celowo utrudnione.

11.2. Gettery i settery — klasyczne metody dostępu

W klasycznym podejściu (znanym np. z Javy) zamiast bezpośrednio modyfikować atrybuty obiektu, tworzymy oddzielne metody do ich pobierania (`get`) i ustawiania (`set`). Umożliwia to zablokowanie ustawienia błędnych wartości, ale wymusza stosowanie nawiasów `()` przy każdym użyciu.

Przykład — klasyczny getter i setter z walidacją:

```
class KontoBankowe:
    def __init__(self, saldo):
        self.__saldo = 0
        self.set_saldo(saldo) # Użycie settera już w konstruktorze dla walidacji

    # Getter — metoda pobierająca
    def get_saldo(self):
        return self.__saldo

    # Setter — metoda ustawiająca
    def set_saldo(self, kwota):
        if kwota >= 0:
            self.__saldo = kwota
        else:
            print("Błąd: Saldo nie może być ujemne!")

konto = KontoBankowe(100)
konto.set_saldo(150)          # Ustawienie nowej wartości (wywołanie metody)
print(konto.get_saldo())      # Pobranie wartości → 150
konto.set_saldo(-50)          # Próba przypisania ujemnej wartości zakończona błędem
```

11.3. Dekorator `@property` — pythoniczny getter/setter

W Pythonie klasyczne gettery i settery są rzadko używane, ponieważ wymuszają zmianę składni w wywołaniach. Lepszym sposobem jest dekorator `@property`. Sprawia on, że ukryta logika walidacji odbywa się **automatycznie** podczas zwykłego przypisywania wartości znakiem `=`.

Przykład — ten sam mechanizm za pomocą `@property`:

```
class Temperatura:
    def __init__(self, celsjusz):
        # Przypisujemy do atrybutu publicznego (self.celsjusz), a nie prywatnego!
        # To sprawia, że setter zostanie od razu wywołany, walidując wartość startową.
        self.celsjusz = celsjusz

    @property
    def celsjusz(self):
```

```

    # Ten kod uruchomi się, gdy ktoś wywoła np.: print(t.celsjusz)
    return self.__celsjusz

@celsjusz.setter
def celsjusz(self, wartosc):
    # Ten kod uruchomi się, gdy ktoś wywoła np.: t.celsjusz = 20
    if wartosc >= -273.15:
        self.__celsjusz = wartosc
    else:
        print("Błąd: Temperatura poniżej zera absolutnego!")

@property
def fahrenheit(self):
    # Wirtualny atrybut tylko do odczytu (nie ma settera).
    # Atrybut "fahrenheit" nie istnieje jako zmienna, jest liczony w locie na podstawie celsjusza.
    return self.__celsjusz * 9 / 5 + 32

t = Temperatura(25)
t.celsjusz = 30      # Setter działa w tle – waliduje i przypisuje nową wartość
print(t.celsjusz)    # Getter działa w tle – zwraca 30
print(t.fahrenheit)  # Oblicza się w locie i zwraca 86.0

t.celsjusz = -300    # Wywołuje setter i blokuje ujemną temperaturę błędem!
# t.fahrenheit = 100 # Spowodowałoby błąd AttributeError, bo ten atrybut jest tylko do odczytu

```

12. Programowanie obiektowe — zmienne klasowe i metody statyczne

12.1. Zmienne klasowe (static fields)

Zmienne zdefiniowane bezpośrednio w klasie (poza `__init__`). Są współdzielone przez **wszystkie** instancje tej klasy.

```

class Pracownik:
    liczba_pracownikow = 0 # Zmienna klasowa

    def __init__(self, imie):
        self.imie = imie
        Pracownik.liczba_pracownikow += 1

p1 = Pracownik("Adam")
p2 = Pracownik("Ewa")
print(Pracownik.liczba_pracownikow) # 2

```

12.2. Zmienne klasowe — tablice i kolekcje (Pułapka!)

Jeśli zmienna klasowa jest mutowalna (np. lista), każda zmiana w jednym obiekcie wpłynie na wszystkie inne!

```

class KlasaZBłędem:
    lista = [] # UWAGA: wspólna dla wszystkich!

obj1 = KlasaZBłędem()
obj2 = KlasaZBłędem()

```

```
obj1.lista.append(1)
print(obj2.lista) # [1] → Zmiana widoczna w drugim obiekcie!
```

12.3. Metody statyczne — `@staticmethod`

Metoda, która nie ma dostępu do `self` ani `cls`. Zachowuje się jak zwykła funkcja, ale jest umieszczona wewnątrz klasy dla lepszej organizacji.

```
class Narzedzia:
    @staticmethod
    def czy_parzysta(n):
        return n % 2 == 0

print(Narzedzia.czy_parzysta(10)) # True
```

12.4. Metody klasowe — `@classmethod`

Jako pierwszy argument przyjmuje klasę (`cls`), a nie instancję. Często używana jako **alternatywny konstruktor** (wzorzec Fabryka).

```
class Data:
    def __init__(self, dzien, miesiac, rok):
        self.dzien, self.miesiac, self.rok = dzien, miesiac, rok

    @classmethod
    def z_tekstu(cls, tekst):
        """Tworzy obiekt na podstawie stringa 'DD-MM-YYYY'"""
        d, m, r = map(int, tekst.split("-"))
        return cls(d, m, r) # Wywołuje __init__

data = Data.z_tekstu("15-05-2024")
print(data.rok) # 2024
```

13. Programowanie obiektowe — konstruktory zaawansowane

13.1. Konstruktor z wartościami domyślnymi

Python nie obsługuje przeciążania metod (wiele metod o tej samej nazwie), dlatego używamy parametrów domyślnych.

```
class Punkt:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

p1 = Punkt() # (0, 0)
p2 = Punkt(5, 10) # (5, 10)
```

13.2. Wzorzec z `None` — pełny zamiennik dwóch konstruktorów

Jeśli chcemy, aby pole było inicjalizowane nową listą dla każdego obiektu, używamy `None`.

```
class Student:
    def __init__(self, imie, oceny=None):
        self.imie = imie
        # Zapewnia unikalną listę dla każdego obiektu
        if oceny is None:
            self.oceny = []
        else:
            self.oceny = oceny
```

13.3. Walidacja w konstruktorze

Zawsze warto sprawdzić dane wejściowe przed ich przypisaniem.

```
class Konto:
    def __init__(self, balans):
        if balans < 0:
            raise ValueError("Balans nie może być ujemny!")
        self.balans = balans
```

13.4. Kopiowanie obiektów

Standardowe przypisanie `obj2 = obj1` kopiuje tylko **referencję** (oba wskazują na ten sam obiekt). Aby stworzyć kopię, używamy modułu `copy`.

```
import copy

class Graf:
    def __init__(self, wezly):
        self.wezly = wezly

g1 = Graf([1, 2, 3])

# Kopia płytka (Shallow copy)
g2 = copy.copy(g1)

# Kopia głęboka (Deep copy) - kopiuje też zawartość list
g3 = copy.deepcopy(g1)
```

14. Programowanie obiektowe — dziedziczenie, polimorfizm i klasy abstrakcyjne

14.1. Podstawy dziedziczenia

Klasa pochodna (dziecko) dziedziczy wszystkie atrybuty i metody klasy bazowej (rodzica).

```
class Zwierze:
    def oddychaj(self):
        print("Oddycham...")

class Pies(Zwierze):
```

```
def szczekaj(self):
    print("Hau!")

p = Pies()
p.oddechaj() # Metoda z klasy Zwierze
p.szczekaj() # Metoda z klasy Pies
```

14.2. `super()` — wywoływanie metod rodzica

Służy głównie do wywoływania konstruktora klasy bazowej wewnątrz konstruktora klasy pochodnej.

```
class Pojazd:
    def __init__(self, marka):
        self.marka = marka

class Auto(Pojazd):
    def __init__(self, marka, paliwo):
        super().__init__(marka) # Wywołanie __init__ rodzica
        self.paliwo = paliwo
```

14.3. Hermetyzacja a dziedziczenie

- Pola **publiczne** i **chronione** (`_`) są dostępne w klasach pochodnych.
- Pola **prywatne** (`__`) **nie** są bezpośrednio dostępne (trzeba użyć gettera rodzica).

14.4. Wzorzec: logika stanów z dziedziczeniem

Dziedziczenie świetnie nadaje się do modelowania obiektów, które zmieniają swoje zachowanie w zależności od stanu (np. zamówienie, postać w grze).

```
class StanZamowienia:
    def nastepny(self): pass
    def __str__(self): return self.__class__.__name__

class Nowe(StanZamowienia):
    def nastepny(self): return Opłacone()

class Opłacone(StanZamowienia):
    def nastepny(self): return Wysłane()

class Wysłane(StanZamowienia):
    def nastepny(self): return self # Koniec

class Zamowienie:
    def __init__(self):
        self.stan = Nowe()

    def pchnij_dalej(self):
        self.stan = self.stan.nastepny()
        print(f"Obecny stan: {self.stan}")

z = Zamowienie()
```



```
z.pchnij_dalej() # Obecny stan: Opłacone
z.pchnij_dalej() # Obecny stan: Wysłane
```

14.5. Sprawdzanie typu i dziedziczenia

Aby sprawdzić, czy obiekt jest instancją danej klasy (lub jej klasy pochodnej), używamy `isinstance()`. Aby sprawdzić relację między klasami — `issubclass()`.

```
isinstance(p, Pies)      # True
isinstance(p, Zwierze)   # True (bo Pies dziedziczy po Zwierze)
issubclass(Pies, Zwierze) # True
```

14.6. Podsumowanie — kiedy co stosować w OOP

MECHANIZM	KIEDY STOSOWAĆ?
Klasa	Gdy potrzebujesz zgrupować dane i funkcje w jeden "byt".
Pola prywatne (<code>__</code>)	Zawsze dla danych, które nie powinny być zmieniane bezpośrednio (np. balans konta).
<code>@property</code>	Gdy chcesz, by dostęp do pola był prosty (<code>obj.x</code>), ale wymagał walidacji.
<code>@classmethod</code>	Gdy potrzebujesz stworzyć obiekt w nietypowy sposób (np. z pliku JSON).
Dziedziczenie	Tylko gdy zachodzi relacja "jest rodzajem" (np. Pies jest rodzajem Zwierzęcia).
Kompozycja	Gdy obiekt "posiada" inne obiekty (np. Samochód posiada Silnik). Zazwyczaj lepsza niż dziedziczenie!
Klasa abstrakcyjna	Gdy tworzysz fundament dla innych klas i chcesz wymusić na nich wspólny interfejs.

14.7. Polimorfizm

Zdolność różnych obiektów do reagowania na to samo wywołanie metody w sposób specyficzny dla swojej klasy. W Pythonie opiera się to na **Duck Typing** ("Jeśli coś kwacze jak kaczka, to jest kaczką").

```
class Kaczka:
    def kwacz(self): print("Kwa kwa!")

class Czlowiek:
    def kwacz(self): print("Naśladuję kaczkę!")

def wydaj_dzwiek(byt):
    byt.kwacz() # Polimorficzne wywołanie

wydaj_dzwiek(Kaczka())
wydaj_dzwiek(Czlowiek())
```

14.8. Klasy abstrakcyjne — moduł `abc`

Klasa abstrakcyjna to taka, z której nie można stworzyć obiektu. Służy jako interfejs wymuszający implementację konkretnych metod w klasach pochodnych.

```
from abc import ABC, abstractmethod

class Kształt(ABC):
    @abstractmethod
```

```

def pole(self):
    """Metoda musi być nadpisana w dziecku"""
    pass

class Kwadrat(Kształt):
    def __init__(self, a):
        self.a = a

    def pole(self):
        return self.a ** 2

# k = Kształt() # BŁĄD: Can't instantiate abstract class
kw = Kwadrat(4)
print(kw.pole()) # 16

```

14.9. Kompleksowa implementacja: System Quiz

Wzorcowy przykład łączący: klasy abstrakcyjne, dziedziczenie, hermetyzację i listę obiektów.

```

from abc import ABC, abstractmethod

class Pytanie(ABC):
    def __init__(self, tresc, punkty):
        self._tresc = tresc      # Chronione
        self._punkty = punkty

    @abstractmethod
    def wyswietl(self): pass

    @abstractmethod
    def sprawdz(self, odpowiedz): pass

class PytanieZamkniete(Pytanie):
    def __init__(self, tresc, punkty, opcje, poprawna_index):
        super().__init__(tresc, punkty)
        self.__opcje = opcje      # Prywatne
        self.__poprawna = poprawna_index

    def wyswietl(self):
        print(f"\n[Pytanie za {self._punkty} pkt]: {self._tresc}")
        for i, o in enumerate(self.__opcje, 1):
            print(f"{i}. {o}")

    def sprawdz(self, odp):
        return str(odp) == str(self.__poprawna)

class PytanieOtwarte(Pytanie):
    def __init__(self, tresc, punkty, poprawna_odp):
        super().__init__(tresc, punkty)
        self.__poprawna = poprawna_odp.lower().strip()

    def wyswietl(self):
        print(f"\n[Pytanie OTWARTE za {self._punkty} pkt]: {self._tresc}")

```

```

def sprawdz(self, odp):
    return odp.lower().strip() == self.__poprawna

class Quiz:
    def __init__(self, tytul):
        self.tytul = tytul
        self.__pytania = []
        self.__wynik = 0

    def dodaj_pytanie(self, p):
        if isinstance(p, Pytanie):
            self.__pytania.append(p)

    def uruchom(self):
        print(f"=== {self.tytul.upper()} ===")
        for p in self.__pytania:
            p.wyswietl()
            user_odp = input("Odpowiedź: ")
            if p.sprawdz(user_odp):
                print("DOBRZE!")
                self.__wynik += p._punkty
            else:
                print("ŹLE!")

        print(f"\nKONIEC. Twój wynik: {self.__wynik} pkt.")

# --- Demo ---
moj_quiz = Quiz("Test Wiedzy")
moj_quiz.dodaj_pytanie(PytanieZamkniete("Stolica Polski?", 1, ["Kraków", "Warszawa"], 2))
moj_quiz.dodaj_pytanie(PytanieOtwarte("Jak ma na imię twórca Pythona?", 2, "Guido"))

# moj_quiz.uruchom() # Odkomentuj, aby przetestować interaktywnie

```

15. Algorytmy w Pythonie

Algorytmy to precyzyjne przepisy na rozwiązanie problemu. Składają się z jednoznacznie określonych kroków, prowadzących od danych wejściowych do pożądanego wyniku. Poniżej przedstawione są najważniejsze algorytmy, pogrupowane tematycznie, z pełnymi implementacjami i wyjaśnieniami.

Złożoność algorytmów — krótkie wprowadzenie:

NOTACJA	NAZWA	OPIS	PRZYKŁAD
O(1)	Stała	Czas nie zależy od rozmiaru danych	Dostęp do elementu listy po indeksie
O(n)	Liniowa	Czas rośnie proporcjonalnie	Wyszukiwanie liniowe
O(n ²)	Kwadratowa	Czas rośnie kwadratowo	Sortowanie bąbelkowe, przez wybieranie
O(log n)	Logarytmiczna	Bardzo efektywna	Wyszukiwanie binarne
O(n log n)	Liniowo-logarytmiczna	Typowa dla szybkich sortowań	Merge Sort, Quick Sort

15.1. Sortowanie i porządkowanie danych

15.1.1. SORTOWANIE PRZEZ WYBIERANIE (SELECTION SORT)

Idea: W każdym kroku szukamy elementu minimalnego w nieposortowanej części tablicy i zamieniamy go z pierwszym nieposortowanym elementem.

```
def sortowanie_przez_wybieranie(tablica):
    n = len(tablica)
    for i in range(n - 1):
        indeks_min = i
        for j in range(i + 1, n):
            if tablica[j] < tablica[indeks_min]:
                indeks_min = j
        # Zamiana miejscami
        tablica[i], tablica[indeks_min] = tablica[indeks_min], tablica[i]
    return tablica
```

15.1.2. SORTOWANIE BĄBELKOWE (BUBBLE SORT)

Idea: Porównujemy sąsiednie elementy i zamieniamy je, jeśli są w złej kolejności. Powtarzamy, aż tablica będzie posortowana.

```
def sortowanie_babelkowe(tablica):
    n = len(tablica)
    for i in range(n - 1):
        zamieniono = False
        for j in range(n - 1 - i):
            if tablica[j] > tablica[j + 1]:
                tablica[j], tablica[j + 1] = tablica[j + 1], tablica[j]
                zamieniono = True
        if not zamieniono:
            break # Jeśli nie było zamiany, tablica jest posortowana
    return tablica
```

15.1.3. SORTOWANIE PRZEZ WSTAWIANIE (INSERTION SORT)

Idea: Pobieramy kolejny element i "wstawiamy" go w odpowiednie miejsce wśród elementów już posortowanych (podobnie jak układanie kart w dłoni).

```
def sortowanie_przez_wstawianie(tablica):
    for i in range(1, len(tablica)):
        klucz = tablica[i]
        j = i - 1
        # Przesuwamy elementy większe od klucza o jedną pozycję w prawo
        while j >= 0 and tablica[j] > klucz:
            tablica[j + 1] = tablica[j]
            j -= 1
        tablica[j + 1] = klucz
    return tablica
```

15.1.4. SORTOWANIE PRZEZ SCALANIE (MERGE SORT)

Idea: Strategia „dziel i zwyciężaj”. Dzielimy tablicę na połowy, aż zostaną pojedyncze elementy, a następnie scalamy je w porządku rosnącym.

```
def merge_sort(tablica):
    if len(tablica) > 1:
        srodek = len(tablica) // 2
        L = tablica[:srodek]
        R = tablica[srodek:]

        merge_sort(L) # Sortuj lewą połowę
        merge_sort(R) # Sortuj prawą połowę

        i = j = k = 0
        # Scalanie
        while i < len(L) and j < len(R):
            if L[i] < R[j]:
                tablica[k] = L[i]
                i += 1
            else:
                tablica[k] = R[j]
                j += 1
            k += 1

        # Dopisanie pozostałych elementów
        while i < len(L):
            tablica[k] = L[i]; i += 1; k += 1
        while j < len(R):
            tablica[k] = R[j]; j += 1; k += 1
    return tablica
```

15.1.5. SORTOWANIE SZYBKIE (QUICK SORT)

Idea: Wybieramy element obrotowy (pivot). Dzielimy tablicę na elementy mniejsze od pivota, równe mu i większe, a następnie rekurencyjnie sortujemy podtablice.

```
def quick_sort(tablica):
    if len(tablica) ≤ 1:
        return tablica
    pivot = tablica[len(tablica) // 2]
    lewe = [x for x in tablica if x < pivot]
    srodkowe = [x for x in tablica if x == pivot]
    prawe = [x for x in tablica if x > pivot]
    return quick_sort(lewe) + srodkowe + quick_sort(prawe)
```

15.1.6. SORTOWANIE MALEJĄCE WŁASNYM ALGORYTMEM

Idea: To wariant sortowania przez wybieranie. Zamiast najmniejszego elementu szukamy największego i ustawiamy go na początku nieposortowanej części tablicy.

```
def sortowanie_malejaco(tablica):
    wynik = tablica[:]
    n = len(wynik)

    for i in range(n - 1):
        indeks_max = i
```

```

        for j in range(i + 1, n):
            if wynik[j] > wynik[indeks_max]:
                indeks_max = j
        wynik[i], wynik[indeks_max] = wynik[indeks_max], wynik[i]

    return wynik

```

15.1.7. LICZENIE LICZBY ZAMIAN W BUBBLE SORT

Idea: Oprócz sortowania zliczamy, ile razy algorytm musiał zamienić dwa sąsiednie elementy. To częste zadanie sprawdzające zrozumienie działania algorytmu.

```

def bubble_sort_z_licznikiem(tablica):
    wynik = tablica[:]
    zamiany = 0
    n = len(wynik)

    for i in range(n - 1):
        zamieniono = False
        for j in range(n - 1 - i):
            if wynik[j] > wynik[j + 1]:
                wynik[j], wynik[j + 1] = wynik[j + 1], wynik[j]
                zamiany += 1
                zamieniono = True
        if not zamieniono:
            break

    return wynik, zamiany

```

15.1.8. SPRAWDZANIE, CZY TABLICA JEST POSORTOWANA

Idea: Przechodzimy po parach sąsiednich elementów. Jeśli choć raz poprzedni element jest większy od następnego, tablica nie jest posortowana rosnąco.

```

def czy_posortowana_rosnaco(tablica):
    for i in range(1, len(tablica)):
        if tablica[i - 1] > tablica[i]:
            return False
    return True

```

15.1.9. SCALANIE DWÓCH POSORTOWANYCH TABLIC

Idea: Mając dwie już posortowane tablice, budujemy trzecią. Za każdym razem wybieramy mniejszy z dwóch aktualnie rozważanych elementów.

```

def scal_posortowane(a, b):
    wynik = []
    i = j = 0

    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            wynik.append(a[i])
            i += 1

```

```

        else:
            wynik.append(b[j])
            j += 1

    while i < len(a):
        wynik.append(a[i])
        i += 1

    while j < len(b):
        wynik.append(b[j])
        j += 1

    return wynik

```

15.1.10. PODZIAŁ TABLICY WZGLĘDEM PIVOTA

Idea: Dzielimy dane na trzy grupy: mniejsze od pivota, równe pivotowi i większe od pivota. To fragment logiki wykorzystywany w Quick Sort.

```

def podziel_wzgledem_pivota(tablica, pivot):
    mniejsze = []
    rowne = []
    wieksze = []

    for x in tablica:
        if x < pivot:
            mniejsze.append(x)
        elif x == pivot:
            rowne.append(x)
        else:
            wieksze.append(x)

    return mniejsze, rowne, wieksze

```

15.2. Wyszukiwanie i analiza list

15.2.1. WYSZUKIWANIE LINIOWE

Idea: Sprawdzamy każdy element po kolei, aż znajdziemy szukaną wartość.

```

def wyszukaj liniowo(tablica, szukana):
    for i in range(len(tablica)):
        if tablica[i] == szukana:
            return i
    return -1

```

15.2.2. WYSZUKIWANIE Z WARTOWNIKIEM (SENTINEL SEARCH)

Idea: Dodajemy szukaną wartość na koniec tablicy jako "wartownika", aby uprościć warunek pętli (brak sprawdzania końca tablicy w każdym kroku).

```
def wyszukaj_z_wartownikiem(tablica, szukana):
    n = len(tablica)
    tablica.append(szukana) # Dodaj wartownika
    i = 0
    while tablica[i] != szukana:
        i += 1
    tablica.pop() # Usuń wartownika
    return i if i < n else -1
```

15.2.3. WYSZUKIWANIE BINARNE (BINARY SEARCH)

Idea: W posortowanej tablicy sprawdzamy środkowy element. Jeśli to nie on, odrzucamy połowę tablicy, w której na pewno nie ma szukanej wartości.

```
def wyszukiwanie_binarne(tablica, szukana):
    lewy, prawy = 0, len(tablica) - 1
    while lewy <= prawy:
        srodek = (lewy + prawy) // 2
        if tablica[srodek] == szukana:
            return srodek
        elif tablica[srodek] < szukana:
            lewy = srodek + 1
        else:
            prawy = srodek - 1
    return -1
```

15.2.4. ZNAJDOWANIE MINIMUM, MAKSYMUM I ICH INDEKSÓW

Idea: Przechodzimy po tablicy jeden raz i zapamiętujemy indeks najmniejszego oraz największego elementu.

```
def min_max_z_indeksami(tablica):
    if not tablica:
        return None

    indeks_min = 0
    indeks_max = 0

    for i in range(1, len(tablica)):
        if tablica[i] < tablica[indeks_min]:
            indeks_min = i
        if tablica[i] > tablica[indeks_max]:
            indeks_max = i

    return {
        "min": tablica[indeks_min],
        "indeks_min": indeks_min,
        "max": tablica[indeks_max],
        "indeks_max": indeks_max,
    }
```


15.2.5. DRUGIE NAJWIĘKSZE I DRUGIE NAJMNIEJSZE

Idea: Szukamy drugich skrajnych wartości bez sortowania. Wersja poniżej zakłada, że interesują nas drugie różne wartości, a nie drugie pozycje w tablicy.

```
def drugie_skrajne(tablica):
    if len(set(tablica)) < 2:
        return None

    min1 = min2 = None
    max1 = max2 = None

    for x in tablica:
        if min1 is None or x < min1:
            min2 = min1
            min1 = x
        elif x != min1 and (min2 is None or x < min2):
            min2 = x

        if max1 is None or x > max1:
            max2 = max1
            max1 = x
        elif x != max1 and (max2 is None or x > max2):
            max2 = x

    return {"drugie_min": min2, "drugie_max": max2}
```

15.2.6. WYSZUKIWANIE PIERWSZEGO I OSTATNIEGO WYSTĄPIENIA WARTOŚCI

Idea: Przechodzimy po całej tablicy. Pierwszy indeks ustawiamy tylko raz, a ostatni aktualizujemy przy każdym trafieniu.

```
def pierwsze_i_ostatnie_wystapienie(tablica, szukana):
    pierwszy = -1
    ostatni = -1

    for i, wartosc in enumerate(tablica):
        if wartosc == szukana:
            if pierwszy == -1:
                pierwszy = i
            ostatni = i

    return pierwszy, ostatni
```

15.2.7. WYSZUKIWANIE BINARNE Z LICZENIEM KROKÓW

Idea: To zwykłe wyszukiwanie binarne, ale dodatkowo liczymy, ile razy sprawdzono środkowy element.

```
def wyszukiwanie_binarne_z_krokami(tablica, szukana):
    lewy = 0
    prawy = len(tablica) - 1
    kroki = 0

    while lewy <= prawy:
        kroki += 1
```

```

srodek = (lewy + prawy) // 2

if tablica[srodek] == szukana:
    return srodek, kroki
if tablica[srodek] < szukana:
    lewy = srodek + 1
else:
    prawy = srodek - 1

return -1, kroki

```

15.2.8. ZNAJDOWANIE BRAKUJĄCEJ LICZBY Z ZAKRESU 1..N

Idea: Jeśli w tablicy znajduje się zakres `1..n` z jedną brakującą liczbą, można porównać sumę oczekiwaną z rzeczywistą.

```

def brakujaca_liczba(tablica):
    n = len(tablica) + 1
    suma_oczekiwana = n * (n + 1) // 2
    suma_rzeczywista = sum(tablica)
    return suma_oczekiwana - suma_rzeczywista

```

15.2.9. ZNAJDOWANIE ELEMENTÓW WSPÓLNYCH DWÓCH LIST

Idea: Szukamy wartości, które występują w obu listach. Wynik nie zawiera duplikatów i zachowuje kolejność z pierwszej listy.

```

def elementy_wspolne(a, b):
    zbior_b = set(b)
    dodane = set()
    wynik = []

    for x in a:
        if x in zbior_b and x not in dodane:
            wynik.append(x)
            dodane.add(x)

    return wynik

```

15.2.10. ZNAJDOWANIE ELEMENTÓW WYSTĘPUJĄCYCH TYLKO RAZ

Idea: Najpierw zliczamy wystąpienia, potem wybieramy tylko te wartości, których licznik wynosi `1`.

```

def elementy_wystepujace_raz(tablica):
    liczniki = {}

    for x in tablica:
        liczniki[x] = liczniki.get(x, 0) + 1

    return [x for x in tablica if liczniki[x] == 1]

```

15.2.11. USUWANIE DUPLIKATÓW Z LISTY Z ZACHOWANIEM KOLEJNOŚCI

Idea: Zbiór `widziane` służy tylko do szybkiego sprawdzania, czy element był już dodany do wyniku.

```
def usun_duplikaty_z_listy(tablica):
    widziane = set()
    wynik = []

    for x in tablica:
        if x not in widziane:
            wynik.append(x)
            widziane.add(x)

    return wynik
```

15.2.12. ZLICZANIE WYSTĄPIEŃ WARTOŚCI W DANYCH

```
def zlicz_wystapienia(tablica):
    liczniki = {}
    for x in tablica:
        liczniki[x] = liczniki.get(x, 0) + 1
    return liczniki
```

15.2.13. OBLICZANIE ŚREDNIEJ ARYTMETYCZNEJ

```
def srednia(tablica):
    return sum(tablica) / len(tablica) if tablica else 0
```

15.2.14. WYODRĘBNIANIE ELEMENTÓW SPEŁNIAJĄCYCH WARUNEK

```
def filtruj_parzyste(tablica):
    return [x for x in tablica if x % 2 == 0]
```

15.3. Działania matematyczne i dzielniki

15.3.1. ALGORYTM EUKLIDESA (NWD)

Idea: NWD dwóch liczb to największa liczba, która dzieli obie bez reszty. Opiera się na reszcie z dzielenia.

```
def nwd(a, b):
    a = abs(a)
    b = abs(b)
    while b != 0:
        a, b = b, a % b
    return a
```

15.3.2. NAJMNIEJSZA WSPÓLNA WIELOKROTNOŚĆ (NWW)

Idea: Najmniejsza liczba będąca wielokrotnością obu liczb. Wykorzystuje zależność: $NWW(a, b) = |a * b| / NWD(a, b)$.

```
def nwd(a, b):
    a = abs(a)
    b = abs(b)
```

```

while b != 0:
    a, b = b, a % b
return a

def nww(a, b):
    if a == 0 or b == 0: return 0
    return abs(a * b) // nwd(a, b)

```

15.3.3. NWD DLA WIELU LICZB

Idea: Liczymy NWD pierwszych dwóch liczb, potem wynik łączymy z kolejną liczbą itd.

```

def nwd(a, b):
    a = abs(a)
    b = abs(b)
    while b != 0:
        a, b = b, a % b
    return a

def nwd_wielu(liczby):
    if not liczby:
        return 0

    wynik = liczby[0]
    for liczba in liczby[1:]:
        wynik = nwd(wynik, liczba)

    return wynik

```

15.3.4. NWW DLA WIELU LICZB

Idea: Liczymy NWW kolejnych par, podobnie jak przy NWD dla wielu liczb.

```

def nwd(a, b):
    a = abs(a)
    b = abs(b)
    while b != 0:
        a, b = b, a % b
    return a

def nww(a, b):
    if a == 0 or b == 0:
        return 0
    return abs(a * b) // nwd(a, b)

def nww_wielu(liczby):
    if not liczby:
        return 0

    wynik = liczby[0]
    for liczba in liczby[1:]:
        wynik = nww(wynik, liczba)

```

```
return wynik
```

15.3.5. LICZBY WZGLĘDNIPIERWSZE

Idea: Dwie liczby są względnie pierwsze, jeśli ich największy wspólny dzielnik wynosi `1`.

```
def nwd(a, b):  
    a = abs(a)  
    b = abs(b)  
    while b != 0:  
        a, b = b, a % b  
    return a  
  
def czy_wzgleownie_pierwsze(a, b):  
    return nwd(a, b) == 1
```

15.3.6. SUMA DZIELNIKÓW LICZBY

Idea: Sprawdzamy dzielniki do pierwiastka z liczby. Jeśli `d` dzieli `n`, to drugim dzielnikiem jest `n // d`.

```
def suma_dzielnikow(n, wlasciwe=False):  
    if n <= 0:  
        return 0  
  
    suma = 0  
    d = 1  
  
    while d * d <= n:  
        if n % d == 0:  
            suma += d  
            drugi = n // d  
            if drugi != d:  
                suma += drugi  
            d += 1  
  
    if wlasciwe:  
        suma -= n  
  
    return suma
```

15.3.7. LICZBA DOSKONAŁA

Idea: Liczba doskonała jest równa sumie swoich dzielników właściwych, czyli dzielników mniejszych od niej samej.

```
def suma_dzielnikow_wlasciwych(n):  
    suma = 0  
    for d in range(1, n):  
        if n % d == 0:  
            suma += d  
    return suma  
  
def czy_doskonała(n):
```

```

if n ≤ 1:
    return False
return suma_dzielnikow_wlasciwych(n) == n

```

15.3.8. LICZBY ZAPRZYJAŻNIONE

Idea: Dwie liczby są zaprzyjaźnione, jeśli suma dzielników właściwych pierwszej daje drugą, a suma dzielników właściwych drugiej daje pierwszą.

```

def suma_dzielnikow_wlasciwych(n):
    suma = 0
    for d in range(1, n):
        if n % d == 0:
            suma += d
    return suma

def czy_zaprzyjaznione(a, b):
    if a == b:
        return False
    return (
        suma_dzielnikow_wlasciwych(a) == b
        and suma_dzielnikow_wlasciwych(b) == a
    )

```

15.3.9. SKRACANIE UŁAMKA

Idea: Licznik i mianownik dzielimy przez ich NWD. Dodatkowo przenosimy znak minus do licznika.

```

def nwd(a, b):
    a = abs(a)
    b = abs(b)
    while b ≠ 0:
        a, b = b, a % b
    return a

def skroc_ulamek(licznik, mianownik):
    if mianownik == 0:
        raise ValueError("Mianownik nie może być zerem")

    dzielnik = nwd(licznik, mianownik)
    licznik /= dzielnik
    mianownik /= dzielnik

    if mianownik < 0:
        licznik = -licznik
        mianownik = -mianownik

    return licznik, mianownik

```

15.3.10. DODAWANIE DWÓCH UŁAMKÓW

Idea: Sprowadzamy ułamki do wspólnego mianownika, dodajemy liczniki, a wynik skracamy.

```
def nwd(a, b):
    a = abs(a)
    b = abs(b)
    while b != 0:
        a, b = b, a % b
    return a

def skroc_ułamek(licznik, mianownik):
    if mianownik == 0:
        raise ValueError("Mianownik nie może być zerem")

    dzielnik = nwd(licznik, mianownik)
    licznik /= dzielnik
    mianownik /= dzielnik

    if mianownik < 0:
        licznik = -licznik
        mianownik = -mianownik

    return licznik, mianownik

def dodaj_ułamki(l1, m1, l2, m2):
    licznik = l1 * m2 + l2 * m1
    mianownik = m1 * m2
    return skroc_ułamek(licznik, mianownik)
```

15.4. Liczby pierwsze, cyfry i cechy liczb

15.4.1. SPRAWDZANIE CZY LICZBA JEST PIERWSZA

Idea: Sprawdzamy dzielniki liczby od 2 do jej pierwiastka. Jeśli żaden nie dzieli liczby, jest ona pierwsza.

```
def jest_pierwsza(n):
    if n < 2:
        return False

    d = 2
    while d * d ≤ n:
        if n % d == 0:
            return False
        d += 1

    return True
```

15.4.2. SITO ERATOSTENESA

Idea: Wykreślamy wielokrotności kolejnych liczb pierwszych, aby znaleźć wszystkie liczby pierwsze w zakresie od 2 do n.

```
def sito(n):
    if n < 2:
        return []
```

```

pierwsze = [True] * (n + 1)
pierwsze[0] = False
pierwsze[1] = False

for p in range(2, int(n ** 0.5) + 1):
    if pierwsze[p]:
        for i in range(p * p, n + 1, p):
            pierwsze[i] = False

return [i for i in range(2, n + 1) if pierwsze[i]]

```

15.4.3. ROZKŁAD LICZBY NA CZYNNIKI PIERWSZE

Idea: Dzielimy liczbę przez najmniejszy możliwy dzielnik tak długo, jak się da. Potem przechodzimy do kolejnego dzielnika.

```

def rozklad_na_czynniki(n):
    czynniki = []
    d = 2

    while d * d ≤ n:
        while n % d == 0:
            czynniki.append(d)
            n /= d
        d += 1

    if n > 1:
        czynniki.append(n)

    return czynniki

```

15.4.4. ROZKŁAD LICZBY NA CZYNNIKI PIERWSZE Z POTĘGAMI

Idea: Zamiast powtarzać ten sam czynnik wiele razy, zapisujemy, ile razy wystąpił.

```

def rozklad_z_potegami(n):
    czynniki = {}
    d = 2

    while d * d ≤ n:
        while n % d == 0:
            czynniki[d] = czynniki.get(d, 0) + 1
            n /= d
        d += 1

    if n > 1:
        czynniki[n] = czynniki.get(n, 0) + 1

    return czynniki

```

15.4.5. NAJBLIŻSZA LICZBA PIERWSZA WIĘKSZA OD N

Idea: Sprawdzamy kolejne liczby większe od `n`, aż trafimy na liczbę pierwszą.


```
def jest_pierwsza(n):
    if n < 2:
        return False

    d = 2
    while d * d ≤ n:
        if n % d == 0:
            return False
        d += 1

    return True

def nastepna_pierwsza(n):
    kandydat = n + 1

    while not jest_pierwsza(kandydat):
        kandydat += 1

    return kandydat
```

15.4.6. LICZENIE LICZB PIERWSZYCH W PRZEDZIALE

Idea: Testujemy każdą liczbę z przedziału i dopisujemy ją do wyniku, jeśli jest pierwsza.

```
def jest_pierwsza(n):
    if n < 2:
        return False

    d = 2
    while d * d ≤ n:
        if n % d == 0:
            return False
        d += 1

    return True

def liczby_pierwsze_w_przedziale(a, b):
    pierwsze = []

    for liczba in range(a, b + 1):
        if jest_pierwsza(liczba):
            pierwsze.append(liczba)

    return pierwsze
```

15.4.7. LICZBA ARMSTRONGA

Idea: Liczba Armstronga jest równa sumie swoich cyfr podniesionych do potęgi równej liczbie cyfr, np. $153 = 1^3 + 5^3 + 3^3$.

```
def czy_armstrong(n):
    if n < 0:
        return False
```

```

cyfry = [int(c) for c in str(n)]
potega = len(cyfry)
suma = 0

for cyfra in cyfry:
    suma += cyfra ** potega

return suma == n

```

15.4.8. STATYSTYKA CYFR LICZBY

Idea: Zliczamy, ile razy każda cyfra od 0 do 9 występuje w zapisie liczby.

```

def statystyka_cyfr(n):
    liczniki = {cyfra: 0 for cyfra in range(10)}

    for znak in str(abs(n)):
        liczniki[int(znak)] += 1

    return liczniki

```

15.4.9. ODWRACANIE CYFR LICZBY BEZ ZAMIANY NA NAPIS

Idea: Ostatnią cyfrę pobieramy przez $n \% 10$, a potem usuwamy ją przez $n // 10$.

```

def odwroc_liczbe(n):
    znak = -1 if n < 0 else 1
    n = abs(n)
    wynik = 0

    while n > 0:
        cyfra = n % 10
        wynik = wynik * 10 + cyfra
        n //= 10

    return znak * wynik

```

15.5. Systemy liczbowe i operacje binarne

15.5.1. KONWERSJA SYSTEMÓW LICZBOWYCH (DZIESIĘTNY <-> BINARNY)

```

def dec_to_bin(n):
    if n == 0:
        return "0"

    wynik = ""

    while n > 0:
        wynik = str(n % 2) + wynik
        n //= 2

```

```

    return wynik

def bin_to_dec(b):
    wynik = 0

    for i, cyfra in enumerate(reversed(b)):
        if cyfra == '1':
            wynik += 2 ** i

    return wynik

```

15.5.2. KONWERSJA DZIESIĘTNA NA DOWOLNY SYSTEM OD 2 DO 16

Idea: Dzielimy liczbę przez podstawę systemu, a reszty z dzielenia zapisujemy od końca.

```

def dec_to_base(n, podstawa):
    if podstawa < 2 or podstawa > 16:
        raise ValueError("Podstawa musi być z zakresu 2..16")

    if n == 0:
        return "0"

    cyfry = "0123456789ABCDEF"
    znak = "-" if n < 0 else ""
    n = abs(n)
    wynik = ""

    while n > 0:
        wynik = cyfry[n % podstawa] + wynik
        n //= podstawa

    return znak + wynik

```

15.5.3. KONWERSJA Z DOWOLNEGO SYSTEMU OD 2 DO 16 NA DZIESIĘTNY

Idea: Przechodzimy po cyfrach od lewej do prawej. Każdy krok mnoży dotychczasowy wynik przez podstawę i dodaje wartość aktualnej cyfry.

```

def base_to_dec(napis, podstawa):
    if podstawa < 2 or podstawa > 16:
        raise ValueError("Podstawa musi być z zakresu 2..16")

    cyfry = "0123456789ABCDEF"
    napis = napis.upper()
    znak = -1 if napis.startswith("-") else 1

    if napis.startswith("-"):
        napis = napis[1:]

    wynik = 0

    for znak_cyfry in napis:

```

```

wartosc = cyfry.index(znak_cyfry)
if wartosc ≥ podstawa:
    raise ValueError("Cyfra nie pasuje do podstawy systemu")
wynik = wynik * podstawa + wartosc

return znak * wynik

```

15.5.4. DODAWANIE DWÓCH LICZB BINARNYCH ZAPISANYCH JAKO NAPISY

Idea: Dodajemy od końca, tak jak pisemnie. Pamiętamy przeniesienie do kolejnej kolumny.

```

def dodaj_binarne(a, b):
    i = len(a) - 1
    j = len(b) - 1
    przeniesienie = 0
    wynik = []

    while i ≥ 0 or j ≥ 0 or przeniesienie:
        suma = przeniesienie

        if i ≥ 0:
            suma += int(a[i])
            i -= 1
        if j ≥ 0:
            suma += int(b[j])
            j -= 1

        wynik.append(str(suma % 2))
        przeniesienie = suma // 2

    return "".join(reversed(wynik)).rstrip("0") or "0"

```

15.6. Teksty, słowniki i zliczanie

15.6.1. PALINDROMY I ANAGRAMY

```

def czy_palindrom(tekst):
    t = tekst.replace(" ", "").lower()
    return t == t[::-1]

def czy_anagram(s1, s2):
    t1 = s1.replace(" ", "").lower()
    t2 = s2.replace(" ", "").lower()
    return sorted(t1) == sorted(t2)

```

15.6.2. ZLICZANIE LITER W TEKŚCIE

Idea: Pomijamy znaki, które nie są literami, a litery sprowadzamy do małych znaków.

```

def zlicz_litery(tekst):
    liczniki = {}

```

```
for znak in tekst.lower():
    if znak.isalpha():
        liczniki[znak] = liczniki.get(znak, 0) + 1

return liczniki
```

15.6.3. NAJCZĘŚCIEJ WYSTĘPUJĄCY ZNAK

Idea: Najpierw zliczamy znaki, a potem wybieramy ten z największym licznikiem.

```
def najczestszy_znak(tekst):
    liczniki = {}

    for znak in tekst:
        if znak != " ":
            liczniki[znak] = liczniki.get(znak, 0) + 1

    if not liczniki:
        return None

    najlepszy = None

    for znak, liczba in liczniki.items():
        if najlepszy is None or liczba > liczniki[najlepszy]:
            najlepszy = znak

    return najlepszy, liczniki[najlepszy]
```

15.6.4. NAJCZĘŚCIEJ WYSTĘPUJĄCE SŁOWO W ZDANIU

Idea: Usuwamy proste znaki interpunkcyjne, dzielimy tekst na słowa i zliczamy wystąpienia.

```
def najczestsze_slowo(tekst):
    oczyszczony = ""

    for znak in tekst.lower():
        if znak.isalnum() or znak.isspace():
            oczyszczony += znak
        else:
            oczyszczony += " "

    liczniki = {}

    for slowo in oczyszczony.split():
        liczniki[slowo] = liczniki.get(slowo, 0) + 1

    if not liczniki:
        return None

    najlepsze = None

    for slowo, liczba in liczniki.items():
        if najlepsze is None or liczba > liczniki[najlepsze]:
```

```
najlepsze = slowo

return najlepsze, liczniki[najlepsze]
```

15.6.5. USUWANIE WSZYSTKICH ZNAKÓW NIEBĘDĄCYCH LITERAMI

Idea: Budujemy nowy napis tylko z tych znaków, dla których `isalpha()` zwraca `True`.

```
def zostaw_tylko_litery(tekst):
    wynik = ""

    for znak in tekst:
        if znak.isalpha():
            wynik += znak

    return wynik
```

15.6.6. SPRAWDZANIE PANGRAMU

Idea: Pangram to tekst, który zawiera wszystkie litery alfabetu. W wersji egzaminacyjnej najczęściej sprawdzamy alfabet angielski `a-z`.

```
def czy_pangram(tekst):
    alfabet = set("abcdefghijklmnopqrstuvwxyz")
    litery = set()

    for znak in tekst.lower():
        if "a" <= znak <= "z":
            litery.add(znak)

    return alfabet.issubset(litery)
```

15.6.7. SPRAWDZANIE POPRAWNOŚCI NAWIASÓW

Idea: Używamy stosu. Nawias otwierający odkładamy na stos, a przy zamykającym sprawdzamy, czy pasuje do ostatniego otwartego.

```
def czy_poprawne_nawiasy(tekst):
    pary = {"(": ")", "[": "]", "{": "}"}.values()
    otwierajace = set(pary)
    stos = []

    for znak in tekst:
        if znak in otwierajace:
            stos.append(znak)
        elif znak in pary:
            if not stos or stos[-1] != pary[znak]:
                return False
            stos.pop()

    return len(stos) == 0
```

15.6.8. USUWANIE POWTARZAJĄCYCH SIĘ ZNAKÓW SĄSIEDNICH

Idea: Iterujemy od drugiego znaku i dopisujemy znak tylko wtedy, gdy różni się od poprzedniego.

```
def usun_powtarzajace_sie_znaki(tekst):
    if not tekst:
        return ""

    czysty_tekst = tekst[0]
    for i in range(1, len(tekst)):
        if tekst[i] != tekst[i - 1]:
            czysty_tekst += tekst[i]
    return czysty_tekst
```

15.6.9. USUWANIE SĄSIEDNICH DUPLIKATÓW AŻ DO SKUTKU

Idea: Stos pozwala usuwać pary sąsiednich znaków także wtedy, gdy po usunięciu jednej pary powstaje kolejna para.

```
def usun_sasiednie_duplikaty_do_skutku(tekst):
    stos = []

    for znak in tekst:
        if stos and stos[-1] == znak:
            stos.pop()
        else:
            stos.append(znak)

    return "".join(stos)
```

15.6.10. NAJDŁUŻSZY WSPÓLNY PREFIKS LISTY SŁÓW

Idea: Zaczynamy od pierwszego słowa jako kandydata. Skracamy go, aż każde słowo zaczyna się tym prefiksem.

```
def najdluzszy_wspolny_prefiks(slowa):
    if not slowa:
        return ""

    prefiks = slowa[0]

    for slowo in slowa[1:]:
        while not slowo.startswith(prefiks):
            prefiks = prefiks[:-1]
            if prefiks == "":
                return ""

    return prefiks
```

15.6.11. SPRAWDZANIE, CZY DWA TEKSTY SĄ IZOMORFICZNE

Idea: Teksty są izomorficzne, jeśli każdemu znakowi z pierwszego tekstu odpowiada dokładnie jeden znak z drugiego tekstu i odwrotnie.

```
def czy_izomorficzne(a, b):
    if len(a) != len(b):
        return False

    mapa_ab = {}
```

```

mapa_ba = {}

for znak_a, znak_b in zip(a, b):
    if znak_a in mapa_ab and mapa_ab[znak_a] != znak_b:
        return False
    if znak_b in mapa_ba and mapa_ba[znak_b] != znak_a:
        return False

    mapa_ab[znak_a] = znak_b
    mapa_ba[znak_b] = znak_a

return True

```

15.7. Szyfry i kodowanie tekstu

15.7.1. SZYFR CEZARA

Idea: Przesunięcie każdej litery tekstu o stałą liczbę pozycji w alfabecie.

```

def szyfruj_cezar(tekst, klucz):
    wynik = ""

    for znak in tekst:
        if "a" <= znak <= "z":
            nowy = (ord(znak) - ord("a") + klucz) % 26
            wynik += chr(nowy + ord("a"))
        elif "A" <= znak <= "Z":
            nowy = (ord(znak) - ord("A") + klucz) % 26
            wynik += chr(nowy + ord("A"))
        else:
            wynik += znak

    return wynik

```

15.7.2. ODSZYFROWANIE SZYFRU CEZARA

Idea: Odszyfrowanie jest tym samym algorytmem, ale z przeciwnym przesunięciem.

```

def szyfruj_cezar(tekst, klucz):
    wynik = ""

    for znak in tekst:
        if "a" <= znak <= "z":
            nowy = (ord(znak) - ord("a") + klucz) % 26
            wynik += chr(nowy + ord("a"))
        elif "A" <= znak <= "Z":
            nowy = (ord(znak) - ord("A") + klucz) % 26
            wynik += chr(nowy + ord("A"))
        else:
            wynik += znak

    return wynik

```



```
def odszyfruj_cezar(tekst, klucz):  
    return szyfruj_cezar(tekst, -klucz)
```

15.7.3. ŁAMANIE SZYFRU CEZARA METODĄ BRUTE FORCE

Idea: Szyfr Cezara ma tylko 26 możliwych przesunięć, więc można wypisać wszystkie kandydаты.

```
def szyfruj_cezar(tekst, klucz):  
    wynik = ""  
  
    for znak in tekst:  
        if "a" ≤ znak ≤ "z":  
            nowy = (ord(znak) - ord("a") + klucz) % 26  
            wynik += chr(nowy + ord("a"))  
        elif "A" ≤ znak ≤ "Z":  
            nowy = (ord(znak) - ord("A") + klucz) % 26  
            wynik += chr(nowy + ord("A"))  
        else:  
            wynik += znak  
  
    return wynik  
  
def lam_cezara(tekst):  
    proby = []  
  
    for klucz in range(26):  
        proby.append((klucz, szyfruj_cezar(tekst, -klucz)))  
  
    return proby
```

15.7.4. SZYFR VIGENERE'A

Idea: Każda litera jest przesuwana o wartość wynikającą z kolejnej litery klucza. Klucz powtarza się cyklicznie.

```
def szyfruj_vigenere(tekst, klucz):  
    klucz = "".join(z for z in klucz.lower() if "a" ≤ z ≤ "z")  
  
    if not klucz:  
        raise ValueError("Klucz musi zawierać przynajmniej jedną literę a-z")  
  
    wynik = ""  
    indeks_klucza = 0  
  
    for znak in tekst:  
        przesuniecie = ord(klucz[indeks_klucza % len(klucz)]) - ord("a")  
  
        if "a" ≤ znak ≤ "z":  
            nowy = (ord(znak) - ord("a") + przesuniecie) % 26  
            wynik += chr(nowy + ord("a"))  
            indeks_klucza += 1  
        elif "A" ≤ znak ≤ "Z":  
            nowy = (ord(znak) - ord("A") + przesuniecie) % 26
```

```

        wynik += chr(nowy + ord("A"))
        indeks_klucza += 1
    else:
        wynik += znak

    return wynik

```

15.8. Rekurencja i dziel i zwyciężaj

15.8.1. SILNIA I FIBONACCI REKURENCYJNIE

Idea: Technika, w której funkcja wywołuje samą siebie. Wymaga przypadku bazowego (stopu).

```

# Silnia (n!)
def silnia(n):
    return 1 if n <= 1 else n * silnia(n - 1)

# Fibonacci
def fib(n):
    return n if n <= 1 else fib(n-1) + fib(n-2)

```

15.8.2. GENEROWANIE CIĄGU FIBONACCIEGO ITERACYJNIE

Idea: Zamiast rekurencji używamy dwóch zmiennych przechowujących dwa ostatnie wyrazy ciągu.

```

def fibonacci_iteracyjnie(n):
    if n <= 0:
        return []

    wynik = []
    a = 0
    b = 1

    for _ in range(n):
        wynik.append(a)
        a, b = b, a + b

    return wynik

```

15.8.3. SZYBKIE POTĘGOWANIE

```

def poteguj_szybko(a, n):
    if n == 0:
        return 1
    if n < 0:
        return 1 / poteguj_szybko(a, -n)

    polowa = poteguj_szybko(a, n // 2)

    if n % 2 == 0:

```

```
    return polowa * polowa
return polowa * polowa * a
```

15.8.4. WIEŻE HANOI

Idea: Aby przenieść n krążków, najpierw przenosimy $n-1$ krążków na słupek pomocniczy, potem największy krążek na cel, a na końcu $n-1$ krążków na cel.

```
def hanoi(n, zrodlo, pomocniczy, cel, ruchy=None):
    if ruchy is None:
        ruchy = []

    if n == 0:
        return ruchy

    hanoi(n - 1, zrodlo, cel, pomocniczy, ruchy)
    ruchy.append((zrodlo, cel))
    hanoi(n - 1, pomocniczy, zrodlo, cel, ruchy)

    return ruchy
```

15.8.5. SUMA ELEMENTÓW TABLICY REKURENCYJNIE

Idea: Suma pustej części tablicy wynosi 0 . W innym przypadku dodajemy aktualny element do sumy reszty tablicy.

```
def suma_rekurencyjnie(tablica, indeks=0):
    if indeks == len(tablica):
        return 0

    return tablica[indeks] + suma_rekurencyjnie(tablica, indeks + 1)
```

15.9. Tablice dwuwymiarowe

15.9.1. PRZESZUKIWANIE TABLICY 2D

Idea: Sprawdzamy każdy wiersz i każdą kolumnę. Po znalezieniu elementu zwracamy parę indeksów.

```
def znajdz_w_macierzy(macierz, szukana):
    for i, wiersz in enumerate(macierz):
        for j, wartosc in enumerate(wiersz):
            if wartosc == szukana:
                return i, j

    return -1, -1
```

15.9.2. SUMA PRZEKĄTNYCH MACIERZY KWADRATOWEJ

Idea: Dodajemy elementy z przekątnej głównej i pobocznej. Jeśli rozmiar macierzy jest nieparzysty, środkowy element został policzony dwa razy, więc odejmujemy go raz.

```
def suma_przekatnych(macierz):
    n = len(macierz)
```

```

suma = 0

for i in range(n):
    suma += macierz[i][i]
    suma += macierz[i][n - 1 - i]

if n % 2 == 1:
    srodek = n // 2
    suma -= macierz[srodek][srodek]

return suma

```

15.9.3. OBRÓT MACIERZY O 90 STOPNI W PRAWO

Idea: Pierwszy wiersz wyniku powstaje z pierwszej kolumny oryginału czytanej od dołu.

```

def obroci_macierz_90(macierz):
    if not macierz:
        return []

    liczba_wierszy = len(macierz)
    liczba_kolumn = len(macierz[0])
    wynik = []

    for kolumna in range(liczba_kolumn):
        nowy_wiersz = []
        for wiersz in range(liczba_wierszy - 1, -1, -1):
            nowy_wiersz.append(macierz[wiersz][kolumna])
        wynik.append(nowy_wiersz)

    return wynik

```

15.10. Walidacja danych i zadania plikowe

15.10.1. WALIDACJA DANYCH Z WAGAMI (NP. PESEL)

Idea: Mnożymy kolejne cyfry przez odpowiednie wagi, sumujemy wyniki i wyliczamy cyfrę kontrolną.

```

def suma_kontrolna(liczby, wagi):
    suma = sum(int(l) * w for l, w in zip(liczby, wagi))
    return (10 - (suma % 10)) % 10

def poprawny_pesel(pesel):
    if len(pesel) != 11 or not pesel.isdigit():
        return False

    wagi = [1, 3, 7, 9, 1, 3, 7, 9, 1, 3]
    cyfra_kontrolna = suma_kontrolna(pesel[:10], wagi)

    return cyfra_kontrolna == int(pesel[10])

```

15.10.2. PROSTY SYSTEM PUNKTACJI Z PLIKU CSV

Idea: Wczytujemy uczniów z pliku CSV, liczymy sumę i średnią punktów, a potem zwracamy ranking od najlepszego wyniku.

Przykładowe kolumny pliku: `imie,nazwisko,zadanie1,zadanie2,zadanie3`.

```
import csv

def ranking_uczniow_csv(sciezka):
    uczniowie = []

    with open(sciezka, newline="", encoding="utf-8") as plik:
        czytnik = csv.DictReader(plik)

        for wiersz in czytnik:
            pola_punktow = [
                nazwa
                for nazwa in wiersz.keys()
                if nazwa not in ("imie", "nazwisko")
            ]

            punkty = []
            for pole in pola_punktow:
                if wiersz[pole] != "":
                    punkty.append(float(wiersz[pole]))

            suma = sum(punkty)
            srednia_punktow = suma / len(punkty) if punkty else 0

            uczniowie.append({
                "imie": wiersz["imie"],
                "nazwisko": wiersz["nazwisko"],
                "suma": suma,
                "srednia": srednia_punktow,
            })

    return sorted(uczniowie, key=lambda u: u["suma"], reverse=True)
```

15.10.3. ZAPIS RANKINGU DO PLIKU CSV

Idea: To rozwinięcie poprzedniego przykładu. Po policzeniu rankingu zapisujemy wynik do nowego pliku.

```
import csv

def zapisz_ranking_csv(ranking, sciezka_wyjsciowa):
    pola = ["miejsce", "imie", "nazwisko", "suma", "srednia"]

    with open(sciezka_wyjsciowa, "w", newline="", encoding="utf-8") as plik:
        zapis = csv.DictWriter(plik, fieldnames=pola)
        zapis.writeheader()

        for miejsce, uczen in enumerate(ranking, start=1):
            zapis.writerow({
                "miejsce": miejsce,
                "imie": uczen["imie"],
```

```
        "nazwisko": uczen["nazwisko"],
        "suma": uczen["suma"],
        "srednia": round(uczen["srednia"], 2),
    })
```

16. Tablice dwuwymiarowe

Tablica dwuwymiarowa (2D) to „tablica tablic” — lista, której każdy element jest również listą. Reprezentuje dane w formie wierszy i kolumn, jak tabela lub macierz. Jest niezbędna przy pracy z siatkami, planszami gier i zestawami danych.

16.1. Tworzenie tablic 2D

```
# Ręczne tworzenie
macierz = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Dynamiczne tworzenie — n wierszy × m kolumn, wypełnione zerami
wiersze = 4
kolumny = 6
tablica = [[0] * kolumny for _ in range(wiersze)]
```

Częsty błąd — multiplikacja listy:

```
# ❌ BŁĄD — wszystkie wiersze to referencje do TEGO SAMEGO obiektu!
tablica = [[0] * 3] * 3
tablica[0][0] = 99
print(tablica) # [[99, 0, 0], [99, 0, 0], [99, 0, 0]] — zmienione wszystkie!

# ✅ POPRAWNIE — każdy wiersz jest osobnym obiektem
tablica = [[0] * 3 for _ in range(3)]
tablica[0][0] = 99
print(tablica) # [[99, 0, 0], [0, 0, 0], [0, 0, 0]] — zmieniony tylko pierwszy
```

16.2. Dostęp i iteracja

```
macierz = [[10, 20, 30], [40, 50, 60], [70, 80, 90]]

# Dostęp: [wiersz][kolumna]
print(macierz[0][0]) # 10
print(macierz[1][2]) # 60
print(macierz[2][1]) # 80

# Iteracja po elementach
for wiersz in macierz:
    for element in wiersz:
        print(f"{element:>4}", end="")
```

```

print()

# Iteracja z indeksami
for i in range(len(macierz)):
    for j in range(len(macierz[i])):
        print(f"[{i}][{j}]={macierz[i][j]}", end=" ")
    print()

```

16.3. Wypełnianie i wyświetlanie

```

import random

def stworz_tablice_losowa(wiersze, kolumny, minimum, maksimum):
    """Tworzy tablicę 2D wypełnioną losowymi liczbami."""
    return [[random.randint(minimum, maksimum) for _ in range(kolumny)]
            for _ in range(wiersze)]

def wyswietl_tablice(tablica, szerokosc=5):
    """Wyświetla tablicę 2D z wyrównaniem kolumn."""
    for i, wiersz in enumerate(tablica):
        print(f"Wiersz {i+1}: ", end="")
        for element in wiersz:
            print(f"{element:>{szerokosc}}", end="")
        print()

dane = stworz_tablice_losowa(3, 6, 1, 49)
wyswietl_tablice(dane)

```

16.4. Operacje na tablicach 2D

Kilka przydatnych operacji na tablicach dwuwymiarowych:

Suma elementów w wierszu i kolumnie:

```

macierz = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Suma każdego wiersza
for i, wiersz in enumerate(macierz):
    print(f"Wiersz {i}: suma = {sum(wiersz)}")

# Wiersz 0: suma = 6
# Wiersz 1: suma = 15
# Wiersz 2: suma = 24

# Suma każdej kolumny
kolumny = len(macierz[0])
for j in range(kolumny):
    suma_kol = 0
    for wiersz in macierz:
        suma_kol += wiersz[j]

```

```
print(f"Kolumna {j}: suma = {suma_kol}")

# Kolumna 0: suma = 12
# Kolumna 1: suma = 15
# Kolumna 2: suma = 18
```

Wyszukiwanie elementu w tablicy 2D:

```
def szukaj_2d(macierz, szukana):
    """Zwraca (wiersz, kolumna) lub None jeśli nie znaleziono."""
    for i in range(len(macierz)):
        for j in range(len(macierz[i])):
            if macierz[i][j] == szukana:
                return (i, j)
    return None

pozycja = szukaj_2d(macierz, 5)
if pozycja:
    print(f"Znaleziono na pozycji [{pozycja[0]}][{pozycja[1]}]")
# Znaleziono na pozycji [1][1]
```

Transpozycja macierzy (zamiana wierszy z kolumnami):

```
macierz = [
    [1, 2, 3],
    [4, 5, 6]
]

# Transpozycja
transpozycja = []
for j in range(len(macierz[0])):
    nowy_wiersz = []
    for i in range(len(macierz)):
        nowy_wiersz.append(macierz[i][j])
    transpozycja.append(nowy_wiersz)

print(transpozycja)
# [[1, 4], [2, 5], [3, 6]]

# Lub jednolinijkowo z zip:
transpozycja = [list(wiersz) for wiersz in zip(*macierz)]
```

17. Testowanie jednostkowe — unittest i pytest

17.1. Czym jest testowanie jednostkowe

Testowanie jednostkowe (unit testing) polega na automatycznym sprawdzaniu poprawności **pojedynczych** jednostek kodu (funkcji, metod, klas) w izolacji od reszty programu. Każdy test to oddzielna metoda, która:

1. **Przygotowuje** dane wejściowe (Arrange)
2. **Wywołuje** testowaną funkcję/metodę (Act)

3. Sprawdź, czy wynik jest zgodny z oczekiwanym — asercja (Assert)

Testy automatyczne pozwalają szybko wykryć, gdy zmiana w kodzie zepsuje istniejącą funkcjonalność. Zamiast ręcznie testować program po każdej modyfikacji, uruchamiamy zestaw testów, które w kilka sekund sprawdzą wszystkie kluczowe scenariusze.

17.2. Moduł `unittest` — podstawy

`unittest` jest wbudowany w Pythona — nie wymaga instalacji. Testy piszemy jako metody klasy dziedziczącej po `unittest.TestCase`. Każda metoda testowa musi zaczynać się od `test_`.

```
import unittest

# Funkcje do przetestowania
def dodaj(a, b):
    return a + b

def jest_parzysta(liczba):
    return liczba % 2 == 0

# Klasa testowa — dziedziczy po unittest.TestCase
class TestMatematyki(unittest.TestCase):

    def test_dodawanie_liczb_dodatnich(self):
        wynik = dodaj(2, 3)
        self.assertEqual(wynik, 5) # Czy wynik == 5?

    def test_dodawanie_z_zerem(self):
        self.assertEqual(dodaj(5, 0), 5)

    def test_dodawanie_liczb_ujemnych(self):
        self.assertEqual(dodaj(-2, -3), -5)

    def test_parzysta(self):
        self.assertTrue(jest_parzysta(4)) # Czy True?

    def test_nieparzysta(self):
        self.assertFalse(jest_parzysta(3)) # Czy False?

# Uruchomienie testów
if __name__ == "__main__":
    unittest.main()
```

Uruchomienie z terminala:

```
python -m unittest test_plik.py          # Uruchom testy z pliku
python -m unittest test_plik.py -v      # Tryb verbose (szczegółowy)
```

17.3. Asercje — kompletna lista

Asercje to metody sprawdzające warunki. Jeśli warunek nie jest spełniony, test jest oznaczany jako **FAIL**.

ASERCJA	SPRAWDZA	PRZYKŁAD
<code>assertEqual(a, b)</code>	<code>a == b</code>	<code>self.assertEqual(2+2, 4)</code>
<code>assertNotEqual(a, b)</code>	<code>a != b</code>	<code>self.assertNotEqual(2+2, 5)</code>
<code>assertTrue(x)</code>	<code>x is True</code>	<code>self.assertTrue(5 > 3)</code>
<code>assertFalse(x)</code>	<code>x is False</code>	<code>self.assertFalse(3 > 5)</code>
<code>assertIsNone(x)</code>	<code>x is None</code>	<code>self.assertIsNone(None)</code>
<code>assertIsNotNone(x)</code>	<code>x is not None</code>	<code>self.assertIsNotNone(42)</code>
<code>assertIn(a, b)</code>	<code>a in b</code>	<code>self.assertIn(3, [1,2,3])</code>
<code>assertNotIn(a, b)</code>	<code>a not in b</code>	<code>self.assertNotIn(5, [1,2,3])</code>
<code>assertGreater(a, b)</code>	<code>a > b</code>	<code>self.assertGreater(5, 3)</code>
<code>assertGreaterEqual(a, b)</code>	<code>a ≥ b</code>	<code>self.assertGreaterEqual(5, 5)</code>
<code>assertLess(a, b)</code>	<code>a < b</code>	<code>self.assertLess(3, 5)</code>
<code>assertLessEqual(a, b)</code>	<code>a ≤ b</code>	<code>self.assertLessEqual(3, 3)</code>
<code>assertAlmostEqual(a, b)</code>	<code>a ≈ b</code> (float)	<code>self.assertAlmostEqual(0.1+0.2, 0.3, places=5)</code>
<code>assertIsInstance(a, T)</code>	<code>isinstance(a, T)</code>	<code>self.assertIsInstance(42, int)</code>
<code>assertRaises(E)</code>	Czy rzuca wyjątek	<code>with self.assertRaises(ValueError):</code>

17.4. Struktura projektu z testami

Typowa organizacja plików:

```
projekt/
├─ modul.py          # Kod z testowanymi funkcjami/klasami
├─ main.py           # Program główny
└─ test_modul.py     # Plik z testami
```

modul.py:

```
def szyfruj(tekst, klucz):
    wynik = ""
    for znak in tekst:
        if 'a' ≤ znak ≤ 'z':
            numer = ord(znak) - ord('a')
            nowy = (numer + klucz) % 26
            wynik += chr(nowy + ord('a'))
        else:
            wynik += znak
    return wynik
```

test_modul.py:

```
import unittest
from modul import szyfruj
```

```

class TestSzyfrowania(unittest.TestCase):

    def test_klucz_dodatni(self):
        """Klucz dodatni, wartość mniejsza niż 26."""
        self.assertEqual(szyfruj("abc", 3), "def")

    def test_zawijanie(self):
        """Litera wychodzi poza alfabet – zawijanie."""
        self.assertEqual(szyfruj("xyz", 3), "abc")

    def test_odszyfrowanie(self):
        """Klucz ujemny – odszyfrowanie."""
        self.assertEqual(szyfruj("def", -3), "abc")

    def test_klucz_wiekszy_niz_26(self):
        """Klucz większy niż długość alfabetu."""
        self.assertEqual(szyfruj("abc", 29), "def")

    def test_spacja(self):
        """Spacja pozostaje bez zmian."""
        self.assertEqual(szyfruj("ab cd", 2), "cd ef")

    def test_klucz_zero(self):
        """Klucz 0 – tekst bez zmian."""
        self.assertEqual(szyfruj("abc", 0), "abc")

    def test_pusty_tekst(self):
        """Pusty łańcuch znaków."""
        self.assertEqual(szyfruj("", 5), "")

if __name__ == "__main__":
    unittest.main()

```

17.5. Nazewnictwo testów

Dobre nazwy testów są kluczowe — powinny opisywać **co jest testowane** i **jaki jest oczekiwany wynik**:

```

# ✅ Dobra konwencja: test_co_testujemy_jaki_scenariusz
def test_szyfruj_klucz_dodatni_przesuwa_litery(self): ...
def test_szyfruj_spacja_pozostaje_bez_zmian(self): ...
def test_rzut_wartosc_w_zakresie_1_do_6(self): ...
def test_blokowanie_kost_nie_zmienia_wartosci(self): ...

# ❌ Zła konwencja – niejasna
def test_1(self): ...
def test_cos(self): ...

```

17.6. `setUp` i `tearDown` — przygotowanie i sprzątnie

Metoda `setUp` wykonuje się **przed każdym** testem, a `tearDown` **po każdym**. Przydatne, gdy wiele testów potrzebuje tych samych obiektów:

```

class TestKalkulatora(unittest.TestCase):

    def setUp(self):
        """Wywoływana PRZED każdym testem – przygotowanie danych."""
        self.dane = [3, 1, 4, 1, 5, 9, 2, 6]
        self.puste = []

    def tearDown(self):
        """Wywoływana PO każdym teście – sprzątanie."""
        pass # Np. zamknięcie pliku, usunięcie danych tymczasowych

    def test_dlugosc(self):
        self.assertEqual(len(self.dane), 8)

    def test_pusty(self):
        self.assertEqual(len(self.puste), 0)

```

17.7. Testowanie klas i obiektów

```

import unittest

class Licznik:
    def __init__(self, poczatkowa=0):
        self.__wartosc = poczatkowa

    def zwieksz(self):
        self.__wartosc += 1

    def pobierz(self):
        return self.__wartosc

    def resetuj(self):
        self.__wartosc = 0

class TestLicznika(unittest.TestCase):

    def setUp(self):
        self.licznik = Licznik()

    def test_wartosc_poczatkowa_domyslna(self):
        self.assertEqual(self.licznik.pobierz(), 0)

    def test_wartosc_poczatkowa_niestandardowa(self):
        l = Licznik(10)
        self.assertEqual(l.pobierz(), 10)

    def test_zwiekszenie(self):
        self.licznik.zwieksz()
        self.assertEqual(self.licznik.pobierz(), 1)

```

```

def test_wielokrotne_zwiekszenie(self):
    for _ in range(5):
        self.licznik.zwieksz()
    self.assertEqual(self.licznik.pobierz(), 5)

def test_resetowanie(self):
    self.licznik.zwieksz()
    self.licznik.zwieksz()
    self.licznik.resetuj()
    self.assertEqual(self.licznik.pobierz(), 0)

if __name__ == "__main__":
    unittest.main()

```

17.8. Testowanie wartości losowych

Testowanie kodu korzystającego z generatora losowego wymaga sprawdzania **zakresów**, nie konkretnych wartości:

```

import unittest
import random

class TestLosowan(unittest.TestCase):

    def test_zakres_1_do_6(self):
        """Czy wylosowana wartość mieści się w zakresie 1-6."""
        for _ in range(1000): # Wielokrotne sprawdzenie
            wartosc = random.randint(1, 6)
            self.assertGreaterEqual(wartosc, 1)
            self.assertLessEqual(wartosc, 6)

    def test_sample_unikalne(self):
        """Czy losowanie sample daje unikalne wartości."""
        zestaw = random.sample(range(1, 50), 6)
        self.assertEqual(len(zestaw), len(set(zestaw))) # Brak duplikatów

```

17.9. `pytest` — prostsza alternatywa

`pytest` to popularna alternatywa dla `unittest` z prostszą składnią — nie wymaga klas ani specjalnych metod, zamiast `self.assertEqual()` używamy zwykłego `assert`:

Instalacja:

```
pip install pytest
```

Pisanie testów:

```

# test_funkcje.py
from modul import dodaj, szyfruj

def test_dodawanie():
    assert dodaj(2, 3) == 5

```

```
def test_dodawanie_ujemnych():
    assert dodaj(-1, -2) == -3

def test_szyfruj_klucz_3():
    assert szyfruj("abc", 3) == "def"

def test_szyfruj_zawijanie():
    assert szyfruj("xyz", 3) == "abc"
```

Uruchomienie:

```
pytest test_funkcje.py           # Uruchom testy
pytest test_funkcje.py -v        # Tryb verbose
pytest test_funkcje.py -v --tb=short # Krótki traceback
```

Porównanie assert w pytest:

Pytest automatycznie rozpisuje szczegóły nieudanej asercji — nie trzeba używać specjalnych metod:

```
# W pytest - jasny komunikat o błędzie
def test_lista():
    wynik = [1, 2, 3]
    assert len(wynik) == 3
    assert 2 in wynik
    assert wynik[0] == 1

def test_slownik():
    dane = {"imie": "Jan", "wiek": 25}
    assert "imie" in dane
    assert dane["wiek"] > 18
```

Parametryzacja testów (pytest):

Pytest pozwala na eleganckie uruchamianie tego samego testu z różnymi danymi wejściowymi:

```
import pytest

@pytest.mark.parametrize("wejście, oczekiwany", [
    ("abc", "def"),
    ("xyz", "abc"),
    ("", ""),
])

def test_szyfruj(wejście, oczekiwany):
    assert szyfruj(wejście, 3) == oczekiwany
```

Porównanie `unittest` vs `pytest`:

CECHA	UNITTEST	PYTEST
Instalacja	Wbudowany w Pythona	<code>pip install pytest</code>
Klasy testowe	Wymagane (<code>TestCase</code>)	Opcjonalne
Asercje	<code>self.assertEqual(a, b)</code>	<code>assert a == b</code>
Uruchomienie	<code>python -m unittest</code>	<code>pytest</code>
Składnia	Bardziej formalna	Prostsza, krótsza

17.10. Podsumowanie — uruchamianie testów

```
# unittest
python -m unittest test_plik.py           # Cały plik
python -m unittest test_plik.py -v        # Szczegółowo
python -m unittest test_plik.TestKlasa    # Konkretna klasa
python -m unittest test_plik.TestKlasa.test_metoda # Konkretny test

# pytest
pytest                                    # Wszystko w katalogu
pytest test_plik.py                       # Konkretny plik
pytest test_plik.py -v                    # Szczegółowo
pytest test_plik.py::test_nazwa           # Konkretna funkcja
pytest test_plik.py::TestKlasa::test_metoda # Konkretna metoda
```

18. Przydatne wzorce i idiomy Pythona

Na zakończenie — zbiór przydatnych wzorców i idiomów, które pojawiają się regularnie w programowaniu w Pythonie. Znajomość tych wzorców pozwala pisać kod bardziej idiomatyczny ("pythoniczny"), czyli taki, który jest naturalny i czytelny dla innych programistów Pythona.

18.1. Zamiana wartości dwóch zmiennych

Python pozwala na zamianę bez zmiennej tymczasowej:

```
a, b = 5, 10
a, b = b, a    # Teraz a=10, b=5

# W kontekście listy — zamiana elementów
tablica = [1, 2, 3, 4, 5]
tablica[0], tablica[4] = tablica[4], tablica[0]
# [5, 2, 3, 4, 1]
```

18.2. `if __name__ == "__main__":`

Ten warunek sprawdza, czy plik jest uruchamiany bezpośrednio (nie importowany). Kod wewnątrz tego bloku wykona się tylko przy bezpośrednim uruchomieniu skryptu:

```
def moja_funkcja():
    return 42
```

```
class MojaKlasa:
    pass

# Ten kod wykona się TYLKO przy: python plik.py
# NIE wykona się przy: from plik import MojaKlasa

if __name__ == "__main__":
    print("Program uruchomiony bezpośrednio")
    wynik = moja_funkcja()
    print(f"Wynik: {wynik}")
```

To jest szczególnie ważne przy testach jednostkowych — moduł z testowanym kodem musi pozwalać na importowanie bez uruchamiania programu głównego.

18.3. Rozpakowywanie z `*` (operatorem gwiazdki)

Gwiazdka `*` pozwala na rozpakowywanie dowolnej liczby elementów:

```
# Rozpakowywanie z resztą
lista = [1, 2, 3, 4, 5]
pierwszy, *srodek, ostatni = lista
print(pierwszy) # 1
print(srodek)   # [2, 3, 4]
print(ostatni)  # 5

# Połączenie dwóch list
a = [1, 2, 3]
b = [4, 5, 6]
polaczone = [*a, *b] # [1, 2, 3, 4, 5, 6]
```

18.4. Dictionary comprehension

Analogicznie do list comprehension, ale tworzy słowniki:

```
# Kwadraty liczb jako słownik
kwadraty = {x: x**2 for x in range(1, 6)}
print(kwadraty) # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# Konwersja dwóch list na słownik
klucze = ["a", "b", "c"]
wartosci = [1, 2, 3]
słownik = {k: v for k, v in zip(klucze, wartosci)}
print(słownik) # {'a': 1, 'b': 2, 'c': 3}
```

18.5. Formatowanie wypisywania tabel w konsoli

Przy wyświetlaniu danych tabelarycznych warto zadbać o wyrównanie kolumn:

```
# Wyrównanie kolumn za pomocą f-stringów
dane = [
    ("Jan", 25, "Warszawa"),
    ("Anna", 30, "Kraków"),
    ("Piotr Wiśniewski", 22, "Gdańsk")
]
```



```

]

print(f'Imię:<20>{>5}  {<15>}')
print("-" * 42)
for imie, wiek, miasto in dane:
    print(f'{imie:<20>{>5}  {miasto:<15>}')

# Wynik:
# Imię           Wiek  Miasto
# -----
# Jan            25   Warszawa
# Anna           30   Kraków
# Piotr Wiśniewski 22   Gdańsk

```

18.6. Podkreślnik `_` — konwencje

Podkreślnik ma kilka konwencji w Pythonie:

UŻYCIE	ZNACZENIE	PRZYKŁAD
<code>_</code>	Zmienna, której nie używamy	<code>for _ in range(5):</code>
<code>_pole</code>	Pole chronione (konwencja)	<code>self._saldo</code>
<code>__pole</code>	Pole prywatne (name mangling)	<code>self.__hasło</code>
<code>__metoda__</code>	Metoda specjalna Pythona	<code>__init__</code> , <code>__str__</code>

18.7. Najczęstsze pułapki i błędy w Pythonie

Poniżej zebrane są najczęstsze pułapki, na które warto uważać:

1. Modyfikowalna wartość domyślna parametru:

```

# ❌ PUŁAPKA — lista jest współdzielona między wywołaniami!
def dodaj_element(element, lista=[]):
    lista.append(element)
    return lista

print(dodaj_element(1)) # [1]
print(dodaj_element(2)) # [1, 2] — nie [2]!
print(dodaj_element(3)) # [1, 2, 3] — nie [3]!

# ✅ POPRAWNIE — używamy None jako domyślnej wartości
def dodaj_element(element, lista=None):
    if lista is None:
        lista = []
    lista.append(element)
    return lista

```

2. Porównanie `=` vs `is`:

```

a = [1, 2, 3]
b = [1, 2, 3]

```

```
print(a == b)    # True - te same WARTOŚCI
print(a is b)    # False - to RÓŻNE obiekty w pamięci

# Dla None ZAWSZE używaj 'is'
x = None
if x is None:    # ✅ Poprawnie
    print("None")
```

3. Kopiowanie listy — referencja vs kopia:

```
oryg = [1, 2, 3]

# ❌ REFERENCJA - obie zmienne wskazują na TĘ SAMĄ listę
kopia = oryg
kopia.append(4)
print(oryg)    # [1, 2, 3, 4] - oryginał też się zmienił!

# ✅ KOPIA - nowa, niezależna lista
kopia = oryg.copy()    # lub: kopia = oryg[:]
kopia.append(5)
print(oryg)    # [1, 2, 3, 4] - oryginał bez zmian
```

4. Dzielenie zwraca float:

```
wynik = 10 / 2
print(wynik)    # 5.0 (float, nie int!)
print(type(wynik))    # <class 'float'>

# Gdy potrzebujesz int:
wynik = 10 // 2    # 5 (int)
```

5. Indeksowanie od 0:

```
lista = ["A", "B", "C"]

# ❌ Częsty błąd - lista[3] nie istnieje
# print(lista[3])    # IndexError!

# ✅ Ostatni element: lista[-1] lub lista[len(lista)-1]
print(lista[-1])    # "C"
print(lista[2])    # "C"
```

18.8. Wzorzec menu programu (pętla główna)

Najczęstszy wzorzec programu konsolowego — nieskończona pętla wyświetlająca menu opcji, pobierająca wybór użytkownika i wykonująca odpowiednią akcję. Program działa, dopóki użytkownik nie wybierze opcji wyjścia.

```
def menu():
    """Wyświetla menu główne i zwraca wybór użytkownika."""
    print("\n" + "=" * 30)
```

```

print("    MENU GŁÓWNE")
print("=" * 30)
print("1. Dodaj element")
print("2. Wyświetl elementy")
print("3. Wyszukaj element")
print("4. Sortuj elementy")
print("0. Wyjście")
print("=" * 30)
return input("Wybierz opcję: ")

def dodaj(lista):
    element = input("Podaj element: ")
    lista.append(element)
    print(f"Dodano: {element}")

def wyswietl(lista):
    if not lista:
        print("Lista jest pusta.")
    else:
        for i, el in enumerate(lista, 1):
            print(f"{i}. {el}")

def szukaj(lista):
    szukany = input("Podaj szukany element: ")
    if szukany in lista:
        print(f"Znaleziono na pozycji {lista.index(szukany) + 1}")
    else:
        print("Nie znaleziono.")

# PETLA GLOWNA PROGRAMU
dane = []

while True:
    wybor = menu()

    if wybor == "1":
        dodaj(dane)
    elif wybor == "2":
        wyswietl(dane)
    elif wybor == "3":
        szukaj(dane)
    elif wybor == "4":
        dane.sort()
        print("Posortowano.")
    elif wybor == "0":
        print("Do widzenia!")
        break
    else:
        print("Nieprawidłowa opcja, spróbuj ponownie.")

```

Ten wzorzec jest fundamentem wielu programów konsolowych. Kluczowe elementy:

- `while True` — nieskończona pętla
- `break` — wyjście z pętli (opcja 0)
- **Oddzielne funkcje** — każda opcja menu to osobna funkcja
- `elif` — obsługa wielu opcji

18.9. Sortowanie obiektów — parametr `key`

Sortowanie list zawierających obiekty (instancje klas) wymaga wskazania, po jakim polu (atrybucie) chcemy sortować. Do tego służy parametr `key` w funkcjach `sorted()` i `.sort()`.

```
class Uczeń:
    def __init__(self, imie, wiek, srednia):
        self.imie = imie
        self.wiek = wiek
        self.srednia = srednia

    def __repr__(self):
        return f"Uczen('{self.imie}', {self.wiek}, {self.srednia})"

uczniowie = [
    Uczen("Jan", 17, 4.2),
    Uczen("Anna", 16, 5.0),
    Uczen("Piotr", 18, 3.5),
    Uczen("Maria", 17, 4.8)
]

# Sortowanie po imieniu (alfabetycznie)
wg_imienia = sorted(uczniowie, key=lambda u: u.imie)
# [Anna, Jan, Maria, Piotr]

# Sortowanie po średniej (malejąco — najlepsi pierwsi)
wg_sredniej = sorted(uczniowie, key=lambda u: u.srednia, reverse=True)
# [Anna(5.0), Maria(4.8), Jan(4.2), Piotr(3.5)]

# Sortowanie po wielu kryteriach — krotka w key
# Najpierw po wieku (rosnaco), potem po imieniu (rosnaco)
wg_wielu = sorted(uczniowie, key=lambda u: (u.wiek, u.imie))

# Sortowanie NA MIEJSCU (modyfikuje oryginalną listę)
uczniowie.sort(key=lambda u: u.srednia)
```

Aby wyświetlić posortowaną listę:

```
for u in sorted(uczniowie, key=lambda u: u.srednia, reverse=True):
    print(f"{u.imie}: średnia {u.srednia}")

# Anna: średnia 5.0
# Maria: średnia 4.8
# Jan: średnia 4.2
# Piotr: srednia 3.5
```

Zamiast `lambda` można użyć `attrgetter` z modułu `operator`. Robi to samo, ale jest nieco szybszy:

```
from operator import attrgetter

wg_sredniej = sorted(uczniowie, key=attrgetter("srednia"), reverse=True)
wg_wielu = sorted(uczniowie, key=attrgetter("wiek", "imie"))
```

18.10. Generatory i `yield`

Generator to specjalny rodzaj funkcji, która **nie zwraca wszystkich wartości naraz** (jak lista), tylko produkuje je **po jednej na żądanie**. Dzięki temu zużywa minimalną pamięć nawet przy miliardach elementów.

Różnica między zwykłą funkcją a generatorem to słowo kluczowe `yield` zamiast `return`.

Podstawowy przykład — generator vs lista:

```
# Zwykła funkcja — tworzy całą listę w pamięci
def liczby_lista(n):
    wynik = []
    for i in range(n):
        wynik.append(i * i)
    return wynik

# Generator — produkuje wartości po jednej
def liczby_generator(n):
    for i in range(n):
        yield i * i  # zwróć jedną wartość i "zamróz" funkcję

# Użycie jest identyczne w pętli for
for kwadrat in liczby_generator(5):
    print(kwadrat)  # 0, 1, 4, 9, 16

# Można też zamienić generator na listę
print(list(liczby_generator(5)))  # [0, 1, 4, 9, 16]
```

Jak działa `yield` — "zamrażanie" funkcji:

```
def przykladowy_generator():
    print("Krok 1")
    yield 10  # zatrzymaj się, zwróć 10
    print("Krok 2")
    yield 20  # zatrzymaj się, zwróć 20
    print("Krok 3")
    yield 30  # zatrzymaj się, zwróć 30
    print("Koniec")

g = przykladowy_generator()  # tworzy obiekt generatora (nic nie wykonuje!)
```

```
print(next(g))    # "Krok 1", potem zwraca 10
print(next(g))    # "Krok 2", potem zwraca 20
print(next(g))    # "Krok 3", potem zwraca 30
# next(g) teraz rzuciłoby StopIteration (generator wyczerpany)
```

Praktyczny przykład — generator parzystych liczb:

```
def parzyste(limit):
    """Generator liczb parzystych od 0 do limit."""
    liczba = 0
    while liczba <= limit:
        yield liczba
        liczba += 2

# Wypisz parzyste do 10
for p in parzyste(10):
    print(p)    # 0, 2, 4, 6, 8, 10

# Pobierz tylko pierwszą parzystą
g = parzyste(100)
print(next(g))  # 0
print(next(g))  # 2
print(next(g))  # 4
```

Generator expression — jednoliniowy generator (jak list comprehension):

```
# List comprehension — tworzy całą listę w pamięci
kwadraty_lista = [x**2 for x in range(10)]

# Generator expression — produkuje wartości leniwie (nawiasy okrągłe!)
kwadraty_gen = (x**2 for x in range(10))

# Działanie takie same:
for k in kwadraty_gen:
    print(k)

# Generator expression bezpośrednio w funkcji:
print(sum(x**2 for x in range(10))) # 285 (bez tworzenia listy)
print(max(x**2 for x in range(10))) # 81
```

Kiedy używać generatorów:

SYTUACJA	ROZWIĄZANIE
Duża kolekcja, używasz każdego elementu raz	Generator
Mała kolekcja lub potrzebujesz dostępu losowego	Lista
Odczyt dużego pliku linia po linii	Generator
Przetwarzanie strumienia danych	Generator
Wielokrotna iteracja przez tę samą kolekcję	Lista

```
# Przykład: odczyt dużego pliku generatorem (bez wczytywania całego do pamięci)
def czytaj_linie(nazwa_pliku):
    with open(nazwa_pliku, encoding="utf-8") as f:
        for linia in f:
            yield linia.strip() # zwraca linie po linii

# Użycie:
# for linia in czytaj_linie("duzy_plik.txt"):
#     print(linia)
```

Porównanie: `return` vs `yield` :

CECHA	<code>RETURN</code> (ZWYKŁA FUNKCJA)	<code>YIELD</code> (GENERATOR)
Kiedy zwraca	Po wykonaniu całej funkcji	Po każdym <code>yield</code>
Pamięć	Cały wynik naraz	Jedna wartość na raz
Można iterować	Tak (jeśli zwróci listę)	Tak (zawsze)
Wznawia wykonanie	Nie (zaczyna od nowa)	Tak (kontynuuje)
Typ zwracanej wartości	Dowolny	Obiekt <code>generator</code>

19. Najczęstsze błędy (Wyjątki i błędy konsolowe)

W tym rozdziale opisano rzeczywiste błędy wykonania i wyjątki wyrzucane przez interpreter Pythona (widoczne w konsoli), w odróżnieniu od logicznych błędów projektowych.

19.1. SyntaxError

Pojawia się, gdy kod niezgodny jest ze składnią języka Python (np. brakujący dwukropek, niezamknięty nawias).

❌ Kod powodujący błąd:

```
if True
    print('Prawda')
```

Komunikat w konsoli:

```
SyntaxError: expected ':'
```

✅ Poprawny kod:

```
if True:
    print('Prawda')
```

Należy dodać brakujący dwukropek na końcu instrukcji warunkowej.

19.2. IndentationError

Występuje, gdy wcięcia w kodzie są nieprawidłowe lub niespójne.

✗ Kod powodujący błąd:

```
def funkcja():
    print('Witaj')
```

Komunikat w konsoli:

```
IndentationError: expected an indented block
```

✓ Poprawny kod:

```
def funkcja():
    print('Witaj')
```

W Pythonie bloki kodu definiuje się za pomocą wcięć. Należy dodać wcięcie (np. 4 spacje).

19.3. NameError

Wyjątek zgłaszany, gdy program próbuje użyć zmiennej lub funkcji, która nie została wcześniej zdefiniowana.

✗ Kod powodujący błąd:

```
wynik = x + 5
```

Komunikat w konsoli:

```
NameError: name 'x' is not defined
```

✓ Poprawny kod:

```
x = 10
wynik = x + 5
```

Przed użyciem zmiennej `x`, należy przypisać jej wartość.

19.4. TypeError

Występuje, gdy operacja lub funkcja jest stosowana do obiektu niewłaściwego typu.

✗ Kod powodujący błąd:


```
wynik = 'Wiek: ' + 25
```

Komunikat w konsoli:

```
TypeError: can only concatenate str (not "int") to str
```

✅ **Poprawny kod:**

```
wynik = 'Wiek: ' + str(25)
```

Nie można łączyć napisów (str) z liczbami (int). Należy najpierw rzutować liczbę na napis używając `str()`.

19.5. ValueError

Zgłaszany, gdy funkcja otrzymuje argument o poprawnym typie, ale niewłaściwej wartości.

❌ **Kod powodujący błąd:**

```
liczba = int('abc')
```

Komunikat w konsoli:

```
ValueError: invalid literal for int() with base 10: 'abc'
```

✅ **Poprawny kod:**

```
liczba = int('123')
```

Funkcja `int()` oczekuje napisu, który można zinterpretować jako liczbę całkowitą.

19.6. IndexError

Pojawia się przy próbie dostępu do elementu listy (lub krotki) poza jej zakresem.

❌ **Kod powodujący błąd:**

```
lista = [1, 2, 3]
element = lista[5]
```

Komunikat w konsoli:

```
IndexError: list index out of range
```

✅ **Poprawny kod:**

```
lista = [1, 2, 3]
if len(lista) > 5:
    element = lista[5]
```

Należy upewnić się, że podany indeks mieści się w rozmiarze listy (od 0 do `len(lista) - 1`).

19.7. KeyError

Zgłaszany podczas próby dostępu do wartości w słowniku przy użyciu klucza, który nie istnieje.

✗ Kod powodujący błąd:

```
słownik = {'a': 1}
wartosc = słownik['b']
```

Komunikat w konsoli:

```
KeyError: 'b'
```

✓ Poprawny kod:

```
słownik = {'a': 1}
wartosc = słownik.get('b', 0)
```

Aby uniknąć błędu, można użyć metody `get()`, która pozwala określić domyślną wartość, jeśli klucz nie istnieje.

19.8. AttributeError

Pojawia się, gdy próbuje się wywołać nieistniejącą metodę lub pole na obiekcie.

✗ Kod powodujący błąd:

```
napis = 'test'
napis.append('!')
```

Komunikat w konsoli:

```
AttributeError: 'str' object has no attribute 'append'
```

✓ Poprawny kod:

```
napis = 'test'
napis += '!'
```

Klasa `str` (napis) nie posiada metody `append()`. Do łączenia napisów używa się operatora `+`.

19.9. ImportError

Występuje, gdy moduł został znaleziony, ale nie udało się zaimportować z niego określonej nazwy.

❌ Kod powodujący błąd:

```
from math import potega
```

Komunikat w konsoli:

```
ImportError: cannot import name 'potega' from 'math'
```

✅ Poprawny kod:

```
from math import pow
```

Należy sprawdzić poprawność nazwy importowanej funkcji lub klasy. Funkcja potęgowania to `pow`.

19.10. ModuleNotFoundError

Zgłaszany, gdy próba importu modułu kończy się niepowodzeniem, ponieważ plik lub biblioteka nie istnieje.

❌ Kod powodujący błąd:

```
import nieznana_biblioteka
```

Komunikat w konsoli:

```
ModuleNotFoundError: No module named 'nieznana_biblioteka'
```

✅ Poprawny kod:

```
# Pip install wymaganej biblioteki przed importem  
import requests
```

Upewnij się, że biblioteka została zainstalowana (np. za pomocą `pip install`) i czy nazwa jest poprawna.

19.11. ZeroDivisionError

Pojawia się, gdy próbuje się podzielić liczbę przez zero.

❌ Kod powodujący błąd:

```
wynik = 10 / 0
```

Komunikat w konsoli:

```
ZeroDivisionError: division by zero
```

✅ Poprawny kod:

```
dzielnik = 0
if dzielnik != 0:
    wynik = 10 / dzielnik
else:
    wynik = None
```

Zawsze należy sprawdzać, czy mianownik przed wykonaniem operacji dzielenia nie wynosi 0.

19.12. FileNotFoundError

Zgłaszany w przypadku próby otwarcia pliku, który nie istnieje we wskazanym miejscu.

❌ Kod powodujący błąd:

```
with open('brakujacy_plik.txt', 'r') as f:
    dane = f.read()
```

Komunikat w konsoli:

```
FileNotFoundError: [Errno 2] No such file or directory: 'brakujacy_plik.txt'
```

✅ Poprawny kod:

```
import os
if os.path.exists('moj_plik.txt'):
    with open('moj_plik.txt', 'r') as f:
        dane = f.read()
```

Należy upewnić się, że ścieżka i nazwa pliku są poprawne, lub użyć modułu `os` do sprawdzenia, czy plik istnieje.

19.13. RecursionError

Występuje, gdy funkcja wywołuje samą siebie zbyt wiele razy, co powoduje przepełnienie limitu głębokości rekurencji.

❌ Kod powodujący błąd:

```
def rekurencja():
    return rekurencja()
rekurencja()
```

Komunikat w konsoli:

```
RecursionError: maximum recursion depth exceeded
```

✅ Poprawny kod:

```
def rekurencja(n):
    if n <= 0:
        return 0
```

```
    return 1 + rekurencja(n - 1)

rekurencja(10)
```

Należy upewnić się, że funkcja rekurencyjna posiada tzw. warunek stopu (bazowy), który przerywa ciąg wywołań.

19.14. StopIteration

Zgłaszany przez funkcję `next()` lub generator, sygnalizujący że nie ma więcej elementów w obiekcie iterowalnym.

❌ Kod powodujący błąd:

```
it = iter([1, 2])
next(it)
next(it)
next(it)
```

Komunikat w konsoli:

```
StopIteration
```

✅ Poprawny kod:

```
it = iter([1, 2])
while True:
    try:
        element = next(it)
    except StopIteration:
        break
```

Należy obsługiwać wyjątek `StopIteration` w blokach iterujących (lub korzystać z pętli `for`, która obsługuje go automatycznie).

19.15. UnboundLocalError

Występuje w ciele funkcji, gdy próbujemy odwołać się do zmiennej lokalnej przed przypisaniem jej wartości.

❌ Kod powodujący błąd:

```
x = 10
def dodaj():
    x += 1
dodaj()
```

Komunikat w konsoli:

```
UnboundLocalError: local variable 'x' referenced before assignment
```

✅ Poprawny kod:

```
x = 10
def dodaj():
```

```
global x  
x += 1  
dodaj()
```

Jeśli chcemy zmodyfikować zmienną globalną, należy użyć słowa kluczowego `global`. W przeciwnym razie trzeba zadeklarować nową zmienną lokalną.

19.16. NotImplementedError

Wyjątek często zgłaszany w metodach abstrakcyjnych, informujący że dana funkcjonalność nie została jeszcze zaimplementowana.

❌ Kod powodujący błąd:

```
class Figura:  
    def pole(self):  
        raise NotImplementedError()  
  
f = Figura()  
f.pole()
```

Komunikat w konsoli:

```
NotImplementedError
```

✅ Poprawny kod:

```
class Kwadrat(Figura):  
    def pole(self):  
        return 10 * 10  
  
k = Kwadrat()  
print(k.pole())
```

Należy utworzyć klasę pochodną, która nadpisuje (implementuje) metodę abstrakcyjną.

19.17. TabError

Zgłaszany w przypadku, gdy w bloku kodu stosowane są jednocześnie spacje i tabulatory niezgodnie z formatowaniem Pythona.

❌ Kod powodujący błąd:

```
def test():  
    print('linia ze spacjami')  
    print('linia z tabulatorem')
```

Komunikat w konsoli:

```
TabError: inconsistent use of tabs and spaces in indentation
```

✅ Poprawny kod:

```
def test():  
    print('linia ze spacjami')  
    print('linia ze spacjami')
```

W Pythonie powinno się korzystać tylko z jednego rodzaju wcięć (zalecane są 4 spacje) w całym pliku.

19.18. PermissionError

Występuje, gdy program próbuje uruchomić operację bez wystarczających praw dostępu (np. otworenie pliku zablokowanego tylko do odczytu).

❌ Kod powodujący błąd:

```
with open('/root/hasla.txt', 'w') as f:  
    f.write('sekret')
```

Komunikat w konsoli:

```
PermissionError: [Errno 13] Permission denied: '/root/hasla.txt'
```

✅ Poprawny kod:

```
# Uruchom jako root lub zmień ścieżkę na katalog z prawem zapisu  
with open('moje_dane.txt', 'w') as f:  
    f.write('dane')
```

Należy zmienić prawa pliku/katalogu w systemie operacyjnym lub upewnić się, że ścieżka odnosi się do dozwolonego obszaru.

19.19. OverflowError

Pojawia się, gdy wynik operacji arytmetycznej przekracza zakres dopuszczalny dla danego typu danych (zwykle zmiennoprzecinkowych float).

❌ Kod powodujący błąd:

```
import math  
math.exp(1000)
```

Komunikat w konsoli:

```
OverflowError: math range error
```

✅ Poprawny kod:

```
import math  
# Zastosowanie odpowiednich skali logarytmicznych lub mniejszych potęg  
math.exp(100)
```

Należy kontrolować wielkość przetwarzanych wartości liczbowych lub używać wyspecjalizowanych bibliotek do dużych liczb.

19.20. UnicodeDecodeError

Pojawia się, gdy proces dekodowania bajtów do ciągu znaków napotka dane, które nie pasują do wybranego kodowania.

✗ Kod powodujący błąd:

```
with open('plik_utf8.txt', 'r', encoding='ascii') as f:  
    tekst = f.read()
```

Komunikat w konsoli:

```
UnicodeDecodeError: 'ascii' codec can't decode byte 0xc3 in position 10
```

✓ Poprawny kod:

```
with open('plik_utf8.txt', 'r', encoding='utf-8') as f:  
    tekst = f.read()
```

Zawsze podawaj odpowiednie kodowanie (najczęściej `utf-8`) podczas pracy z plikami i strumieniami tekstowymi.